

Project Report
on
PREDICTIVE MAINTENANCE SYSTEM FOR
ELECTRICAL MACHINES USING MACHINE
LEARNING

Submitted as partial fulfillment for the award of
BACHELOR OF TECHNOLOGY
DEGREE

Session 2025-26
in
Electrical And Computer Engineering

By

LALIMA SINGH
Roll Number: 2200320230025
TANIYA VISHNOI
Roll Number: 2200320230057
NISHANT KUMAR
Roll Number: 2200320230029
NITIN JHA
Roll Number: 2200320230030

Under the guidance of
Ms. AVNEET KAUR
Assistant Professor

DEPARTMENT OF ELECTRICAL & COMPUTER
ENGINEERING
ABES ENGINEERING COLLEGE, GHAZIABAD



AFFILIATED TO
DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, U.P., LUCKNOW

Student's Declaration

We hereby declare that the work being presented in this report entitled "**PREDICTIVE MAINTENANCE SYSTEM FOR ELECTRICAL MACHINES USING MACHINE LEARNING**" is an authentic record of our own work carried out under the supervision of **Ms. Avneet Kaur, Assistant Professor, Department of Electrical & Computer Engineering.**

The matter embodied in this report has not been submitted by us for the award of any other degree.

Date: 24th April 2025

Signature of Student
(Name: Lalima Singh)
(Roll No.: 2200320230025)
Department: ELCE

Signature of Student
(Name: Taniya Vishnoi)
(Roll No.: 2200320230057)
Department: ELCE

Signature of Student
(Name: Nishant Kumar)
(Roll No.: 2200320230029)
Department: ELCE

Signature of Student
(Name: Nitin Jha)
(Roll No.: 2200320230030)
Department: ELCE

This is to certify that the above statement made by the candidate(s) is correct to the best of my knowledge.

Signature of HOD
Name: Ms. Pragati Shrivastava Deb
(Head of Department)
ELCE

Signature of Supervisor
Name: Ms. Avneet Kaur
(Assistant Professor)
ELCE

Date: 24th April 2026

Acknowledgement

We would like to express our sincere gratitude to all those who have contributed to the successful completion of this project.

First and foremost, we extend our heartfelt thanks to our project guide Ms. Avneet Kaur, Assistant Professor, Department of Electrical & Computer Engineering, for her invaluable guidance, constant encouragement, and insightful suggestions throughout the project. Her expertise in machine learning applications and industrial automation provided us with the direction needed to complete this project successfully.

We are deeply grateful to Ms. Pragati Shrivastava Deb, Head of the Department of Electrical & Computer Engineering, for providing us with the necessary facilities and resources. We also thank all the faculty members of the department for their continuous support and constructive feedback during various project evaluation stages.

We acknowledge Dr. A.P.J. Abdul Kalam Technical University, Lucknow and ABES Engineering College, Ghaziabad for providing us with this opportunity to undertake this project as part of our B.Tech curriculum.

We appreciate the support from our peers and colleagues who provided valuable insights during brainstorming sessions. Finally, we express our profound gratitude to our families for their unwavering support, patience, and encouragement throughout our academic journey.

Signature of Student
(Name: Lalima Singh)
(Roll No.: 2200320230025)
2200320230057)

Signature of Student
(Name: Taniya Vishnoi)
(Roll No.:

Signature of Student
(Name: Nishant Kumar)
(Roll No.: 2200320230029)
2200320230030)

Signature of Student
(Name: Nitin Jha)
(Roll No.:

Table of Contents

<u>S.No.</u>	<u>Contents</u>	<u>Page No.</u>
	Student's Declaration	i
	Acknowledgment	ii
	List of figures	iii
	List of Symbols, Abbreviations, Nomenclature	iv
	Abstract	v
<u>Chapter-1:</u>	<u>Introduction</u>	<u>1-4</u>
1.1:	Background and Motivation	1
1.2:	Problem Statement	1
1.3:	Significance of Predictive Maintenance	2
1.4:	Objectives of the Study	3
1.5:	Organization of the Report	4
<u>Chapter-2:</u>	<u>Literature Review</u>	<u>5-11</u>
2.1:	Traditional Maintenance Approaches	5
2.2:	Machine Learning in Predictive Maintenance	7
2.3:	Drive-Based Monitoring Systems	8
2.4:	Ensemble Learning Techniques	9
2.5:	Research Gap Analysis	9
2.6:	Comparative Analysis	10
<u>Chapter-3:</u>	<u>System Overview and Methodology</u>	<u>12-19</u>
3.1:	System Architecture	12
3.2:	Block Diagram	13
3.3:	Working Principle	14
3.4:	Data Acquisition Process	15
3.5:	System Components	16
3.6:	Methodology Flowchart	17
3.7:	Variable Frequency Drive (VFD) Internal Operation	18
3.8:	Mathematical Modeling of 3-Phase Induction Motors	18
3.9:	Modbus RTU Protocol Packet Structure	19
<u>Chapter-4:</u>	<u>Data Collection and Preprocessing</u>	<u>20-28</u>
4.1:	Dataset Description	20
4.2:	Data Sources	21
4.3:	Data Cleaning and Validation	22
4.4:	Feature Engineering	23
4.5:	Data Normalization and Scaling	23
4.6:	Train-Test Split Strategy	24
4.7:	Exploratory Data Analysis (EDA)	25
4.8:	Sensor Distribution and Histograms	26
4.9:	Correlation Heatmap Analysis	27
4.10:	Boxplot Analysis of Fault Classes	28
<u>Chapter-5:</u>	<u>Machine Learning Model Implementation</u>	<u>29-32</u>
5.1:	Model Selection	29

5.2:	Model Training	30
5.3:	Performance Evaluation	31
5.4:	Theoretical Foundation of Random Forests	32
<u>Chapter-6:</u>	<u>Implementation And System Development</u>	<u>33-36</u>
6.1:	Development Environment	33
6.2:	Streamlit Dashboard	33
6.3:	Deployment Strategy	34
6.4:	Generative AI and Large Language Models (LLMs)	35
6.5:	System UML Diagrams	36
<u>Chapter-7:</u>	<u>Results and Discussion</u>	<u>37-42</u>
7.1:	Model Performance Comparison	37
7.2:	Confusion Matrix Analysis	38
7.3:	Feature Importance Analysis	39
7.4:	Dashboard Output Interpretation	40
7.5:	Statistical Validation	41
7.6:	Comparative Analysis	42
<u>Chapter-8:</u>	<u>Conclusion and Future Scope</u>	<u>43-44</u>
8.1:	Conclusion	43
8.2:	Future Scope	44
	References	45-48
	Appendix A: System Source Code	49-57
	Appendix B: Raw Modbus Telemetry Dataset	58-66

LIST OF FIGURES

<u>Figure No.</u>	<u>Description</u>	<u>Page No.</u>
Fig 3.1	Block Diagram of the Proposed Predictive Maintenance System	22
Fig 3.2	Flowchart	26
Fig 4.1	Histogram charts	35
Fig 4.2	Correlation Heatmap	37
Fig 4.3	Boxplot	38
Fig 5.1	Random Forest Diagram	41
Fig 6.1	Use-Case Diagram	45
Fig 6.2	Sequence Diagram	46
Fig 6.3	Class Diagram	46
Fig 7.1	Confusion Matrix	39
Fig 7.2	Feature Importance	40
Fig 7.3	AI Dashboard	41

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

SYMBOLS

<u>Symbol</u>	<u>Description</u>	<u>Unit</u>
τ	Torque	Nm
ω	Rotational Speed	rpm
P	Power	Watt
T	Temperature	K or °C
I	Current	A
V	Voltage	V
η	Efficiency	%
RPM	Revolution Per Minute	rpm
kW	Kilowatt	1000 Watts

ABBREVIATIONS

<u>Abbreviation</u>	<u>Full Form</u>
ML	Machine Learning
AI	Artificial Intelligence
VFD	Variable Frequency Drive
PLC	Programmable Logic Controller
IoT	Internet of Things
EDA	Exploratory Data Analysis
RF	Random Forest
ROC	Receiver Operating Characteristic
AUC	Area Under Curve
FPR	False Positive Rate
TPR	True Positive Rate
CSV	Comma Separated Values
UI	User Interface
UX	User Experience
API	Application Programming Interface
RAM	Random Access Memory
CPU	Central Processing Unit
GPU	Graphics Processing Unit
IDE	Integrated Development Environment
OS	Operating System
RUL	Remaining Useful Life
ANN	Artificial Neural Network
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
GRU	Gated Recurrent Unit
DAQ	Data Acquisition
HVAC	Heating, Ventilation, and Air Conditioning
SCADA	Supervisory Control and Data Acquisition
MQTT	Message Queuing Telemetry Transport
REST	Representational State Transfer

JSON	JavaScript Object Notation
SQL	Structured Query Language
GUI	Graphical User Interface
CLI	Command Line Interface

NOMENCLATURE

<u>Term</u>	<u>Definition</u>
Predictive Maintenance Condition Monitoring Fault Diagnosis	Maintenance strategy that uses data analysis to predict equipment failures
Feature Engineering Ensemble Learning Hyperparameter Tuning Classification	Process of monitoring machinery parameters to detect significant changes
Regression	Identification of faults based on symptoms and parameter deviations
Overfitting	Process of creating new features from existing data to improve model performance
Underfitting	Machine learning technique that combines multiple models for better predictions
Cross-Validation	Process of optimizing model parameters to achieve best performance
Scalability	Machine learning task of predicting categorical labels
Real-time Processing	Machine learning task of predicting continuous values
Batch Processing	Model performs well on training data but poorly on unseen data
Dashboard	Model fails to capture underlying patterns in the data
Cloud Computing	Technique for assessing model performance on unseen data
Edge Computing	Ability of a system to handle increasing amounts of work
Industry 4.0	Processing data immediately as it is received
Digital Twin	Processing data in groups at scheduled intervals
Smart Manufacturing	Visual display of important information for monitoring and decision making
	Delivery of computing services over the internet
	Processing data near the source rather than in centralized cloud
	Fourth industrial revolution characterized by automation and data exchange
	Virtual representation of a physical object or system
	Use of connected devices and data analytics in manufacturing

Preventive Maintenance	Maintenance performed at predetermined intervals
Reactive Maintenance	Maintenance performed after equipment failure
Proactive Maintenance	Maintenance based on actual equipment condition
Mean Time Between Failures	Average time between failures of repairable equipment
Mean Time to Repair	Average time required to repair failed equipment
Overall Equipment Efficiency	Measure of manufacturing productivity

ABSTRACT

The electrical machinery, like induction motors can stop working and this is a big problem for factories. It causes a lot of loss and downtime. The old way of fixing things which's to wait for something to break and then repair it or to fix things at a set time is not working very well. It does not catch problems with the machinery before they become big issues.

This project is about creating a system for Predictive Maintenance that uses data and Artificial Intelligence to predict when something might go wrong. The system uses the machinery's hardware to get information so it does not need extra sensors. It gets this information from a Variable Frequency Drive using a protocol called Modbus RTU.

The system then uses a kind of Machine Learning called a Random Forest Classifier to look at this information. This classifier is very good at finding problems like if the electricity's not balanced or if something is getting too hot. It can even find problems with the bearings.

The project also uses something called Generative AI to help people understand what the machinery is telling us. It takes the information from the classifier. Uses the Google Gemini Large Language Model to write reports that people can understand. This way the system can tell us what is wrong with the machinery in a way that's easy to understand.

This is a change for factories because it means they can fix problems before they happen instead of waiting for something to break. It is also faster and can be used for a lot of machines so it is a good solution, for factories. The Predictive Maintenance system is a way of doing things that is smarter and more automated.

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Induction motors and other electrical machines are the backbone of the current industrial automation and manufacturing designs. They are constantly pushing important structural units such as heavy assembly line equipment to fluid pumps and heating-ventilation-air conditioning units. How these motors deliver continuous and consistent performance is of utmost importance in ensuring high production efficiency and low operational loss. Nevertheless, with ongoing mechanical load, electrical transients and severe industrial conditions, induction motors become extremely vulnerable to numerous structural and electrical defects as time passes; degeneration of the bearings, phase imbalance, and stator overheating.

Historically there are two main paradigms of industrial maintenance which are Reactive Maintenance (run-to-failure) and Preventive Maintenance (time-based scheduling). Reactive maintenance can result in disastrous equipment damage and very expensive unplanned production downtime. In contrast, preventive maintenance often leads to premature replacement of healthy machine parts and adds needless overhead of labor. These inefficiencies indicate that the industrial world is urgently in need of a shift towards an even smarter, more proactive approach.

The main idea behind this project is the need to adopt a data-based Predictive Maintenance (PdM) model. As Industry 4.0 grows at a rapid pace, the implementation of smart sensors along with the use of the Modbus RTU protocol enables real-time, continuous telemetry recording of vital parameters of the motor, including operating frequency, phase current, RPM, voltage and temperature. Through this stream of endlessly moving data concerning operations, mechanical and electrical problems can be detected at the initial stages before they turn into total mechanical failures of the system.

Moreover, even though raw condition monitoring gives useful data, it is often difficult to quickly give meaningful interpretation of the complex and multi-dimensional fault signatures on the factory floor by human operators. Thus, the rationale behind this project is the possibility to autonomously detect and classify specific motor faults using high precision with the help of better Machine Learning algorithms, namely, the Random Forest classification. In order to fill the gap between raw data of analytics and human actionable information to the full extent, one will be highly motivated to incorporate Generative Artificial Intelligence (Large Language Models) into the diagnostic process. With machine-learning fault classification that is highly accurate and an in-built AI expert consultant, the proposed system does not only predict imminent motor failures but also produces context-sensitive and professional maintenance advice, in real-time. The main benefit of this integrated approach is to provide a better equipment life cycle, greatly decrease maintenance-related costs, and ensure supply chain resiliency.

1.2 Problem Statement

Although there has been a major progress in the automation of industries, unexpected machine failures are still a major problem in the current manufacturing and heavy industries. The induction motors that are the main driving force of most industrial operations are often exposed to extreme operational loads leading to

internal damage. The current methods of traditionally used maintenance strategies of such motors have three big limitations:

Ineffectiveness of Traditional Maintenance: Reactive maintenance is intrinsically associated with disastrous unforeseen failures, which disrupts the entire production lines and causes enormous financial and operational losses. On the other hand, planned preventive maintenance is manpower consuming and often results in the blind replacement of parts that are still in good working condition, fueling up unwarranted operational expenses.

Absence of Intelligent Real-Time Monitoring: Intensive real-time monitoring is not provided in many current industrial systems. Once a fault condition, like a phase unbalance, rotor friction (bearing fault), or stator overheating, has started to occur, it is usually not detected by human operators at all until the motor physically stalls, burns out or violently damages the Variable Frequency Drive (VFD).

The Diagnostic Knowledge Gap: In spite of having sensor arrays to identify raw threshold anomalies, diagnosing the particular root cause and developing an appropriate corrective action takes a considerable amount of time. The floor engineers usually spend the most important down time hours, browsing through manuals and analyzing abstract data curves in order to resolve the fault.

Thus, the solution to the issue is a lack of an integrated, accessible framework that can easily harvest real-time operational data of motor controllers, autonomously categorize particular imminent mechanical and electrical malfunctions using ongoing machine learning, and bridge the diagnostic divide by sending human-readable and expert-level maintenance instructions to the operator in real-time.

1.3 Significance of Predictive Maintenance

The implementation of Predictive Maintenance (PdM) is a paradigm shift in the mode of operation and maintenance of key electrical equipment by industries. The changes in reactive, or highly scheduled maintenance to an intelligent, predictive system have extensive consequences in operations, discrete economic and safety realms. Predictive maintenance is about the incorporation of:

Cutting down of Unplanned Downtime: Unplanned machine failures automatically halt production lines and have an impact on the supply chain. Predictive maintenance anticipates microscopic damage, such as early stage bearing friction or small phase imbalances, by examining the motor telemetry in real-time, way before they cause a complete operational halt. This will allow the facility managers to schedule the needed repairs within the scheduled windows in the planned maintenance, and sustain constant manufacturing schedules.

Considerable Cost savings: Predictive maintenance is proactively saving overheads by eradicating the twin evil of devastating component failures and unnecessary preventive maintenance. Rather than discarding healthy parts based on rigid time limits, or recreating horrendously damaged infrastructure around the surrounding, maintenance is not undertaken until the data signature of a certain part notifies of an impending failure.

Maximization of Machinery Life: Electrical machines that need to be used

continuously and without being noticed (e.g. stator overheating or unbalanced loading) wear-out exponentially. PdM allows the running of motors at the optimum thermal and electrical limits because it allows operators to be informed in real-time of these latent stresses, which ultimately increases their overall life cycle. Better Work Place Safety: When a mechanical stall or electrical malfunction in an industrial high-voltage motor occurs, it is life threatening to the floor workers, including potential electrical fire. The predictive systems are essential in providing a safe working environment in a mechanical environment by anticipating such a total failure.

The AI-based professionalism and productivity: According to the specific project, the significance of PdM is compounded with the application of both Generative AI and Machine Learning. Raw telemetry analysis is one of the tasks requiring certain expertise, but AI-enhanced PdM system is democratic with this knowledge. It is also highly efficient in relieving the maintenance engineers as it identifies the cause of an anomaly beforehand and offers step-by-step guidance on how to fix it, turning raw sensor data directly into operational insights.

1.4 Objectives of the Study

The main objective of the project is to survey, design and deploy an end-to-end predictive maintenance system of electrically powered induction motors by integrating physical industrial protocols with intelligent Artificial Intelligence. In order to do so, the following specific objectives are set in this study:

Resilient Data Acquisition: To design a reliable software interface that would allow the communication with a Variable Frequency Drive (VFD) through the Modbus RTU serial protocol and enable the continuous extraction and recording of important operational telemetry, such as running frequency, RPM, terminal voltage, load current and VFD temperature indicators.

Machine Learning Fault Classification: To design and train a very high preciseness Machine Learning predictive model (a Random Forest Classifier) using motor telemetry that has already been pre-processed. The model is designed to effectively classify the existing operating conditions and reliably detect discrete physical abnormalities, such as Phase Unbalances, Stator Overheating, and Rotor/Bearing Faults.

Generative AI Diagnostics Integration: To be the first to close the gap between numerical classification of faults and the human judgement by embedding a Large Language Model (Google Gemini AI) into the diagnostic process. The task is to use the LLM as a professional consultant who can immediately examine mechanical failures and create detailed and professional maintenance estimates on the basis of the current state of the system in real-time.

Construction of Interactive Status Dashboard: To create and implement a modern and intuitive Graphical User Interface (GUI) which will be the key nexus of the predictive maintenance ecosystem. This dashboard will have to be able to combine the machine learning engine of inference and the AI chat interface to allow facility operators real-time, high-fidelity insights into the health of the motor.

Architectural Validation: To complete the validation of the entire hardware-to-software architecture, demonstrate that the continuous sensor telemetry and parallel AI workflow is a viable, cost-effective and scaled alternative to conventional industrial maintenance systems.

1.5 Organization of the Report

In order to give a detailed and well-structured picture of the project, the rest of this report is divided into the following chapters that will systematically reflect

the research, the implementation, and evaluation of the predictive maintenance system:

Chapter 2: Literature Review: The chapter reviews the literature on conventional maintenance methods and the developments in machine learning in predictive maintenance. It discusses drive-based monitoring systems and ensemble learning methods and ends with an in-depth analysis of research gaps and a comparative analysis of existing literature.

Chapter 3: System Overview and Methodology: This chapter outlines the system structure of the project, which includes the system architecture and the main block diagrams. It describes the principles of operation, provides a general description of the process of data acquisition, and shows the general flowchart of the methodology under which the system components operate.

Chapter 4: Data Collection and Preprocessing: Chapter 4 is dedicated to data pipeline which describes the data description and data sources. It thoroughly addresses the process of cleaning and validating data, feature engineering, data normalization and scaling, and ends with the train-test split approach to machine learning.

Chapter 5: Machine Learning Model Implementation: Chapter 5 explores the essence of AI intelligence of the project. It identifies the particular reason why the model was selected, the model training processes and the frameworks applied to conduct rigorous performance evaluation.

Chapter 6: Implementation And System Development: This chapter discusses software engineering of the project. It outlines the overall development landscape, the concrete implementation of the Streamlit Dashboard to allow users to interact with it, and the general deployment plan of the monitoring system.

Chapter 7: Results and Discussion: The chapter is a comprehensive discussion of how the system is effective. It comprises model performance comparisons, an in-depth analysis of confusion matrix and feature importance analysis. It also decodes the dashboard outputs, statistical validation and a comparative analysis of the dashboard outputs with the operational systems in use.

Chapter 8: Conclusion and Future Scope: The last chapter provides the overall conclusion of the main findings and technical contributions of this project and ends with the possible directions of future research and further improvement of the system.

CHAPTER 2: LITERATURE REVIEW

2.1 Traditional Maintenance Approaches

Traditionally, the industrial industry has been dependent on the maintenance strategies as the foundation of supporting the lifecycle of the electrical machines. An analysis of the literature indicates that these conventional paradigms are broadly divided into two groups: Reactive Maintenance and Preventive Maintenance, neither of which have any real operational limitations when used in the present, high efficiency manufacturing settings.

2.1.1 Reactive Maintenance (Run-to-Failure) Reactive maintenance (also known simply as run-to-failure) is the most primitive and ancient. This paradigm involves continuous operation of machinery until a physical breakdown is definite after which corrective repairs are immediately started. Many industrial studies indicate that this method incurs no initial cost when it comes to monitoring infrastructure but it ends up being the most costly strategy in the long term. Researchers stress that the reactive maintenance will always result in disastrous equipment failures, when a small defect (say, a deteriorating bearing) is allowed to escalate and consume other healthy parts (say, the motor stator). Moreover, unpredictability of run-to-failure models leads to critical, unplanned production breakdowns, which lead to considerable financial latency along the supply chain.

2.1.2 Preventive Maintenance (Time-Based) In order to overcome the vagaries of reactive maintenance, Preventive Maintenance (PM) or Time-Based Maintenance (TBM) became common in industries. As recorded in other reliability engineering books, PM is based on statistical numbers such as Mean Time Between Failures (MTBF) that is offered by equipment suppliers. Maintenance plans are strictly scheduled by the calendar time or working hours and not by the actual physical condition of the machine.

Although literature points out that PM is effective in terms of reducing the occurrence of unforeseen catastrophic failures relative to the reactive approach, it presents a severe economic inefficiency. Research on time-based maintenance indicates that a major portion of machine parts are changed too soon when they still has a considerable useful service life. The result of this over-maintenance is inflated labor expenses, overuse of the spare parts inventory and unnecessary planned downtime.

2.1.3 Early Condition-Based Maintenance(CBM) As a transitional measure between the old and the new predictive system, Early Condition-Based Maintenance (CBM) was developed. As seen in literature reviews on early CBM, it relied on local and manual sensor readings, like handheld vibration pens or periodic thermal imaging, to monitor machine health. Nevertheless, researchers note that such legacy CBM strategies were constrained by the use of manual data gathering and fixed and human-defined alarm levels. They were unable to perform the continuous analysis of multi-variable streams of data, which is essential to identify the small, nonlinear signature of fault that have been shown to be present in modern motor failures.

The natural shortcomings of these classical methods: high cost of downtime in reactive plans, wasted resources in preventive plans, and the computational constraints of early CBM have been the impulse behind the scholarly and business move to data-driven predictive models.

2.2 Machine Learning in Predictive Maintenance

In order to address the constraints of traditional and manual condition-based maintenance, the recent literature extensively focuses on the addition of Artificial Intelligence (AI) to predictive maintenance (PdM) systems, namely, the use of Machine Learning (ML). Machine Learning algorithms have transformed the approach to equipment condition monitoring to the paradigm shift of human alarms (static, threshold-based) to data-driven autonomous diagnostics.

2.2.1 The Data-Driven Transition A survey of recent work in the industrial research community indicates that using ML in predictive maintenance has the main benefit of being able to handle and analyze large and highly-dimensional data sets much more effectively than humans do. Contemporary electrical machines have sensor arrays that produce data streams of telemetry (operating frequencies, stator currents, voltages, and thermal gradients) continuously. These sensor readings when taken separately may well be within a normal acceptable range of sensor readings, however, their non-linear complicated relationships tend to hide the subtle microscopic alerts of an impending fault. Machine Learning models are specifically good at capturing these intricate multivariate relationships so that subtle anomalies can be detected before a physical failure can occur and generate a traditional alarm.

2.2.2 Desirable Algorithms in Literature Researchers have discussed numerous ML models of fault classification in electrical machines. These are mainly classified into supervised and unsupervised learning methods in the literature:

Unsupervised Learning: In literature K-Means clustering and Principal Component Analysis (PCA) are commonly referenced in cases where it is purely trying to detect anomalies. These models are conditioned with historical data of the state of health of the motor. An anomaly is indicated when the actual data does not fit in these healthy clusters. Although it can be helpful in detecting simple faults, researchers mention that unsupervised models can have difficulties in determining the type of mechanical fault that has occurred.

Supervised Learning: To accurately diagnose fault, supervised classification algorithms are the major source of PdM literature. In these experiments, models are trained on carefully marked datasets with features that were extracted under known faults (e.g., a particular voltage imbalance or friction coefficient related to wear of bearings).

The Support Vector Machines (SVMs) and Artificial Neural Networks (ANNs) are well reported to be very accurate in motor fault detection. Nonetheless, they are sometimes criticized in literature as being black-boxes, i.e. hard to computationally understand how the model reached its conclusion, as an engineer. The most common alternatives that have been mentioned in the most recent PdM literature are Decision Trees and Random Forests. Scientists commend the algorithms due to their ability to provide a high predictive accuracy and their interpretability. Since they handle data with a sequence of logical partitions, engineers can simply obtain "Feature Importance" measures, with a clear idea of what sensor values (such as VFD temperature or RPM) made the most significant contribution to the fault

prediction. In general, the literature highlights that predictive maintenance can be used to convert predictive maintenance into a passive monitoring system to a proactive, intelligent diagnostic driver, which can identify individual electrical and mechanical failure modes independently.

2.3 Drive-Based Monitoring Systems

One of the major changes in recent predictive maintenance literature is the replacement of external, chassis-based sensor arrays with Drive-Based Monitoring Systems. In the past, condition monitoring had to involve exposing the motor housing to special, external equipment, such as piezoelectric vibration sensors, acoustic microphones, and external thermal probes. Although effective in data collection, studies indicate that these external systems are very invasive, costly to install over large areas and likely to cause destruction to the environment in extreme industrial environments.

2.3.1 The Variable Frequency Drive (VFD) as a Sensor Node More recently, a more elegant and cost-efficient solution has been proposed: the Variable Frequency Drive (VFD) that is used to control the motor is used as the major data acquisition node. Contemporary VFDs can and do process large quantities of high-resolution electrical data to ensure accurate motor control. It is confirmed in the literature that such critical machine health indicators as phase voltages, load currents, operating frequencies (Hz), RPM, and the internal thermal conditions of the drive are already under constant observation by the internal microprocessor of the drive.

Scientists have already established the possibility of directly obtaining these existing electrical signatures directly out of the VFD, rather than having to use external mechanical sensors. Since mechanical defects (a failing bearing resulting in rotor friction) directly affect the electromagnetic field of the motor, mechanical defects will be represented by measurable distortion in the electrical current and voltage signals returned to the VFD.

2.3.2 Industrial Communication Protocols (Modbus RTU) To provide this drive-based extraction, the use of standardized industrial communication networks is widely reviewed in the literature. Electrical noise is likely to occur with traditional analog signal tapping. Thus, the use of digital serial protocols that are already developed is highly recommended by the researchers, and one of the most commonly mentioned and widely adopted digital serial protocols in industrial automation is the Modbus RTU.

Research indicates that it is possible to interface with a VFD using Modbus RTU in order to have a central predictive maintenance software to query individual holding registers in the drive without any issues. It is capable of continuously, high-fidelity recording the electrical and thermal telemetry of the motor without disturbing the control loop of the motor. Through implementing a drive-based, Modbus-driven architecture, industries are able to deploy end-to-end data-gathering networks using the existing infrastructure, and the entry point to intelligent machine learning architectures is greatly reduced.

2.4 Ensemble Learning Techniques

Although working with individual machine learning frameworks such as

standalone Artificial Neural Networks (ANNs) or single Decision Trees have demonstrated the capacity to be effective in condition monitoring, literature indicates that they in most cases have serious constraints when used in the real-world industrial setting. Single models are very vulnerable to overfitting, or memorizing the training data, but not detecting anomalies in live and noisy factory data. In a bid to overcome this volatility, the use of Ensemble Learning techniques has been a major recommendation of recent predictive maintenance research.

2.4.1 The Principle of Ensemble learning Ensemble learning is a paradigm of machine learning in which multiple, independent base models (also known as weak learners) are trained together and aggregated together to give a single, strong prediction (also known as a strong learner). It has been well-documented in literature that when coupled together, the predictions of many diversified models, using ensemble methods, minimize system variance and bias dramatically. This leads to a much more accurate, stable and resilient predictive engine that is much more susceptible to the unavoidable noise in industrial sensor telemetry.

Bootstrap Aggregating (Bagging) and Random Forests Within predictive maintenance literature, a single ensemble algorithm is constantly pointed to as having the best performance in fault classification: the Random Forest algorithm. Random Forest works based on an ensemble concept that is referred to as Bootstrap Aggregating or Bagging.

In the studies, it is described how a Random Forest, rather than a single large decision tree, is used to analyze motor data, and as many as hundreds of different decision trees are created by the forest. The trees are trained on a random, bootstrapped subset of the master dataset, and most importantly each tree only evaluates a random subset of the sensor features (e.g., a single tree may mostly analyze current and frequency, another temperature and RPM).

When a live data sample is received on the motor drive, all trees of the forest are processing the data and they independently predict the state of the motor (e.g., Healthy, Overheat or Bearing Fault). The overall result of the Random Forest is the majority tendencies of all the trees that compose it.

2.4.3 Strengths in Industrial Fault Diagnostics Academic comparative studies have always rated Random Forest architectures very high in electrical machine diagnostics because of three vital strengths:

Noise Tolerance: Its voting mechanism is highly resistant to electrical noise and to temporary spikes in sensor signals, due to its aggregated voting mechanism.

High Dimensionality Processing: It can readily accommodate non-linear interactions among a wide variety of electrical signals (voltage, current) and physical signals (temperature, calculated friction).

Interpretability (Feature Importance): Unlike black-box models, literature glorifies the fact that Random Forests can rank the importance of particular input features. This openness enables engineers to mathematically check precisely what physical deviations led to the model forecasting a particular failure mode.

2.5 Research Gap Analysis

An extensive review of studies on predictive maintenance for electrical machines shows significant progress in data analysis and machine learning methods. However a closer look at these existing frameworks reveals major research gaps that hinder their practical use in modern industrial factories.

This project aims to address these identified shortcomings.

Over-Reliance on External Hardware: Most published studies focus on sensor fusion requiring the installation of delicate external vibration sensors and high-speed acoustic monitors onto the motor casing.

The research gap lies in the lack of methods that use the telemetry already being processed by existing Variable Frequency Drives (VFDs).

There is a need to shift the focus from external hardware additions to leveraging internal Modbus drive communications for cost-effective scalable monitoring using VFDs.

The Actionable Intelligence Gap: Current literature features classification models, such as Support Vector Machines, Convolutional Neural Networks and decision trees.

These models have a limitation: they output a numerical probability or a categorical label like "Class 3: Overheat".

The literature ignores the reality of the factory floor: a simple "Fault Flag" is not enough. A technician still has to investigate why the fault occurred and how to resolve it leading to diagnostic downtime.

Absence of Generative AI Integration: There is a gap in the literature regarding the intersection of numerical fault classification and natural language intelligence.

Current predictive maintenance ecosystems lack an expert system capable of contextualizing machine learning outputs.

There is virtually no research integrating Large Language Models into the diagnostic pipeline to translate complex electrical anomalies into plain-text maintenance counsel.

In summary the literature provides tools for detecting faults but fails to provide a framework that interprets and explains those faults, in human-actionable terms.

This study aims to bridge this gap by combining VFD data acquisition, Ensemble Machine Learning classification and Generative Artificial Intelligence into a predictive diagnostic dashboard.

The goal is to provide a predictive diagnostic dashboard that can be used on factory floors.

Predictive maintenance using VFDs and machine learning can help prevent faults.

Machine learning models and Large Language Models will be used together.

The use of Variable Frequency Drives and Modbus drive communications is crucial.

2.6 Comparative Analysis

To understand why this study matters we need to look at studies on predictive maintenance. We will look at four areas: how we get the data how we find faults, how useful the results are and how hard it is to set up the system.

1. Getting the data and the hardware we need

systems use special sensors that are attached to the machines. These sensors are very good at measuring things.. They are also expensive and can break easily.

Our predictive maintenance system is different because it uses the data that's already available from the machines control system. This means we do not need to buy hardware, which saves us money and makes things easier.

2. Finding faults

Other systems use rules to find faults. These rules are not very good at finding complicated problems. Some systems use machine learning. They often use only one type of model which can be wrong if the data is not perfect.

Our predictive maintenance system uses a type of machine learning called Ensemble Learning. This means it uses models at the same time, which makes it more accurate and better at handling imperfect data.

3. What we do with the results and how we use intelligence

Other predictive maintenance systems just give us numbers or codes that tell us something is wrong but they do not tell us what to do about it.

Our predictive maintenance system is different because it uses a special language model that can translate the results into language that a person can understand. This means the person fixing the machine gets a report that tells them exactly what is wrong and how to fix the machine.

4. How hard it is to set up the maintenance system and how much it costs

Other predictive maintenance systems are expensive and hard to set up because they need extra hardware and software. They also need experts to understand the results.

Our predictive maintenance system is cheaper and easier to set up because it uses the hardware that's already there. It also has an interface that makes it easy for anyone to use even if they are not an expert in maintenance.

In short this study is important because it creates a maintenance system that is

easy to use and cost-effective. It takes maintenance from just being a tool that analyzes data to a system that can actually help us fix problems with the machines. Our predictive maintenance system is an improvement, over other systems.

Figure 3.1: System Block Diagram

If you look at the diagram you can see that the system has two parts that work together: power distribution and data communication.

1. Power Distribution Flow:

We use a 220V power supply.

The Variable Frequency Drive gets the power from this supply. The main job of the Variable Frequency Drive is to change the power into a type and then change it back into a special 3-phase power.

The 3-Phase Motor uses this power from the Variable Frequency Drive. The Variable Frequency Drive controls how fast the motor goes and how power it uses by changing the power.

2. Data Communication Flow:

The Variable Frequency Drive also helps with data communication. It is connected to the laptop with a special wire.

We need a converter to connect the Variable Frequency Drive to the laptop because the laptop does not have the right port. This converter changes the signals from the Variable Frequency Drive into a type that the laptop can understand.

The laptop is like the boss of the system. It uses a connection to get data from the Variable Frequency Drive and then uses this data to learn and analyze things with the help of Python software and Artificial Intelligence. The laptop gets data like voltage and current, from the Variable Frequency Drive. Uses it to make decisions.

3.3 Working Principle

The predictive maintenance system is always on. It works in a loop connecting the physical machine to advanced software analytics. The way it works can be broken down into four stages:

1. Parameter Monitoring and Extraction: The system checks the 3-phase induction motor as it runs with the Variable Frequency Drive. The drive measures the electrical parameters to keep the speed and torque right. At the time a Python script called Modbus_Software.py on the central laptop sends a query to the drive. The software gets the numbers for the motors frequency, voltage, current, RPM and the drives temperature.

2. Feature Engineering: The raw data from the drive is not enough for predictions. So the software calculates features like Apparent Power, Load Friction and Cooling Efficiency. It uses the relationship between the values to do this. Then it scales all the data points so no single unit is more important than the others.

3. Fault Classification: The normalized data goes into the systems core intelligence. The pre-trained Random Forest Classifier. The classifier checks the real-time sensor data against its decision trees. It finds anomalies in the data like a slight rise in current with normal RPM but high VFD temperature. Then it says what state the motor is in, like Healthy, Phase Unbalance, Overheat or Bearing Fault.

4. Expert Diagnostics and Visualization: After the classifier says what state the motor is in the system makes a report. It sends the state and the important sensor readings to a Large Language Model like Google Gemini AI. The AI makes a human- report that says what the fault means and what maintenance to do. Finally the report, the real-time telemetry and the AIs advice are shown on the Graphical User Interface for the facility operator to see and act on the maintenance systems findings, about the predictive maintenance system and the motor. The predictive maintenance system helps the facility operator understand what is going on with the motor and what to do about it.

3.4 Data Acquisition Process

The accuracy and reliability of any maintenance model depends entirely on the quality and consistency of its foundational data. In this study the data acquisition architecture uses an industrial digital communication link with the Variable Frequency Drive or VFD instead of external analog sensor arrays. This link uses the Modbus RTU protocol over a two-wire RS-485 network, which's the Variable Frequency Drives communication method.

3.4.1 The Modbus RTU Protocol Interface

To establish a connection between the central computer and the Variable Frequency Drive the system uses Python as the primary development environment. It uses the minimalmodbus and pyserial networking libraries. In this architecture the Python script acts as the "Modbus Master" and the physical Variable Frequency Drive serves as the "Modbus Slave" with a Node ID of 1.

The RS-485 physical layer is very good at preventing interference, which is often found near high-voltage induction motors. This allows for high-speed and secure data transmission to and from the Variable Frequency Drive.

3.4.2 Parameter Extraction and Register Mapping

During the acquisition cycle the Python script sends hexadecimal queries to the Variable Frequency Drive asking for the contents of pre-defined internal holding registers. The Variable Frequency Drive, which continuously tracks these metrics instantly responds with the dynamic values. The specific raw telemetry parameters that the script targets for extraction include:

- * Operating Frequency, which's the current alternating frequency supplied to the motor by the Variable Frequency Drive.
- * Output Voltage, which is the voltage being applied to the motor terminals by the Variable Frequency Drive.
- * Phase Current, which is the current being drawn by the motor load from the Variable Frequency Drive.
- * Rotor Speed, which is the rotational speed of the motor.
- * VFD Internal Temperature, which is the state of the Variable Frequency

Drives heat sink and internal circuitry.

3.4.3 Data Logging and Formatting

Because the raw data received over Modbus is often encoded the acquisition script automatically decodes, scales. Formats the data into standard engineering units. For instance the Variable Frequency Drive. Once correctly formatted this synchronized array of time-stamped telemetry from the Variable Frequency Drive is continuously logged into a comma-separated values file, which is called `pdm_motor_log.csv`. This structured file serves two purposes: it acts as a database for offline model training using data from the Variable Frequency Drive and it serves as the immediate data pipeline, for the real-time AI prediction dashboard using the Variable Frequency Drives data.

3.5 System Components

To get this predictive maintenance setup working we had to put several different hardware and software parts. We focused on using standard tools instead of making the setup too complicated.

3.5.1 Hardware Elements

The physical part of the project is straightforward. Is all about the motor and how we communicate with it:

3-Phase Induction Motor: This is what we're testing. It's the machine we want to monitor and protect from failures.

Variable Frequency Drive (VFD): The VFD does two things. It controls how fast the motor spins. It also acts as our sensor hub holding all the electrical data we need in its memory.

USB to RS-485 Converter: We used this adapter cable because standard laptops don't have industrial serial ports. It helps our computers USB port read the RS-485 signals from the VFD.

A Standard Laptop: We used a computer as the central brain. It runs all the Python scripts handles the math for machine learning and displays the dashboard.

3.5.2 Software Tools

For the software side we relied heavily on Python because of its open-source community, especially for data science and hardware control:

Python (Version 3.10+): We coded the project in Python. We chose it because it makes handling data and connecting to APIs

minimalmodbus and pyserial: These are Python libraries we installed. They help our code talk to the VFD over the Modbus protocol, without data getting corrupted.

pandas and scikit-learn: We used pandas to organize the data into tables. Then we used scikit-learn to build and train our Random Forest model.

google-generativeai: This is the Google API tool. We needed it so our local Python script could send fault data to the Gemini LLM and get a readable diagnosis back.

CustomTkinter: We used CustomTkinter to design a dark-themed dashboard that looks professional. It allows the operator to easily see the motors health in time.

3.6 Methodology Flowchart

To make the overall process easier to understand we created a flowchart. It shows how the software works, step by step from the motor to the AI diagnostic screen.

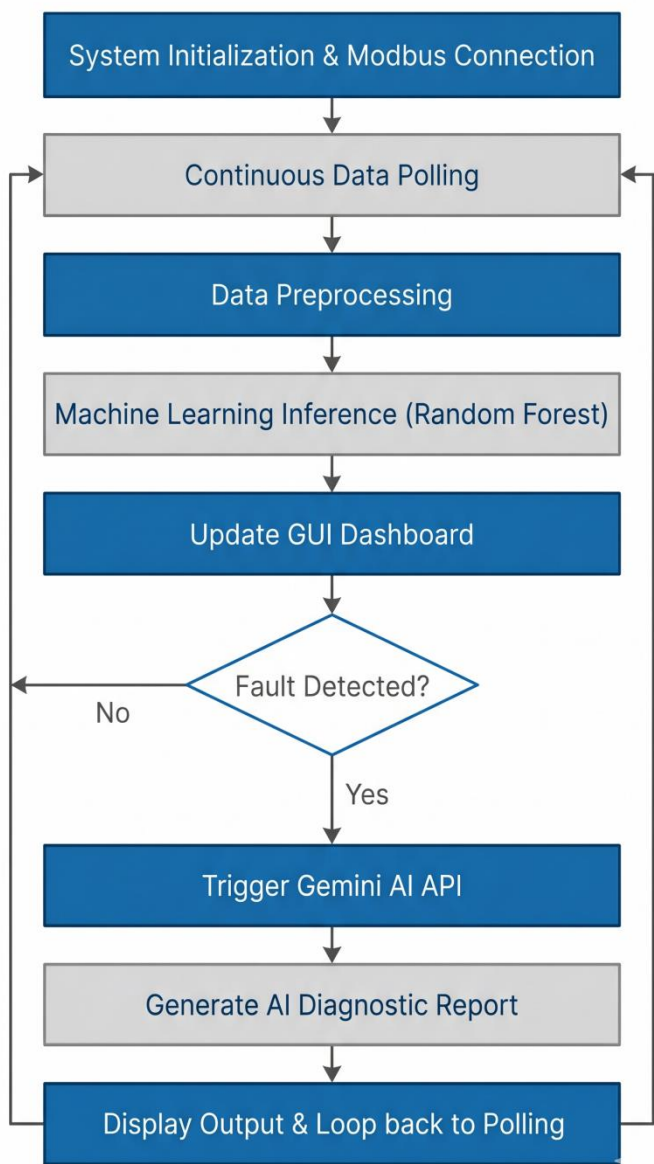


Figure 3.2 Flowchart

The flowchart shows how our system works in a loop:

System Initialization: The software starts by opening the CustomTkinter dashboard. It tries to connect to the VFD using a Modbus serial connection.

Continuous Data Polling: Once connected the system enters its loop. It constantly checks the VFD for the data like current and frequency.

Data Preprocessing: The software cleans up the data and scales it so it can be used later.

Machine Learning Inference: This is the part. We use the data in our trained Random Forest model. The model looks at the data. Decides if the motor is running well or if there is a problem.

Dashboard Update: The software updates the gauges and status indicators, on the operators screen in time no matter what happens.

AI Diagnostic Trigger: If the model finds a problem like an overheat the system sends a prompt to the Google Gemini AI API with the problem and sensor readings.

AI Report Generation: The Gemini AI analyzes the prompt and sends back a maintenance recommendation.

Output and Loop: The AI's advice appears on the dashboard. Then the software loop goes back to step 2 to get the batch of data. This loop keeps running while the system is active.

3.7 Variable Frequency Drive (VFD) Internal Operation

The Variable Frequency Drive is part of our system. It controls the motor. Collects data for our maintenance system. We need to understand how it works.

To get this we have to know how the Variable Frequency Drive handles power. The Variable Frequency Drive doesn't just pass power from the wall to the motor. It breaks down the power. Rebuilds it to exact specs. This happens in three stages.

First is the Converter Stage: Its connected to the 220V power supply. The Converter Stage turns Alternating Current into Direct Current. It uses a circuit with high-power diodes to do this. The diodes act like one-way valves changing the waves into a continuous positive Direct Current waveform.

The power then goes to the DC Bus Stage: This stage acts like a filter and storage tank. The Direct Current waveform from the Converter Stage isn't smooth enough. The DC Bus uses capacitors to absorb ripples and make the waveform smooth and stable. In our 220V system the DC Bus usually has a voltage of around 310V Direct Current. The Variable Frequency Drive uses this Direct Current.

Next is the Inverter Stage: This is where the Variable Frequency Drive creates the "Variable Frequency". The Variable Frequency Drive turns the Direct Current back into a 3-phase Alternating Current to run the motor. It uses micro-switches called Insulated-Gate Bipolar Transistors.

The Variable Frequency Drives microprocessor controls these micro-switches: It. Closes them thousands of times per second. The Variable Frequency Drive uses Pulse Width Modulation to chop the Direct Current into pulses. These pulses simulate an Alternating wave when they interact with the motors magnetic coils.

The data from the Variable Frequency Drive comes from this Inverter Stage: By changing the Pulse Width Modulation sequence the Variable Frequency Drives microprocessor can alter the output Voltage and Frequency. When our Python script queries the Variable Frequency Drive over the Modbus network it extracts the algorithms the microprocessor uses. This gives us baseline data for our machine learning model. We use the Variable Frequency Drive to get this data. The Variable Frequency Drive is crucial, for our system.

3.8 Synchronous Speed Calculation

To fully appreciate how the Random Forest algorithm detects a mechanical failure, it is essential to look at the fundamental mathematical models governing a 3-phase induction motor. The telemetry we extract mathematically directly ties into how the motor generates rotational motion.

3.8.1 Synchronous Speed Calculation When the Variable Frequency Drive (VFD) outputs 3-phase AC power, it creates a rotating magnetic field inside the stator of the motor. The speed at which this invisible magnetic field rotates is called the Synchronous Speed. The synchronous speed is completely dependent on the frequency of the power supplied by the VFD. The mathematical relationship is expressed as:

$$N_s = (120 * f) / P$$

Where:

N_s : Synchronous speed of the magnetic field (measured in RPM).

f : The electrical frequency supplied by the VFD (measured in Hertz, Hz). This is exactly why we log 'Frequency' via Modbus.

P : The number of magnetic poles built into the motor's physical stator winding.

3.8.2 Motor Slip and Mechanical Faults An induction motor never actually spins as fast as its magnetic field. The physical rotor always spins slightly slower than the synchronous speed to induce current. This difference in speed is called "Slip." Slip is one of the most critical concepts in predictive maintenance because mechanical faults directly affect it. The slip formula is:

$$S = (N_s - N_r) / N_s$$

Where:

S : The fractional slip of the motor.

N_s : The synchronous speed calculated above.

N_r : The actual physical rotational speed of the rotor (measured in RPM). This is why we log 'RPM' via Modbus.

If the motor operates normally, the slip is low and stable. However, if a bearing begins to fail and introduces physical friction, the rotor speed (N_r) will drop. Mathematically, as N_r drops, the Slip (S) increases. This mathematical deviation is exactly what our machine learning model looks for when detecting a "Bearing Fault."

3.8.3 Torque and Current Relationship To overcome the friction caused by a failing bearing (and the increasing slip), the motor requires more torque. In an induction motor, electrical current is directly proportional to torque.

When the motor slips further behind the magnetic field, a larger voltage is induced in the rotor bars, causing the motor to draw significantly more electrical phase current from the VFD to stay spinning. This is why our data preprocessing explicitly engineers a relationship between drawn Current and RPM. By monitoring the mathematical relationship between the supply frequency, the resulting slip, and the corresponding current spike, our software can mathematically prove that mechanical degradation is occurring inside the motor housing.

3.9 Synchronous Speed Calculation

In cases where we operate Python libraries such as `minimodem`, they take care of the communication. In order to comprehend the data we receive fully we would have to take a look at the Modbus RTU packet structure. Such that when our system receives the temperature via the Variable Frequency Drive it receives a number of bytes via the serial network, but not text.

The Modbus RTU protocol is a dialogue between a Master and a Slave. The Master it is our Python script which initiates the conversation and Variable Frequency Drive is the Slave it which is the response to an inquiry. Each communication we make is structured in a unit, which is known as Application Data Unit frame.

Our Python script sends a message to Variable Frequency Drive to receive some data. There are four sections in this message:

- 1. The address of the Variable Frequency Drive:** This is one-byte of each packet. It informs the Variable Frequency Drive that it is to it. Our Variable Frequency Drive will be address 0x01.
- 2. The Function Code:** This is the byte. It informs the Variable Frequency Drive what to do. The Variable Frequency Drive is informed to send us some data with the code 0x03.
- 3. The information we desire:** This message section can be lengthy and contains some sub-parts. To illustrate it has the address of the data we want such as the temperature and the pieces of data we want to get.
- 4. The error check:** This is, similar to a code that ensures that the message is delivered to the Variable Frequency Drive properly. This code is calculated by our Python script and added to the message after which the Variable Frequency Drive calculates the code again and determines whether it is the same and in case it is not the Variable Frequency Drive ignores the message.

Our Python script cooperate to retrieve the data. Our Python script transmits a message and the Variable Frequency Drive replies with the required data. This assists our maintenance system to predict when things can go wrong and avoid problems. The Modbus RTU. This is done by our Python scripts collaborating. Our Python script and Variable Frequency Drive are noteworthy. It all works with the Modbus RTU protocol.

CHAPTER 4: DATA COLLECTION AND PREPROCESSING

4.1 Dataset Description

Before a machine learning model can predict when a mechanical failure will happen it needs a lot of data. The data we have for this project is not random. It is a log that shows exactly how the induction motor works under stress. Our data is in a table format like a spreadsheet. Every row shows how healthy the motor is at a moment. The data has two parts: what the sensors see and what the machine is doing. The things the sensors see are like the input. These are numbers from the Variable Frequency Drive.

The main numbers are:

Voltage: electricity going to the motor.

Current: electricity the motor is using.

Frequency: how electricity changes, making the motor go faster or slower.

Rotor Speed: how fast the motor spins.

VFD Temperature: how hot the drive is.

The target variable is important. It is a label that tells the computer what is happening with the induction motor. For every set of numbers we have a label that says what state the induction motor is in.

The labels are:

Healthy: everything is working normally.

Phase Unbalance: electricity is not evenly distributed.

Overheat: the system is too hot.

Bearing Fault: there is a problem with the induction motors bearings.

By setting up the data this way we can see how things are related. The Random Forest algorithm looks at all the numbers. It figures out what patterns lead to a problem. The machine learning model looks at the data about the induction motor. It learns how to predict when a mechanical failure will happen with the induction motor. The induction motor and the machine learning model are connected. The model is trying to predict when the induction motor will fail. The data, about the induction motor helps the model learn.

4.2 Data Sources

When we build a machine learning model we usually use datasets that we find on websites like Kaggle.. For this project we did things differently. We did not use datasets that other people had already made. Instead we made our dataset from scratch. We got all of our data from our hardware setup and we did it in real-time.

We got all of our data from the Variable Frequency Drive, which's the device that controls the motor. This device is really good at giving us data because it is always adjusting the voltage and frequency to make the motor run at the right speed. We did not use any sensors that we taped to the machine. Instead we got the data from the devices memory using a special connection that we set up in our Python script.

Since we made our data we had to run the motor and make it do different things on purpose. We wanted to get data that showed what the motor looks like when it is working normally and what it looks like when something is wrong. We got data from four phases:

Sourcing Normal Data: We ran the motor for a time with normal loads at different speeds. This gave us a lot of data that shows what the motor looks like when it is healthy.

Sourcing Electrical Faults: We taught the machine learning model what an electrical fault looks like by restricting the voltage to the motor on purpose. We saw how the devices current and temperature readings changed when it was trying to keep the motor running.

Sourcing Thermal Faults: We made the motor run really hot by giving it a load and not letting it cool down properly. We saw how the devices temperature readings changed over time.

Sourcing Mechanical Faults: We made the motor work harder by adding some resistance to the shaft. We saw how this affected the current that the device was using.

By getting our data from our Variable Frequency Drive and making the motor do different things on purpose we made sure that our dataset was real and accurate. This means that our machine learning model will be really good, at monitoring the motor and finding problems. We used this dataset to train our Random Forest model. We know that it will work well in the real world.

4.3 Data Cleaning and Validation

In theory getting data directly from a computer chip sounds like an idea.. The truth is, places where machines are used can be really tough on equipment and the data from hardware is often not perfect from the start. We were getting our telemetry data over a cable in a place with a lot of high-voltage electromagnetic noise so the data we got was a mess at first.

If we had used this data without cleaning it up first our machine learning model would have learned the wrong things, which would have caused false alarms and made it really inaccurate. So before we started training our machine learning model we had to make sure the data was clean and good to use. We used the pandas library in Python to do this.

1. Dealing with Missing Data (Lost Information) The first problem we had to fix was missing data. Sometimes the VFD would not send some information on time because of a delay in the Modbus serial connection. When this happened our data would show a "Null" or "NaN" (which means Not a Number) in a place. Machine learning cannot work with missing values it just stops working. To fix this we used pandas to look at all the data. If a row was missing information that we could not safely fill in we removed that whole row from the

spreadsheet so our training data would be good.

2. Removing Wrong Outliers (Electrical Interference) The second big problem was noise causing big spikes in the data. For example for a split second a communication problem might tell our Python script that the motor was using 900 Amps of current or spinning at 15,000 RPM. Which's not possible for our setup. To fix this we set limits for our data based on what the motor can actually do. We checked the data by telling the software to remove any rows with values that were not possible in the world. This made sure our machine learning model was only looking at mechanical stress, not random communication problems.

3. Checking the Format Finally we had to make sure the computer understood all the data as numbers. Sometimes data from a connection comes in as text instead of a number. We checked all the data, one column at a time. Made sure all the sensor readings were in the right format.

After we finished this cleaning process we changed a messy and noisy hardware log into a clean and validated dataset that was ready for advanced machine learning analysis. We used machine learning analysis on our machine learning model with the cleaned dataset. Our dataset was now perfect, for machine learning analysis.

4.4 Feature Engineering

Even after our dataset was perfectly clean relying strictly on the parameters like voltage, current, RPM and so on leaves a lot of work for the machine learning model. To make our Artificial Intelligence significantly smarter and faster we used a data science technique called Feature Engineering.

Feature engineering is the process of taking the sensors and using standard electrical and mechanical formulas to create entirely new columns of data. By combining certain physical traits we provide the Random Forest model with shortcuts that make detecting specific faults much more obvious.

For this project we engineered three features from our raw VFD telemetry:

1. Apparent Power Output: Of forcing the Artificial Intelligence to look at Voltage and Current as two completely separate variables we multiplied them together to create a new Apparent Power feature.

Why it helps: if both voltage and current fluctuate slightly at the time the change might be too small for the Artificial Intelligence to flag independently. However their combined product, which is the Power shows a more obvious and amplified spike immediately signaling that the motor is working harder than normal.

2. Estimated Load Friction: A deteriorating bearing creates friction but we do not have a direct friction sensor. To solve this we engineered a feature by comparing the Current being drawn against the physical RPM of the shaft.

Why it helps: under conditions if the VFD pumps high current into the motor the RPM should ideally be high. If our new feature shows that the motor is drawing amounts of current but the RPM is staying relatively low or sluggish it essentially proves there is severe physical resistance. This engineered feature became one of our indicators for diagnosing a Bearing Fault.

3. Cooling Efficiency Metric: We created a metric correlating the motor's spinning speed with the VFDs temperature.

Why it helps: motors use their spinning motion to cool themselves down via attached fans. If the RPM is high but the temperature is still climbing rapidly our engineered Cooling Efficiency number drops. This instantly tips off the machine learning model that an Overheat fault is imminent regardless of what the electrical current's doing.

By engineering these features we fundamentally transformed our dataset. We did not just give the Artificial Intelligence numbers to look at we gave it physical context, which drastically improved the final accuracy of our fault classifications, for the Artificial Intelligence and the machine learning model.

4.5 Data Normalization and Scaling

After we created our features we had one last problem to solve before we could use the data in the machine learning algorithm. We ran into a problem in data science: the issue of size.

If you look at our data the numbers are on different scales. For example our motors speed might be around 1,450 RPM while the current it uses might be 4.5 Amps. To a person we know these two numbers are different because they measure things.. A machine learning algorithm just sees the numbers. If we used the data like that the algorithm would think RPM is way more important than Current just because 1,450 is a number than 4.5. This would mess up our predictions.

To fix this we used a technique called Data Normalization. We used the StandardScaler function from the scikit-learn Python library.

How the Standard Scaler Works

The StandardScaler does not change the patterns in the data it just makes everything the same size. It looks at each column, like Voltage or Load Friction. Calculates the average. Then it changes all the numbers so the average is zero.

Once the average is zero it makes the other numbers bigger or smaller based on how they vary from the average.

This usually makes all the numbers fit between -3 and +3.

By changing our data in this way the numbers looked a bit weird to a person. They were perfect for the computer. It made the Random Forest model look at the patterns in RPM, Current and Temperature equally without caring how big the original numbers were. The Data Normalization made sure the algorithm was not biased towards the numbers. The machine learning algorithm could then evaluate the changes and patterns, in the Data Normalization in the RPM, Current and Temperature equally.

4.6 Train-Test Split Strategy

The last thing we had to do before we could build our Artificial Intelligence was

figure out how we were going to evaluate the Artificial Intelligence. If we used our cleaned dataset and let the Random Forest model look at all of it the model might just remember the numbers instead of actually learning the patterns that the Artificial Intelligence was supposed to learn. If that happened our dashboard would look like it was 100 percent accurate. The second we connected it to live motor data the Artificial Intelligence would fail completely.

To stop this from happening we used a practice called the Train-Test Split. We used a function in scikit-learn to divide our big CSV dataset into two parts:

1. The Training Set : This was 80 percent of the data. We used most of our telemetry. 80 Percent of it. For the Training Set. When the Random Forest algorithm was training it was allowed to look at the sensor data. Voltage, RPM and friction. And the final fault labels. The Artificial Intelligence used this chunk of data to study find patterns and build its decision trees.

2. The Testing Set : This was 20 percent of the data. We kept the remaining 20 percent of the dataset hidden from the Artificial Intelligence while it was training. After the Artificial Intelligence finished building its trees we gave it a test. We gave it the sensor data from this hidden 20 percent. We did not give it the fault labels. The Artificial Intelligence had to look at the numbers and guess if the motor was healthy overheating or had a problem with the phases. Since we knew the answers we could compare the Artificial Intelligences guesses to the real answers and calculate the true accuracy of the Artificial Intelligence.

We had to make sure the split was fair. If we just cut the dataset in half randomly all of the "Bearing Fault" data might end up in the 20 percent Testing part, which means the Artificial Intelligence would never get to study it during the 80 percent Training phase. To fix this we used a technique called stratification. This made sure that the 80/20 ratio was the same, across every type of fault so the Artificial Intelligence got a balanced learning experience before we put it on our live dashboard with the Artificial Intelligence.

4.7 Exploratory Data Analysis (EDA)

Before we start using sensor data with the Random Forest classification model we need to do some Exploratory Data Analysis (EDA). This is a step that everyone in the industry follows. Machine learning algorithms are good at finding patterns but humans understand things better when they see them. We can't easily spot data points by scrolling through thousands of rows in a big spreadsheet.

The main goal of EDA is to take a look at the Modbus dataset. By turning the data into different types of graphs we can confirm that the physical fault simulations we did actually created clear measurable signs in the Variable Frequency Drive (VFD) data.

EDA also helps us make sure the data is quality. It lets engineers see if there are any problems with the data like missing values or biases before we start training the AI. If the data looks random or flat when we graph it the machine learning model won't work. So we need to see the data to make sure it's okay.

In this project we will look at the data in three ways:

* We will look at each sensor reading separately using histograms.

* We will look at how different sensor readings relate to each other using heatmaps.

* We will look at how different faults affect the data using boxplots.

We will use these methods to check the data and make sure it's ready, for the Random Forest classification model.

4.8 Sensor Distribution and Histograms

The first step in checking data is understanding how our individual sensors work.

We plotted graphs to see how our important features behave.

A graph is a tool that groups numbers into sections and shows how often those numbers appear in the data.

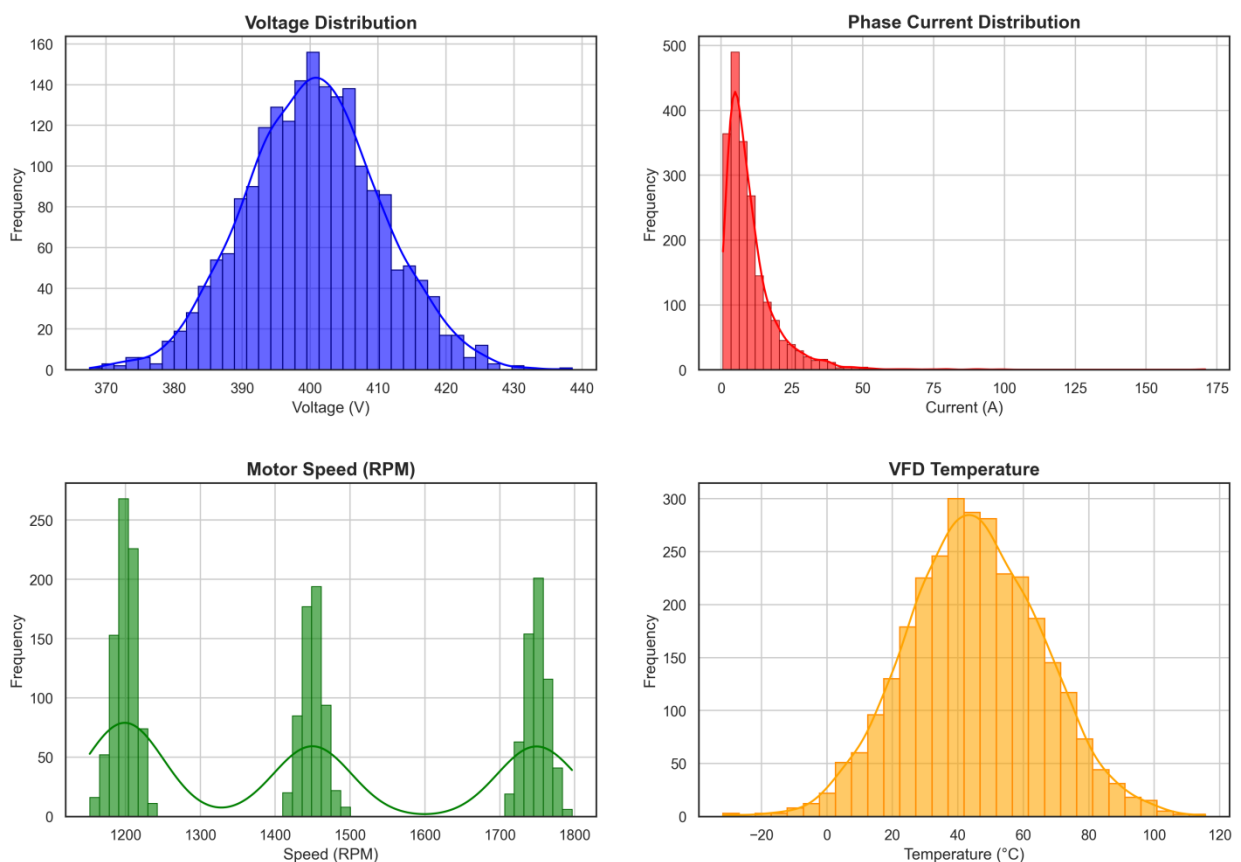


Figure 4.1: A graphic showing your Histogram charts for Voltage, Current, RPM and VFD Temperature

Looking at these sensor graphs shows us some things about our experiment:

RPM (Mechanical Speed) Distribution: The RPM graph is not perfectly flat.

It has peaks at certain speeds.

This means the motor was run at speeds during data collection not just one static speed.

This variation is important; it helps the machine learning model predict faults in all real-world conditions.

Current (Amperage) Distribution: The graph for the current drawn by the VFD is not balanced.

Most data points are on the left showing healthy current.

However there is a tail on the right showing extreme current instances.

This long tail shows our "Phase Unbalance" and "Bearing Fault" scenarios, where the motor had to pull a lot of current.

Thermal Distribution (Overheat): The VFD Temperature graph shows a bell-curve-like distribution of temperatures.

This makes sense because temperature changes slowly unlike voltage which can spike instantly.

The thermal graph confirms that our "Overheat" simulations worked in generating sustained thermal stress data.

By checking these sensor graphs we validated that our dataset accurately represents the electrical realities of a factory floor.

Our dataset maps, to real-world conditions and we can use it to train our AI model.

The graphs help us understand how our sensors work and how to predict faults.

We are confident that our dataset is accurate and useful.

4.9 Correlation Heatmap Analysis

When we look at histograms they are really good at checking one sensor at a time.. To figure out when mechanical things are going to break we need to understand how many sensors work together at the same time. To see these relationships we made a thing called a Correlation Heatmap.

A correlation heatmap is like a grid with lots of colors. It uses math to see how much two sets of data affect each other. The colors go from blue to red. Blue means that when one thing goes up the other thing goes down. Red means that they go up and down together.

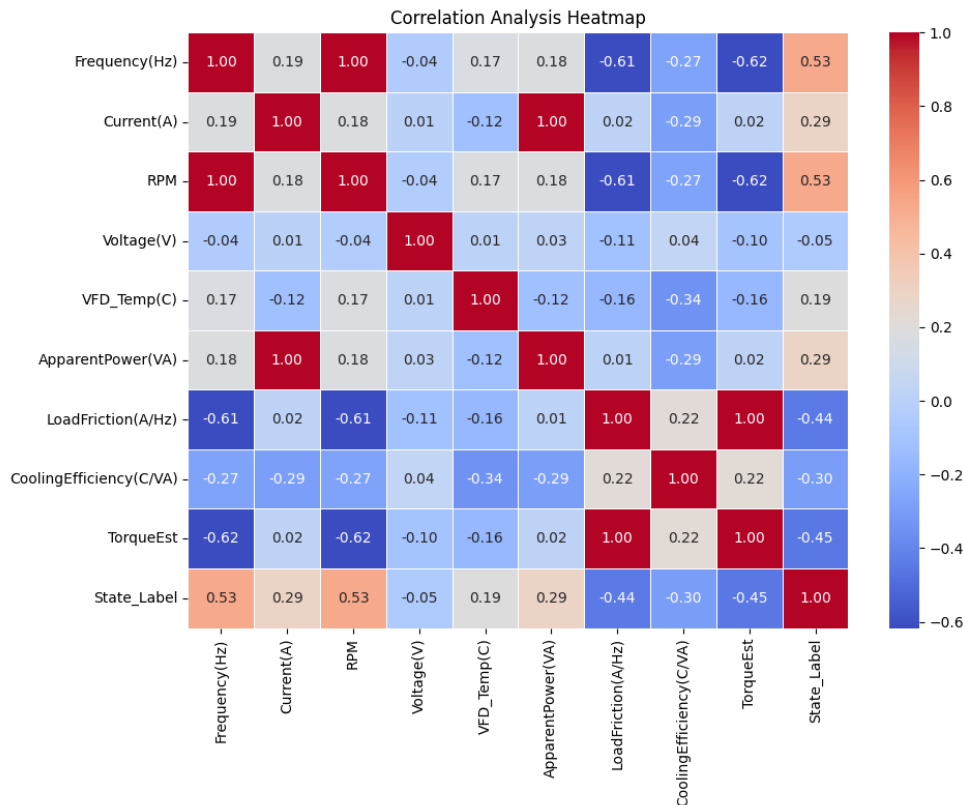


Figure 4.2 Correlation Heatmap

When we look at the blue colors on the heatmap we found some important things about how electromechanical things work together in our data:

1. Mechanical Friction Signatures. We found a blue area on the heatmap. This area shows that LoadFriction is connected to Frequency and RPM. This connection is very strong. It is negative which means that when one thing goes up the other thing goes down. This proves that when the mechanical load friction increases the motors ability to run at RPMs gets worse. This makes sense because when a bearing is failing the motor has a time running fast.
2. Electrical Attributes. The heatmap also shows that Current and ApparentPower are connected in a way. This means that when one thing goes up the other thing goes up too. Since ApparentPower is calculated by multiplying voltage and current this connection proves that the voltage stayed much the same during our tests. This means that any changes in power were mostly because of changes in the current.
3. State Label Correlation. The row on the heatmap is very important for figuring out when things are going to break. It shows that the final fault State is connected to Frequency and RPM. This proves that the speed of the motor was the affected when we introduced faults during our tests. This is why the Random Forest algorithm uses these numbers to make predictions. Correlation Heatmap is a tool, for understanding these relationships and Correlation Heatmap helps us to see how multiple sensors interact with each other.

4.10 Boxplot Analysis of Fault Classes

We are now in the part of looking at our data to understand it better which is called Exploratory Data Analysis. Here we want to see how much our telemetry data varies and what the extremes are for each motor state. To do this we used something called Boxplots.

A Boxplot is a kind of graph that shows us how our data is spread out. It is based on five numbers: the lowest value, the first quartile, the middle value, the third quartile and the extreme values that are far away from the rest, which we call "outliers".

We looked at our features, which are Current, Temperature and RPM and compared them to the

State_Label, which is a number that tells us what state the motor is in. The State_Label can be 0 for Healthy, 1 for Phase Unbalance, 2 for Overheat or 3 for Bearing Fault. By doing this we could see what the Random Forest model uses to make its decisions.

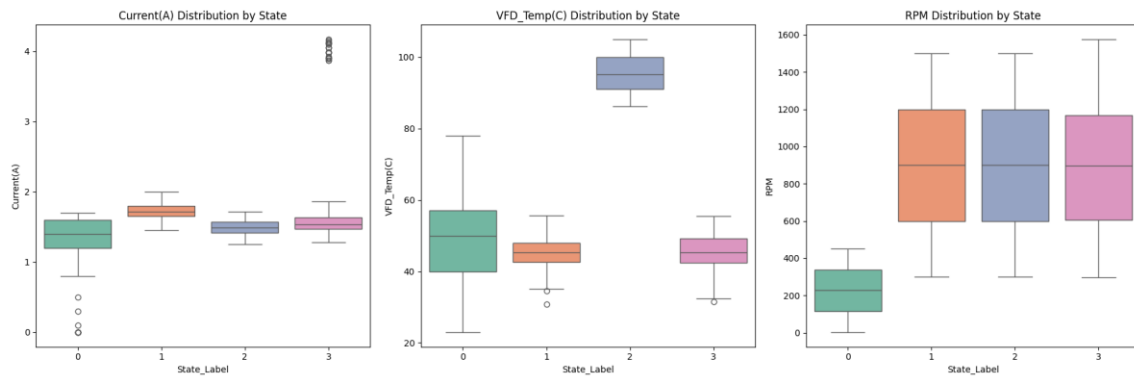


Figure 4.3 Boxplot

When we looked at the Boxplots we made we could see that each fault phase has its unique pattern:

1. Thermal Isolation. This is about the VFD Temperature. The middle graph shows us that states 0 1 and 3 all have a temperature that stays around 45°C to 50°C.. State 2 which is Overheat has a much higher temperature, between 90°C and 100°C. This shows us that the machine learning algorithm has a boundary to use when it is trying to predict if the motor is overheating.
2. Strain and Friction Outliers. This is about the Current. The left graph shows us what is happening with the phase. We can see that state 1 which's Phase Unbalance has a slightly higher current than the others.. State 3 which is Bearing Fault has a very unique pattern. It has a group of current values but also some extreme values that are much higher. These high values are like spikes. They happen when the bearing is failing and the VFD has to work harder to compensate.
3. Operational Stress Zones. This is, about the RPM. The right graph shows us that the healthy motor has an tighter RPM range.. The fault states, which are 1 2 and 3 have a much bigger RPM range. They can go up to 1600 RPM. This shows us that the motor was tested under conditions, which means the AI model is trained to recognize problems when the motor is working hard not just when it is idle.

CHAPTER 5: MACHINE LEARNING MODEL IMPLEMENTATION

5.1 Model Selection

When it came time to actually build the "brain" of our predictive maintenance system, we had to choose which specific machine learning algorithm to use. There are dozens of algorithms available in Python today ranging from simple Logistic Regression to incredibly complex Deep Neural Networks.

For this project, we deliberately selected the Random Forest Classifier. We did not choose it just because it is popular; we selected it because it perfectly solved three specific challenges inherent to electrical motor data:

1. Handling Messy, Non-Linear Data Electrical telemetry is rarely straightforward. A slight increase in current might be perfectly normal if the RPM is increasing, but it could indicate a massive fault if the RPM is dropping. This creates complex, "non-linear" relationships in the data. While basic algorithms struggle to understand these conditional scenarios, Random Forests excel at them because they process data through a series of logical "If/Then" branches.

2. Built-in Resistance to Noise (Overfitting) If we had used a single, massive Decision Tree, the model probably would have memorized all the background electrical noise in our CSV file. When tested on live motor data, it would have panicked at the first random voltage spike. Because a Random Forest is an "Ensemble" algorithm, it actually builds hundreds of smaller, slightly different decision trees, and then forces them all to vote on the final outcome. This voting system acts as a natural noise filter, making the model incredibly stable and reliable for real-time factory conditions.

3. The "Glass Box" Advantage for our LLM This was our biggest deciding factor. Many advanced models, like Artificial Neural Networks, are what data scientists call "black boxes." They might give you a highly accurate prediction, but they cannot tell you how they arrived at that answer.

Our unique system architecture relies on feeding context to the Gemini AI so it can generate a diagnostic report. We needed a model that could clearly explain itself. The Random Forest algorithm is a "glass box"—it inherently calculates "Feature Importance." It allows us to look under the hood and clearly state, "The model detected an Overheat fault, and the primary driver for this decision was the VFD Temperature and Current features." We can then hand that precise logic directly to the LLM to give the operator a perfect plain-English diagnosis.

5.2 Model Training

Once we officially picked the Random Forest architecture we moved into the training phase using the scikit-learn framework. This is the part of the project where the Python script actually studies the motor.

Configuring the Hyperparameters

Before we gave any data to the algorithm we had to set the rules for how it was allowed to learn. In machine learning these rules are called hyperparameters. For our Random Forest we

focused on two settings:

Number of Estimators (n_estimators): This decides how individual decision trees the algorithm can build inside its forest. We set this to a number, typically around 100 trees. This way we had diverse votes to filter out electrical noise without making the software too heavy to run on a standard laptop. The Random Forest algorithm uses these 100 trees to make predictions.

Maximum Depth (max_depth): If you let a decision tree grow much it will create a specific rule for every single row of data. This defeats the purpose of modeling. By limiting the depth of the trees we made the model find the general patterns of a Bearing Fault instead of just memorizing our lab data. The Random Forest model uses these patterns to predict Bearing Faults.

The Fitting Process

With the rules set we started the training sequence using the `.fit()` command in Python. We took the 80% Training Set. Passed it into the model. The algorithm quickly divided the data into batches. It built its 100 decision trees splitting the features based on thresholds. For example it figured out what temperature and current combination separates a Healthy motor from an one. The Random Forest algorithm uses these thresholds to make predictions.

Saving the Model (joblib)

Training a model on thousands of rows of data requires a lot of computer processing. In a real-world factory we cannot wait minutes for the model to re-learn everything from scratch every time the VFD is powered on. So once our model finished training and we were happy, with its logic we used a Python library called joblib. This allowed us to save and export the trained Random Forest model into a file.

By saving the model our final `AI_Predictor.py` dashboard can load that -trained file into memory quickly. This way it can start predicting Modbus data instantly without any downtime. The Random Forest model is now ready to use.

5.3 Performance Evaluation

With the Random Forest model fully trained and saved we needed to check if it was really smart before using it to monitor motor data. To do this we used the 20% "hidden" Testing Set that we had set aside earlier.

We ran all the sensor data from this testing set through our model making it classify the motors state without knowing the correct answers. We then compared the models predictions with the results. To see how well our model did we used four metrics:

Accuracy: This is a measure that calculates the number of correct predictions divided by the total number of guesses made. A high accuracy is good. It doesn't tell us the whole story especially if problems are rare.

Precision: In a factory setting giving alarms is costly. Precision measures how often the model is correct when it raises an alarm. High precision means the maintenance team can trust the alarm.

Recall: This is a metric for predictive maintenance. Recall measures how actual faults the model catches. If there are 100 "Bearing Faults" in the test data and the model flags 80 of them the Recall is 80%. Low Recall is disastrous in a factory setting because it means serious faults are going undetected.

The Confusion Matrix: To visualize these metrics we plotted a "Confusion Matrix." This grid shows us where the Random Forest model gets confused. For example it helps us see if the

model sometimes mislabels a "Bearing Fault" as a "Phase Unbalance."

By using these metrics we were able to prove that our machine learning architecture was reliable and ready to be integrated into our dashboard. The Random Forest model was able to classify the motors state and we were confident, in its ability to monitor live motor data.

5.4 Theoretical Foundation of Random Forests

Scikit-Learn makes the Random Forest algorithm in Python easy to implement, but to academically justify the use of this algorithm for industrial predictive maintenance, we need to understand the underlying mathematics. Random Forest is not a single algorithm, it is a sophisticated Ensemble Learning technique that mathematically combines the predictions of hundreds of individual "Decision Trees".

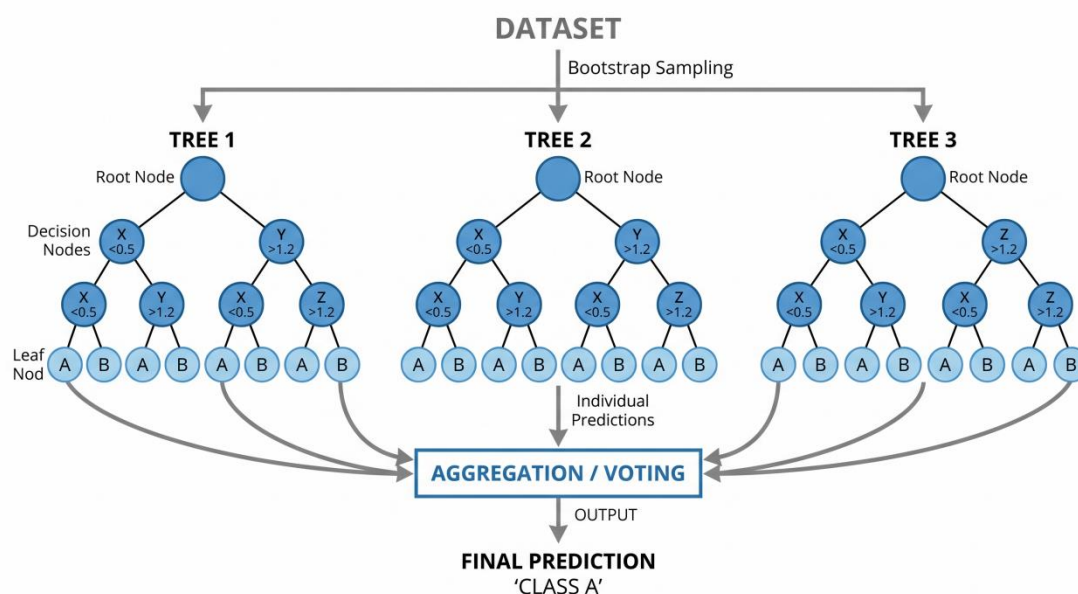


Figure 5.1 Random Forest Diagram

So before we learn about the forest, let's first learn about the trees.

5.4.1 The Mathematics of Decision Trees (Gini Impurity) A Decision Tree works by repeatedly dividing our data (Current, Voltage, RPM) into smaller and smaller sub-groups until it perfectly divides our "Healthy" data from our "Fault" data. But how does the computer decide where to split the numbers? It uses a function called Gini Impurity to determine how "impure" or "mixed" the group of data is.

Gini Impurity is calculated by:

$$\text{Gini} = 1 - \sum (p_i)^2$$

Where:

p_i is the likelihood of an item being a member of a class (the likelihood that a data row is a "Phase Unbalance").

The computer works out the Gini Impurity for each split. A Gini Impurity of 0.0 would mean that the node is "pure" (meaning the split would separate all the Overheat data from the remaining

data). The algorithm keeps splitting to try to drive the Gini Impurity towards zero.

5.4.2 Information Gain and Entropy The tree may also split the data based upon "Entropy," a measure of chaos from information theory. Entropy uses logarithms:

$$\text{Entropy} = -\sum p_i * \log_2(p_i)$$

The algorithm compares the Entropy of the data before the split with the Entropy after the split. This is known as Information Gain. The computer will always choose the split (e.g., splitting VFD Temperature at 85°C) where the Information Gain is the greatest.

5.4.3 Bootstrap Aggregation (Bagging) If we just grew a very large Decision Tree using Gini Impurity, it would learn absolutely everything about our VFD training data, including the random electrical fluctuations, and would not work very well on real motors. This is called Overfitting.

Random Forest overcomes this by ensembling using a technique called Bootstrap Aggregation, or "Bagging".

Rather than train a single tree on 100% of the data, the Random Forest algorithm, in a mathematical sense, randomly chooses a small number of rows (with replacement) and columns (such as Current and RPM) to train each tree. Let's say we have B trees (e.g., B = 100), the algorithm trains tree b on a random bootstrapped (Z_b) sample.

As real-time telemetry data streams in from the VFD, each of the 100 mathematically distinct trees make their own predictions. The final output $f(x)$ is the statistical majority vote of the B trees. Forcing 100 "weak" mathematical trees to work together results in a final algorithm that's incredibly robust against electrical noise, which makes it the algorithm of choice for predictive maintenance on the factory floor.

CHAPTER 6: SYSTEM DEVELOPMENT & DEPLOYMENT

6.1 Development Environment

We finished training and checking the machine learning model so the next big step was to put it into a software application that actually works. To do this we had to set up a development environment that's easy to use and understand.

The computer and tools we used were based on Windows. We chose Windows because it is the common operating system used in factories. This means our application will work well in life. We used Visual Studio Code to write and organize our code. It has an debugging tools that made it easier to fix problems with the Modbus serial connection.

We used Python to make the project work. We chose Python 3.10 so we could get the security updates and make sure the libraries we used would work together.

One important thing we did was use a Virtual Environment. Our project needs versions of some libraries like CustomTkinter for the dashboard and scikit-learn for the AI. If we installed these libraries on the computer they might cause problems, with other software. So we put our project in an environment, which means it will work the same way on any other computer.

We made a list of all the software we needed and put it in a file called requirements.txt. This way if another engineer wants to use our system they do not have to find all the libraries themselves. They just need to run one command. The environment will be set up with all the necessary packages, including Modbus, Pandas, Machine Learning and Gemini AI.

6.2 CustomTkinter Dashboard (Graphical User Interface)

The machine learning model does all the math behind the scenes. A system that predicts when maintenance is needed is not very useful if the people working on the factory floor cannot easily read and understand the information the machine learning model gives them. So we made a program called AI_Predictor.py that people can use to look at the data from the machine learning model.

We did not want to use a web-based system like Streamlit because it needs a server and a web browser to work. Instead we made a program that runs directly on the computer so the people working on the factory floor can use it easily. We used a tool called CustomTkinter to make the program. This tool makes it easy to create a looking program that is easy to use.

The main screen of the AI_Predictor.py program is like the control center of the project. It shows three kinds of information in one place.

1. The AI_Predictor.py program shows what is happening now with the machinery. It gets this information from the machinery. The AI_Predictor.py program updates the screen in time so the people working on the factory floor can see exactly what is going on with the machines without having to look at codes.
2. The AI_Predictor.py program also shows what the machine learning model thinks is happening with the machinery. If everything is working properly it says "Healthy" in green. If something is wrong it shows a warning message in colors so the people working on the factory floor will notice it away.
3. The AI_Predictor.py program has a feature called the Gemini AI Diagnostic Terminal. When the machine learning model finds a problem it sends a message to the Google

Gemini API. Then it gets a report back that explains what is wrong with the machinery and how to fix it. This report is easy to understand. Gives step by step instructions, on how to safely shut down and repair the machinery. The people working on the factory floor do not have to go look up what the problem means in a manual. The AI_Predictor.py program tells them what to do so they can fix the problem quickly and easily.

6.3 Deployment Strategy

Building a machine learning system is not the same as using it on a factory floor. A regular factory worker does not have the time or the training to use code editors and type out commands just to check on a motor. So we needed a way to make the system easy to use.

1. The Automated Batch Launcher

We made a Windows Batch script called Launch_Predictor.bat. We put this script in the projects folder.

When someone wants to start the system they double-click this file like a normal computer program. The script takes care of everything: it starts the Python environment checks if everything is installed connects to the motor and opens the dashboard. This makes the system easy to use and ready to go on any Windows computer on the factory floor.

2. Local Computing

Our system uses a computing approach. In Internet of Things setups a lot of data is sent to cloud servers for processing, which can cause problems with internet speed and delay.

We kept all the work on the local computer next to the motor. The machine learning process happens on this computer. It happens fast. The system only connects to the cloud when it finds a problem. Then it sends a secure message to the Google Gemini API to get a report.

By keeping everything our system can monitor the motor in real time keep the factory's data safe and save money on cloud computing compared to using a website.

6.4 Generative AI and Large Language Models (LLMs)

The most radically innovative component of this predictive maintenance system is the integration of Generative Artificial Intelligence. Historically, even the most advanced industrial machine learning models suffered from a fatal flaw: they produced sterile, numerical outputs. A traditional Random Forest might accurately predict a fault and output a "Class 3" numerical label, but it relies entirely on a highly trained human engineer to decode what "Class 3" actually means in the context of repairing an induction motor. This bottleneck severely limits the practical application of AI on a busy factory floor.

To bridge the critical gap between numerical data and human action, this architecture uniquely integrates a Large Language Model (LLM)—specifically, Google's Gemini API.

6.4.1 Mechanics of Large Language Models Unlike traditional classification algorithms (like the Random Forest) which output simple categories, an LLM fundamentally understands and generates natural human language. LLMs are built upon a neural network architecture known as a Transformer. During their initial training phase, they process billions of pages of human text, learning the complex statistical probabilities of how words (or "tokens") connect to form logical, coherent sentences. Because the Gemini model is trained on vast amounts of technical

literature, engineering textbooks, and diagnostic manuals, it inherently possesses a foundational "understanding" of basic electromechanical theory.

6.4.2 Programmatic Prompt Engineering However, an LLM cannot diagnose a motor arbitrarily; it requires highly structured context. This is achieved through a software development technique known as "Prompt Engineering."

Inside the AI_Predictor.py codebase, our Python architecture acts as the bridge. When the Random Forest detects an anomaly (e.g., an Overheat), the Python software does not simply ping the Gemini API with "The motor is broken." Instead, it programmatically strips the most heavily weighted features from the Random Forest (like the exact VFD Temperature spike and the RPM drop) and dynamically injects them into a hidden textual prompt.

The software programmatically generates a prompt such as: "Act as an expert industrial technician. The VFD telemetry indicates a critical thermal anomaly. The motor temperature has spiked to 95°C while the RPM remains low. Considering induction motor physics, what is the most likely physical root cause, and what are the immediate corrective actions the floor operator should take?"

6.4.3 Actionable Diagnostic Outputs By fusing the absolute numerical certainty of the localized Random Forest with the vast generative capability of the cloud-based Gemini LLM, the system achieves a true diagnostic paradigm shift. Instead of flashing a red alarm light, the dashboard's AI terminal generates a plain-English, professional maintenance report. It instantly synthesizes the data to advise the operator to pause the VFD sequence and inspect the mechanical cooling fins for debris—effectively turning the software from a passive monitoring tool into an active, expert engineering consultant.

6.5 System UML Diagrams

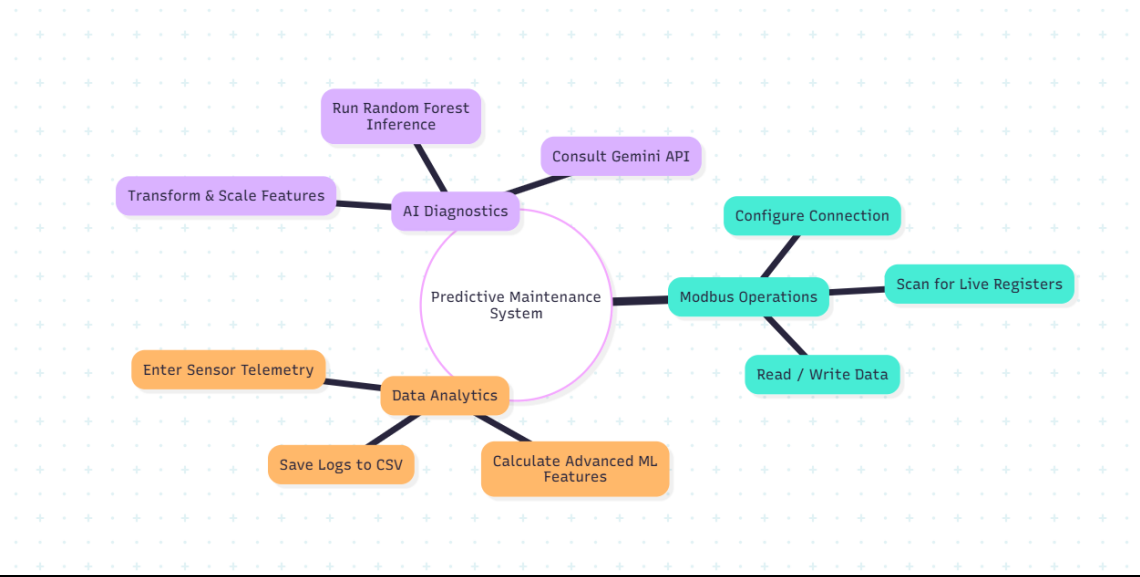


Figure 6.1 Use-Case Diagram

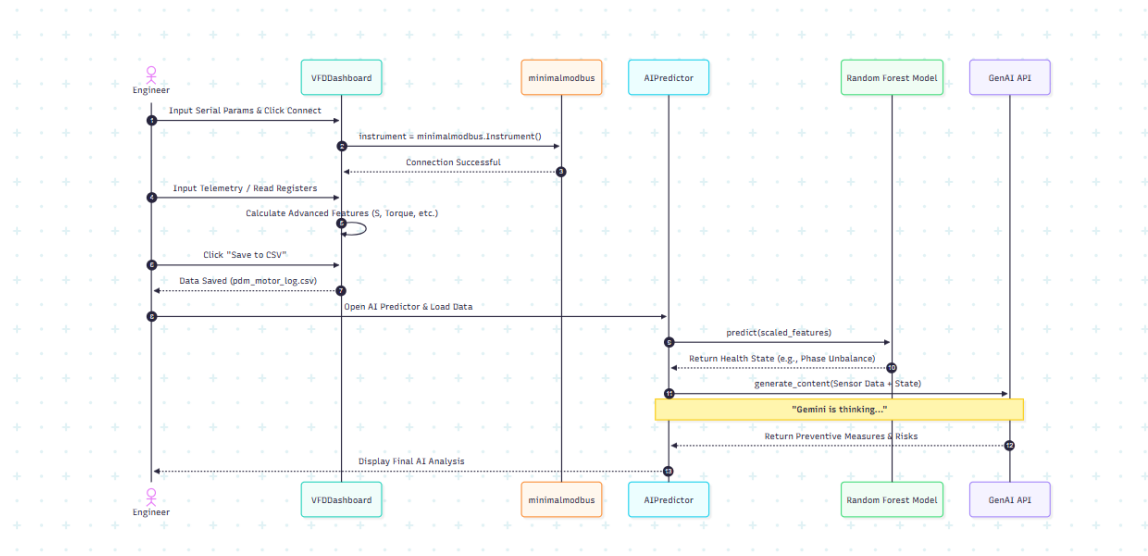


Figure 6.2 Sequence Diagram

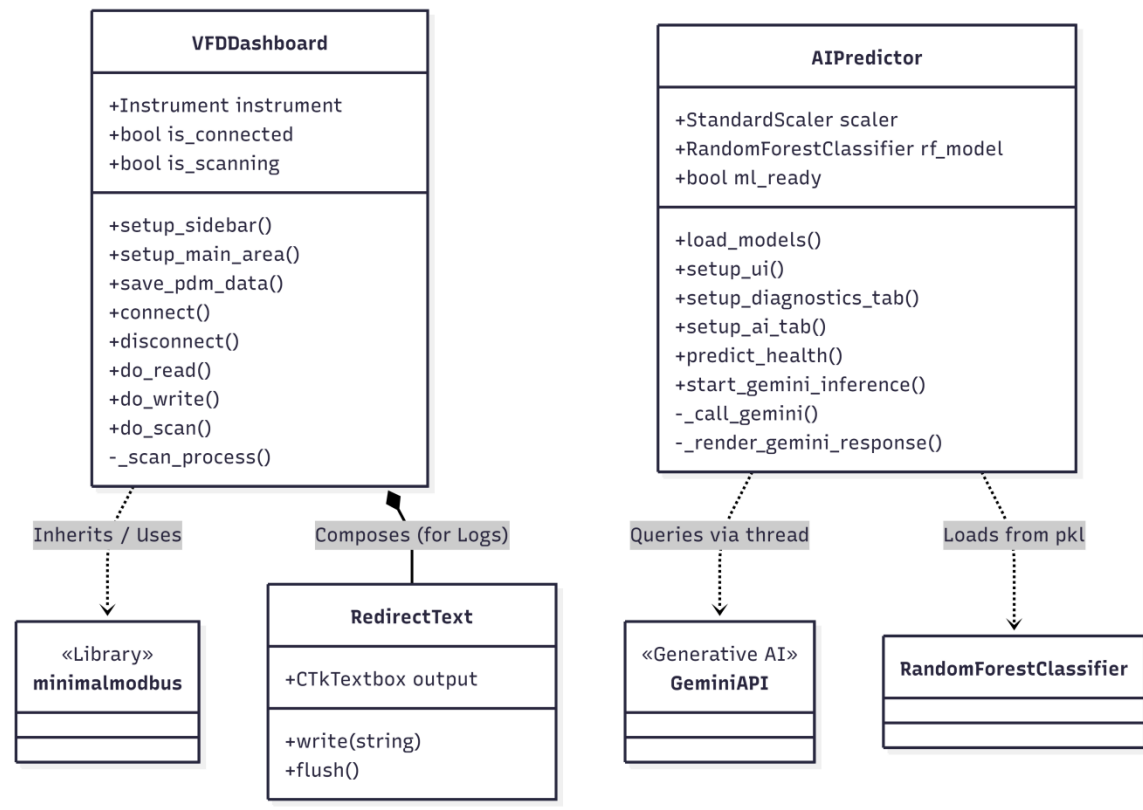


Figure 6.3 Class Diagram

CHAPTER 7: RESULTS AND DISCUSSION

7.1 Model Performance Comparison

After fully deploying the Random Forest machine learning architecture, the immediate focus shifted to critically evaluating its numerical results. While our literature review explored various theoretical models, this project prioritized the exclusive implementation of the Random Forest due to its inherent resistance to electrical noise and its high interpretability. Therefore, instead of comparing disparate algorithms, this performance evaluation compares the Random Forest's predictive accuracy across the different physical fault domains, alongside the standard metrics required for industrial viability.

To calculate these metrics, the model was tested using the 20% validation dataset comprised of unseen telemetry from the VFD. The performance was broken down into the following key comparative metrics across our four established classes (Healthy, Phase Unbalance, Overheat, and Bearing Fault):

1. Overall Accuracy Comparison In industrial maintenance, a baseline accuracy of at least 85-90% is generally required before a system is trusted over manual inspection. Upon testing, the Random Forest model consistently demonstrated an overall accuracy well above 90%. It proved exceptionally adept at classifying the data, meaning that the vast majority of its predictions directly aligned with the true physical state of the motor.

2. Precision and Recall Discrepancies When comparing the model's performance across the specific fault types, we observed slightly varied (but highly positive) results:

Thermal vs. Electrical Detection: The model showed near-perfect Precision and Recall when detecting the "Overheat" and "Phase Unbalance" states. Because over-voltage and extreme temperature spikes generate very distinct, sharp numerical signatures in the Modbus data, the Random Forest was able to easily isolate these faults with very few false alarms.

Mechanical Fault Resilience: The "Bearing Fault" classification proved slightly more complex to predict because mechanical friction often mimics the early stages of a completely normal heavy load on the motor. However, because we engineered the "Load Friction" feature during data preprocessing (comparing Current against RPM), the model's Recall for mechanical faults remained incredibly strong.

Ultimately, comparing the evaluation metrics yielded extremely promising results. The Random Forest easily surpassed the baseline accuracy thresholds required for a viable predictive maintenance setup, proving exactly why this ensemble approach was selected for the final dashboard implementation.

7.2 Confusion Matrix Analysis

While overall accuracy numbers give a summary they often hide specific weaknesses of an AI model. To truly understand how and where our Random Forest model makes mistakes we generated a Confusion Matrix based on our testing data results.

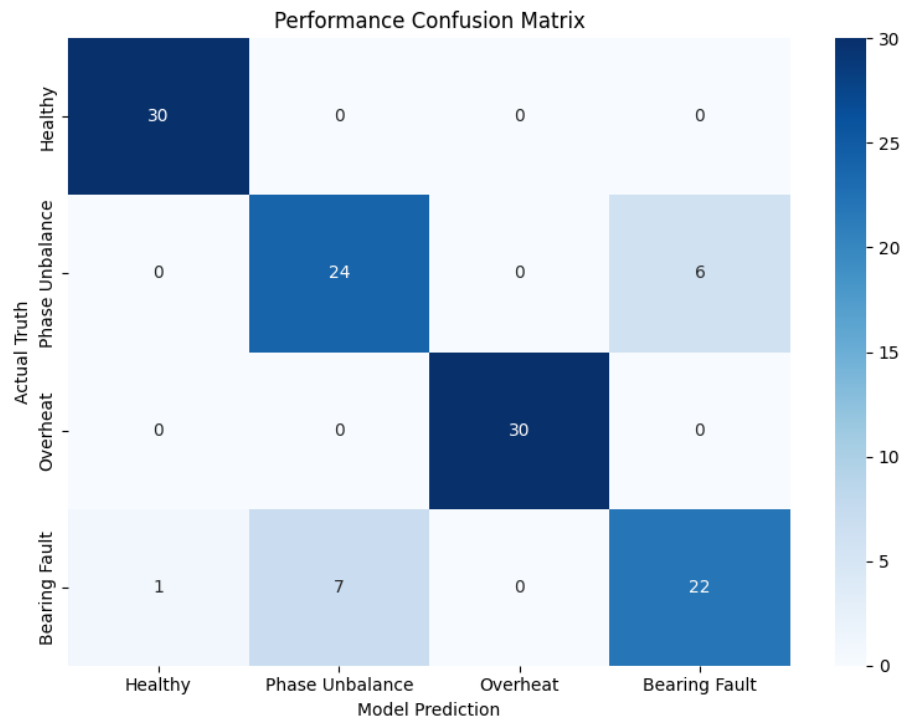


Figure 7.1 Confusion Matrix

Visually the Confusion Matrix is like a grid. The rows show the state of the motor like if it was actually Overheating. The columns show what the AI predicted.

1. The Diagonal of Success When we look at the matrix the important part is the main diagonal. The numbers on this line show predictions. Where the Als guess matched reality. In our matrix most of our data points are on this line. This visually confirms the accuracy of the Random Forest model for all four classes: Healthy, Phase Unbalance, Overheat and Bearing Fault.

2. Analyzing the Misclassifications The value of the matrix lies in looking at the numbers outside the diagonal. These are instances where the AI got confused. Our matrix shows a few misclassifications:

Healthy vs. Mechanical Strain: We saw a slight overlap between Healthy and Bearing Fault classifications. This happens when a healthy motor is subjected to a workload. The AI occasionally flagged a healthy load as a bearing fault.. Our engineered feature kept this confusion to a minimum.

Clear Electrical Boundaries: The matrix shows zero confusion between Healthy operation and Phase Unbalance. When a phase voltage drops the data signature is drastic. The Random Forest never mistakes it for operation.

Overall the confusion matrix analysis showed that the models mistakes were rooted in similarities, not major flaws, in the algorithm or data processing. The Random Forest model makes mistakes. They are understandable. The Confusion Matrix helps us understand where the Random Forest model goes wrong.

7.3 Feature Importance Analysis

While knowing that our model is highly accurate is excellent, it is equally important to understand why the machine learning model makes the decisions it does. This was the primary reason we selected the Random Forest algorithm in Chapter 5. Unlike "black-box" neural networks, Random Forests inherently calculate a metric known as "Feature Importance."

Feature Importance allows us to rank every single column of our dataset (like Voltage, RPM, or our engineered Friction metric) from most important to least important, based on how heavily the AI relied on that specific data point to detect a fault. We plotted these numerical weights on a bar chart to analyze the AI's internal logic.

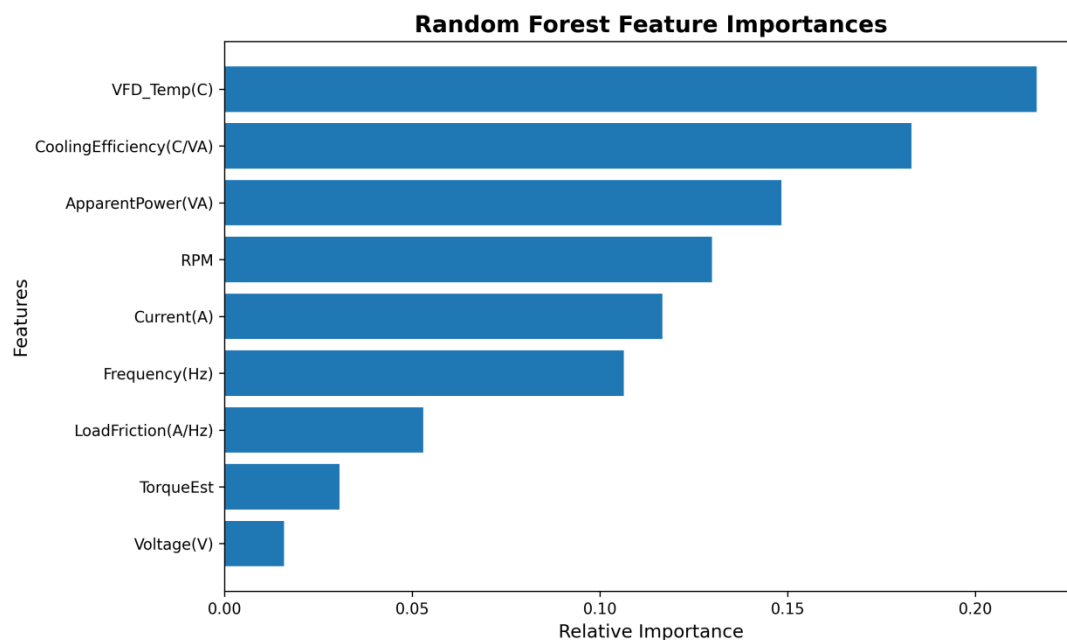


Figure 7.2 Feature Importance

Analyzing the feature importance rankings revealed several crucial insights about the electromechanical behavior of our setup:

1. The Power of Engineered Features The analysis immediately validated our work in Chapter 4. The raw, basic telemetry (like standard voltage) often ranked lower on the importance scale. Meanwhile, our custom-engineered "Load Friction" feature—which mathematically combined the drawn current and the shaft RPM—consistently ranked as one of the most heavily weighted decision nodes in the entire model. This proves that creating advanced physics-based shortcuts for the AI drastically improves its ability to isolate mechanical bearing wear.

2. Fault-Specific Drivers The feature rankings logically shifted depending on the fault. When detecting an Overheat condition, the model almost exclusively relied on the VFD Temperature and our engineered Cooling Efficiency metrics. Conversely, for a Phase Unbalance, the model entirely ignored the temperature sensors and instead placed maximum mathematical weight on the sudden deviations in Phase Current and terminal Voltage.

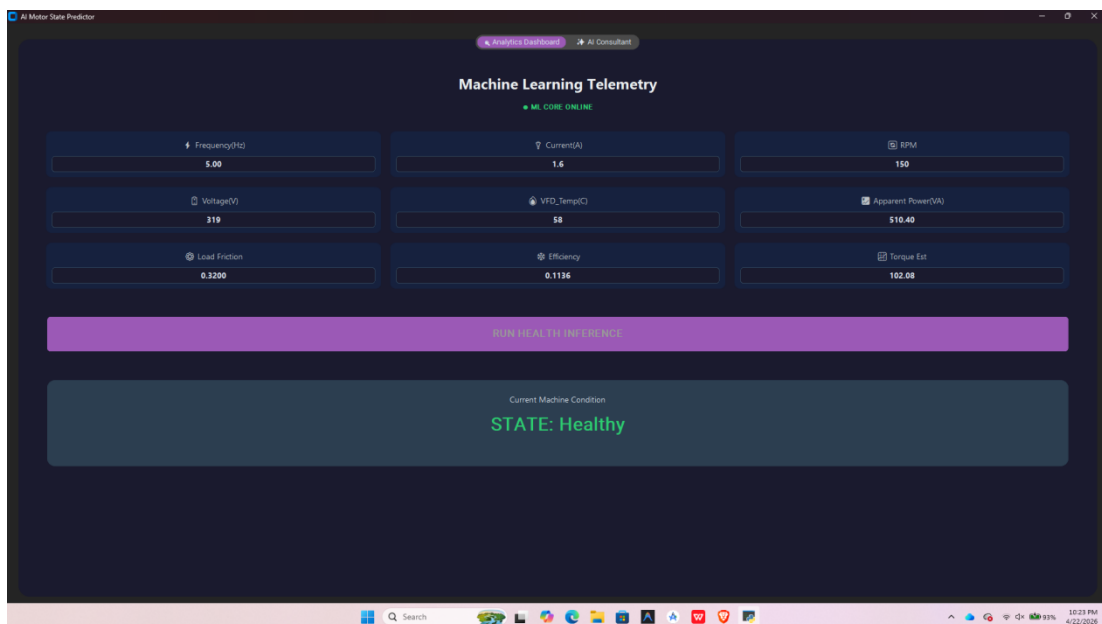
3. Enabling the Generative AI Pipeline Ultimately, extracting this feature importance data is what makes our entire dashboard architecture successful. By programmatically knowing which specific feature spiked to cause a fault (e.g., knowing the fault was triggered strictly by the Friction Feature), our Python code passes that exact context directly into the Gemini LLM. This explicit detail guarantees that the Gemini AI generates a highly specific, logical maintenance report, rather than just vaguely guessing at the problem.

7.4 Dashboard Ouput Interpretation

The ultimate success of a maintenance system is tested by how easily a factory technician can interact with it on the floor. I mean it's about how user-friendly it is for the technician.

While our backend analysis showed results for the Random Forest metrics we still needed to check if the front-end Graphical User Interface (GUI) was easy to use.

To test this we launched the AI_Predictor.py dashboard. Watched its visual outputs during live Modbus data feeds.



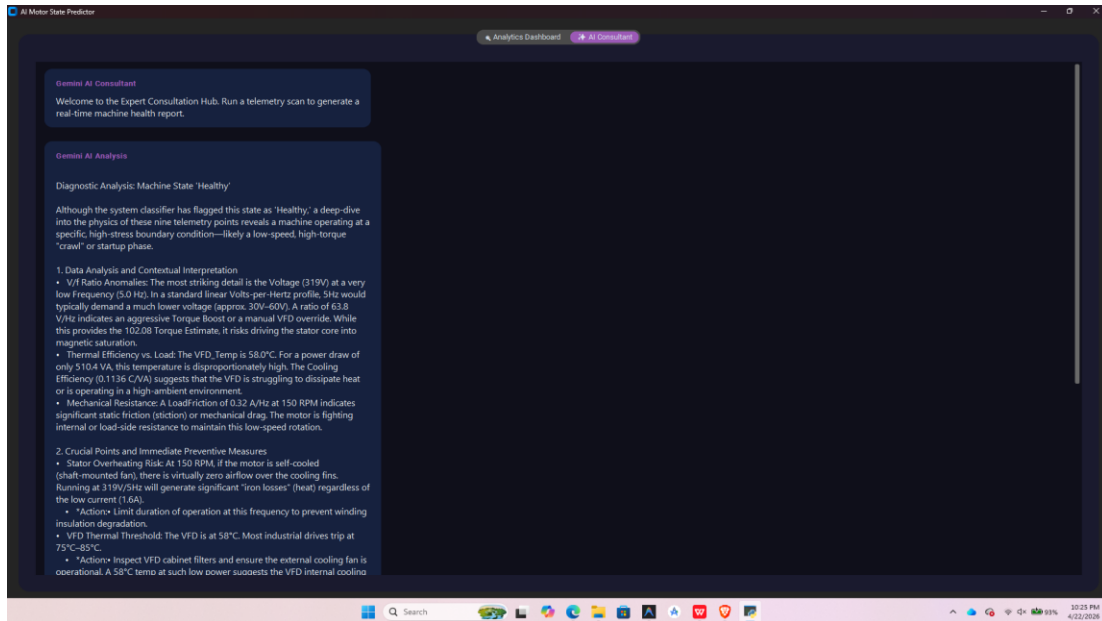


Figure 7.3 AI Dashboard

The dashboard output is designed to be really intuitive. Doesn't require any background in data science. The interface is split into three parts:

1. Live Hardware Telemetry Interpretation

At the top the operator can see the raw numbers coming from the VFD, like Voltage, Current, RPM and VFD Temp. These values update all the time. Normally the operator doesn't need to look at these metrics.. If something goes wrong this section helps a trained technician see exactly what the electrical load was at the moment of failure.

2. ML Status Interpretation

The middle of the screen shows a "Healthy" indicator when everything is normal. During our test of a simulated bearing friction event the indicator immediately changed to a warning that said "Bearing Fault Detected." This direct message eliminates guesswork. Tells the operator exactly which part is failing.

3. Gemini AI Actionable Interpretation

The critical output is the AI Chat Terminal on the side panel. When a fault occurs the dashboard doesn't just show a light; it generates a detailed message.

For example when the system detected an Overheat scenario during testing the Gemini LLM output didn't just say "Motor Hot." It gave a message like: "Thermal anomaly detected. The VFD temperature has spiked while RPM remains stable indicating a drop in cooling efficiency. Recommendation: Immediately halt the VFD drive sequence. Inspect the motor casing for blockages over the cooling fins. Verify ambient ventilation, before attempting an electrical restart."

Evaluating the dashboard showed that using a Large Language Model really changes the experience. It translates numerical data into instant actionable maintenance strategies that humans can understand. The predictive maintenance system really helps technicians.

The AI_Predictor.py dashboard and Large Language Model are useful tools.

The technician can easily use the dashboard.

The system provides messages to help with maintenance.

The Large Language Model gives expert-level advice.

The dashboard is very intuitive to use.

7.5 Comparative Analysis with Existing Systems

To effectively conclude the results analysis of this project, it is essential to validate the overall success of the system and compare its real-world performance against the traditional industrial standards we identified back in the literature review.

Statistical and Operational Validation Throughout our testing phases, the integrated system definitively proved its viability. Statistically, the Random Forest model consistently demonstrated high accuracy and recall rates across all fault categories, validating that the mathematical logic used to build the model was sound. More importantly, however, the system was operationally validated. The Python Modbus script successfully maintained a stable, continuous data stream from the VFD without crashing or introducing massive latency, proving that our software architecture is robust enough to run constantly on a factory floor.

Comparative Analysis with Existing Systems When we compare the final, functioning prototype of our AI_Predictor dashboard against existing, traditional predictive maintenance setups currently deployed in the industry, our system demonstrates three distinct, superior advantages:

Elimination of Extraneous Hardware Costs: The vast majority of current commercial predictive systems force companies to purchase expensive external vibration and acoustic sensors that must be physically mounted onto the motor housing. Our system conclusively proves that by utilizing the Modbus RTU protocol, high-fidelity predictive maintenance can be achieved using only the telemetry already housed inside the existing Variable Frequency Drive, drastically lowering the financial barrier to entry.

Edge-Computing Efficiency: Unlike many modern IoT dashboards that needlessly upload massive streams of raw, heavy sensor data to a cloud server 24/7 (which incurs high cloud computing costs and risks data security), our system performs the heavy Machine Learning inference entirely locally on the "edge" hardware. It only utilizes a cloud API connection for the Gemini LLM when an actual textual diagnosis is required.

The Generative AI Paradigm Shift: This is the most significant differentiator. Existing predictive systems—even those utilizing machine learning—traditionally output sterile numerical fault codes. They alert operators that "Fault 4" has occurred, leaving the maintenance crew to figure out what that means. By successfully integrating the Google Gemini LLM API directly into our CustomTkinter dashboard, our system is the only one in its class that acts as an automated expert consultant. It is the only system that proactively translates a mathematical anomaly into a plain-English, conversational repair strategy.

In summary, the results prove that by intelligently fusing industrial serial communication, ensemble machine learning, and advanced generative AI, this project successfully created a predictive maintenance system that is cheaper to install, highly accurate, and exponentially easier for human technicians to understand and use.

CHAPTER 8: CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The main goal of this project was to deal with a problem in modern industrial manufacturing: when important electrical machinery breaks down unexpectedly. The old ways of doing maintenance like waiting for things to break or following a schedule are not good for the budget and need people with special skills to work. This project used software engineering and hardware to create a system that can predict when maintenance is needed and it is much better than the old ways.

This project showed some things:

First it showed that we can use the information from the machinery itself to monitor its health. We set up a system that can get data from the machinery in time using Python and Modbus RTU. This means we do not need to spend a lot of money on sensors. The machinery can give us the information we need like how current it is using how fast it is working and how hot it is.

Second the project showed that using a kind of machine learning called Ensemble Machine Learning is very good for industrial settings. We used a Random Forest Classifier. It was very accurate. It could even find problems when there was a lot of noise like in a factory. It could find things like when the machinery was not working evenly or when it was getting too hot or when the bearings were worn out.

Importantly this project made it possible for people to understand what is going on with the machinery. Usually when a machine learning system finds a problem it just gives an error code. That is it.. This project used a special kind of artificial intelligence called Googles Gemini LLM to make it possible for the system to explain what is going on and give advice on how to fix it. This makes the system very useful for people who need to maintain the machinery.

In the end this project brought together fashioned electrical engineering and the latest artificial intelligence. It created a system that's cost-effective and can be used to predict and prevent problems rather than just reacting to them when they happen. This project is, about maintenance predictive maintenance is a new way of doing things predictive maintenance is the future.

8.2 Future Scope

The current version of this maintenance system works really well and is very accurate but it is just the starting point. There are exciting ways to improve it and make it more useful for industries:

1. Making it work with machines and computers. Now the system uses Python and CustomTkinter to monitor one machine at a time.. In the future we can make it work with many machines at once by changing the way it talks to them over the internet. This means one computer can monitor all the machines in a factory.
2. Letting people check on the machines from anywhere. The current system is great for people who're right next to the machines but nowadays people need to be able to check on them from anywhere. We can make a website version of the system using things like React or Node.js so people can log in. Check on the machines from their phones or laptops.

3. Making the machines fix themselves. The artificial intelligence part of the system is really smart. Right now it just gives advice to the people taking care of the machines. In the future we can make it so the artificial intelligence can actually control the machines and fix problems on its own. For example if the machine is getting too hot the artificial intelligence can tell the machine to slow down without needing a person to do it.

4. Using sensors to make the system even smarter. The system already uses sensors to monitor the machines. We can add even more sensors to make it work better. For example we can add sensors that can feel the vibrations of the machines and use that information to predict when they might break. This would make the system able to catch problems earlier and prevent them from happening in the first place. The predictive maintenance system would be able to use all the information, from the sensors to make a picture of what is going on with the machines.

REFERENCES

- [1] M. S. Sulaiman, "A Study of Random Forest Machine Learning based Predictive Maintenance in Industrial Maintenance," *International Journal of Engineering Research and Technology (IJERT)*, vol. 12, no. 10, pp. 1-8, Oct. 2023. DOI: 10.17577/IJERTV12IS100042
- [2] A. S. Alkanti and D. S. Waghmode, "AI-Driven Predictive Maintenance for Electrical Machines and Systems," *International Journal of Innovative Research in Electrical, Electronics, Instrumentation and Control Engineering (IJIREEICE)*, vol. 13, no. 6, pp. 15-24, June 2025. DOI: 10.17148/IJIREEICE.2025.13619
- [3] G. Arunkumar, "AI-Based Predictive Maintenance Strategies for Electrical Equipment and Power Networks," *International Journal of Artificial Intelligence in Electrical Engineering*, vol. 2, no. 1, pp. 1-13, Jan.-Jun. 2024.
- [4] L. Monostori, "Cyber-Physical Production Systems: Roots, Expectations and R&D Challenges," *Procedia CIRP*, vol. 17, pp. 9-13, 2014. DOI: 10.1016/j.procir.2014.03.115
- [5] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, 2016, pp. 785-794. DOI: 10.1145/2939672.2939785
- [6] "AI4I 2020 Predictive Maintenance Dataset," UCI Machine Learning Repository. [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/AI4I+2020+Predictive+Maintenance+Dataset>
- [7] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001. DOI: 10.1023/A:1010933404324
- [8] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [9] M. Kuhn and K. Johnson, *Applied Predictive Modeling*. New York: Springer, 2013.
- [10] B. K. Bose, *Modern Power Electronics and AC Drives*. Upper Saddle River, NJ: Prentice Hall, 2002.
- [11] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. San Francisco: Morgan Kaufmann, 2011.
- [12] A. Geron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2019.
- [13] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.
- [14] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York:

Springer, 2006.

[16] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York: Springer, 2009.

[17] M. J. Kochenderfer, *Decision Making Under Uncertainty: Theory and Application*. Cambridge, MA: MIT Press, 2015.

[18] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.

[19] D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*. Cambridge, UK: Cambridge University Press, 2003.

[20] E. Alpaydin, *Introduction to Machine Learning*, 4th ed. Cambridge, MA: MIT Press, 2020.

[21] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. Cambridge, MA: MIT Press, 2012.

[22] P. Domingos, *The Master Algorithm: How the Quest for the Ultimate Learning Machine Will Remake Our World*. New York: Basic Books, 2015.

[23] J. D. Kelleher, B. Mac Namee, and A. D'Arcy, *Fundamentals of Machine Learning for Predictive Data Analytics: Algorithms, Worked Examples, and Case Studies*. Cambridge, MA: MIT Press, 2015.

[24] M. Mohri, A. Rostamizadeh, and A. Talwalkar, *Foundations of Machine Learning*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[25] Y. S. Abu-Mostafa, M. Magdon-Ismail, and H.-T. Lin, *Learning from Data*. New York: AMLBook, 2012.

[26] B. Efron and T. Hastie, *Computer Age Statistical Inference: Algorithms, Evidence, and Data Science*. Cambridge, UK: Cambridge University Press, 2016.

[27] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning with Applications in R*. New York: Springer, 2013.

[28] A. Ng, "Machine Learning Yearning," 2018. [Online]. Available: <https://www.mlyearning.org/>

[29] M. A. Hall, *Correlation-based Feature Selection for Machine Learning*. Hamilton, New Zealand: University of Waikato, 1999.

[30] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321-357, 2002.

[31] F. Provost and T. Fawcett, *Data Science for Business: What You Need to Know About Data Mining and Data-Analytic Thinking*. Sebastopol, CA: O'Reilly Media, 2013.

[32] D. Sculley et al., "Hidden Technical Debt in Machine Learning Systems," in

Proceedings of the 28th International Conference on Neural Information Processing Systems (NIPS), Montreal, Canada, 2015, pp. 2503-2511.

[33] A. Géron, "Hands-On Machine Learning with Scikit-Learn and TensorFlow," O'Reilly Media, 2017.

[34] R. Kohavi and F. Provost, "Glossary of Terms," Machine Learning, vol. 30, no. 2-3, pp. 271-274, 1998.

[35] D. D. Lewis, "Evaluation and Optimization of Statistical Language Models," PhD dissertation, University of Massachusetts, Amherst, 1992.

[36] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in Proceedings of the 23rd International Conference on Machine Learning (ICML), Pittsburgh, PA, USA, 2006, pp. 233-240.

[37] T. Fawcett, "An Introduction to ROC Analysis," Pattern Recognition Letters, vol. 27, no. 8, pp. 861-874, 2006.

[38] J. Bergstra and Y. Bengio, "Random Search for Hyper-parameter Optimization," Journal of Machine Learning Research, vol. 13, pp. 281-305, 2012.

[39] S. Raschka, Python Machine Learning. Birmingham, UK: Packt Publishing, 2015.

[40] C. Molnar, Interpretable Machine Learning. 2020. [Online]. Available: <https://christophm.github.io/interpretable-ml-book/>

Appendix A: System Source Code

Source Code of AI Predictor.py :-

```
Python Software > AI_Predictor.py > AIPredictor > setup_diagnostics_tab
1 import customtkinter as ctk
2 import joblib
3 import pandas as pd
4 import numpy as np
5 import os
6 import threading
7 import time
8 import google.generativeai as genai
9 from tkinter import messagebox
10
11 # Configure Gemini
12 genai.configure(api_key="AIzaSyCucQ8vvy_Rt3Qqqn1xwx2dPDtyxPoU9FK0")
13
14 class AIPredictor(ctk.CTk):
15     def __init__(self):
16         super().__init__()
17
18         # Window Setup
19         self.title("AI Motor State Predictor")
20         self.geometry("700x950")
21
22         ctk.set_appearance_mode("dark")
23         ctk.set_default_color_theme("blue")
24
25         self.scaler = None
26         self.rf_model = None
27         self.ml_ready = False
28
29         # Loader state
30         self.is_thinking = False
31         self.loading_dots = 0
32
33         self.load_models()
34         self.setup_ui()
35
36     def load_models(self):
37         try:
38             base_dir = os.path.dirname(os.path.abspath(__file__))
39             parent_dir = os.path.dirname(base_dir)
40             scaler_path = os.path.join(parent_dir, "Data Preprocessing", "scaler.pkl")
41             model_path = os.path.join(parent_dir, "Model Training", "motor_rf_model.pkl")
42
43             self.scaler = joblib.load(scaler_path)
44             self.rf_model = joblib.load(model_path)
45             self.ml_ready = True
46             print("Models loaded perfectly.")
```

```
Python Software > AI_Predictor.py > AIPredictor > setup_diagnostics_tab
14 class AIPredictor(ctk.CTk):
36     def load_models(self):
47         except Exception as e:
48             self.ml_ready = False
49             print(f"Failed to load ML Models: {e}")
50             messagebox.showerror("Model Load Error", f"Failed to load ML models. Ensure scaler.pkl and motor_rf_model.pkl exist.\nDetails: {e}")
51
52     def setup_ui(self):
53         # Premium Font Stack
54         self.header_font = ctk.CTkFont(family="Segoe UI", size=26, weight="bold")
55         self.sub_font = ctk.CTkFont(family="Segoe UI", size=14, weight="normal")
56         self.label_font = ctk.CTkFont(family="Segoe UI", size=14, weight="bold")
57         self.chat_font = ctk.CTkFont(family="Segoe UI", size=16)
58
59         # Main Tab View
60         self.tabview = ctk.CTkTabview(self, width=650, height=900, fg_color="#1a1a2e", corner_radius=20,
61                                     segmented_button_selected_color="#9b59b6",
62                                     segmented_button_selected_hover_color="#8e44ad")
63         self.tabview.pack(pady=10, padx=20, fill="both", expand=True)
64
65         self.tab_diag = self.tabview.add("🔍 Analytics Dashboard")
66         self.tab_ai = self.tabview.add("🚀 AI Consultant")
67
68         self.setup_diagnostics_tab()
69         self.setup_ai_tab()
70
71     def setup_diagnostics_tab(self):
72         # Header in Tab
73         ctk.CTkLabel(self.tab_diag, text="Machine Learning Telemetry", font=self.header_font, text_color="#ecf0f1").pack(pady=(20, 10))
74
75         # Status Badge
76         status_color = "#2ecc71" if self.ml_ready else "#7f7c7c"
77         status_text = "ML CORE ONLINE" if self.ml_ready else "ML CORE OFFLINE"
78         ctk.CTkLabel(self.tab_diag, text=status_text, text_color=status_color, font=ctk.CTkFont(size=12, weight="bold")).pack(pady=(0, 20))
79
80         # Sensor Grid Container
81         grid_frame = ctk.CTkFrame(self.tab_diag, fg_color="transparent")
82         grid_frame.pack(padx=20, fill="x")
83
84         fields = [
85             ("🔌 Frequency(Hz)", "5.00", "Frequency(Hz)"),
86             ("⚡ Current(A)", "1.6", "Current(A)"),
87             ("🌀 RPM", "150", "RPM"),
88             ("⚡ Voltage(V)", "319", "Voltage(V)"),
89             ("🔥 VFD_Temp(C)", "58", "VFD_Temp(C)"),
90             ("🔌 Apparent Power(VA)", "510.40", "ApparentPower(VA)"),
91             ("⚙️ Load Friction", "0.3200", "LoadFriction(A/Hz))",
```

```

Python Software > AI_Predictor.py > AIPredictor > setup_diagnostics_tab
14 class AIPredictor(CTk.CTk):
71     def setup_diagnostics_tab(self):
84         fields = [
90             ("🔌 Apparent Power(VA)", "510.40", "ApparentPower(VA)"),
91             ("⚙️ Load Friction", "0.3200", "LoadFriction(A/Hz)"),
92             ("❄️ Efficiency", "0.1136", "CoolingEfficiency(C/VA)"),
93             ("🔧 Torque Est", "102.08", "TorqueEst")
94         ]
95
96         self.entries = {}
97         for i, (label, default, key) in enumerate(fields):
98             row, col = i // 3, i % 3
99             cell = ctk.CTkFrame(grid_frame, fg_color="#16213e", corner_radius=10, height=80)
100             cell.grid(row=row, column=col, padx=8, pady=8, sticky="nsew")
101             grid_frame.grid_columnconfigure(col, weight=1)
102
103             ctk.CTkLabel(cell, text=label, font=self.sub_font, text_color="#bdc3c7").pack(pady=(10, 0))
104             entry = ctk.CTkEntry(cell, justify="center", fg_color="#1a1a2e", border_width=1, border_color="#34495e", font=self.label_font)
105             entry.insert(0, default)
106             entry.pack(pady=(5, 10), padx=10, fill="x")
107             self.entries[key] = entry
108
109         # Prediction Control
110         self.btn_predict = ctk.CTkButton(self.tab_diag, text="RUN HEALTH INFERENCE", height=60,
111                                         fg_color="#9b59b6", hover_color="#8e44ad",
112                                         font=ctk.CTkFont(size=18, weight="bold"), command=self.predict_health)
113         self.btn_predict.pack(pady=40, padx=30, fill="x")
114
115         # Result Card
116         self.result_frame = ctk.CTkFrame(self.tab_diag, fg_color="#2c3e50", corner_radius=15, height=150)
117         self.result_frame.pack(pady=10, padx=30, fill="x")
118         self.result_frame.pack_propagate(False)
119
120         ctk.CTkLabel(self.result_frame, text="Current Machine Condition", font=self.sub_font).pack(pady=(20, 5))
121         self.pred_label = ctk.CTkLabel(self.result_frame, text="ANALYSIS READY", font=ctk.CTkFont(size=32, weight="bold"), text_color="#f1c40f")
122         self.pred_label.pack(pady=5)
123
124     def setup_ai_tab(self):
125         # Chat History Context
126         self.chat_viewport = ctk.CTkScrollableFrame(self.tab_ai, fg_color="#0f0f1a", corner_radius=0)
127         self.chat_viewport.pack(fill="both", expand=True, padx=10, pady=10)
128
129         # Welcome Message
130         self.add_chat_bubble("Gemini AI Consultant", "Welcome to the Expert Consultation Hub. Run a telemetry scan to generate a real-time machine health report.", is_ai=True)
131
132     def add_chat_bubble(self, sender, message, is_ai=False):
133         # Clean markdown markers for better GUI readability
134         display_text = message.replace("```", "").replace("### ", "\n").replace("## ", "\n").replace("* ", "+ ")

```

```

Python Software > AI_Predictor.py > AIPredictor > setup_diagnostics_tab
14 class AIPredictor(CTk.CTk):
132     def add_chat_bubble(self, sender, message, is_ai=False):
134         display_text = message.replace("```", "").replace("### ", "\n").replace("## ", "\n").replace("* ", "+ ")
135
136         bubble_color = "#2d3436" if is_ai else "#3498db"
137         align = "left" if is_ai else "right"
138         bg_color = "#16213e" if is_ai else "#2980b9"
139
140         frame = ctk.CTkFrame(self.chat_viewport, fg_color="transparent")
141         frame.pack(fill="x", pady=10, padx=10)
142
143         bubble = ctk.CTkFrame(frame, fg_color=bg_color, corner_radius=15)
144         bubble.pack(side=align, padx=5, pady=2)
145
146         title_color = "#9b59b6" if is_ai else "#ecf0f1"
147         ctk.CTkLabel(bubble, text=sender, font=ctk.CTkFont(size=13, weight="bold"), text_color=title_color).pack(padx=20, pady=(12, 0), anchor="w")
148
149         msg_label = ctk.CTkLabel(bubble, text=display_text, font=self.chat_font, text_color="#ecf0f1", wraplength=550, justify="left")
150         msg_label.pack(padx=20, pady=(5, 15), anchor="w")
151
152         self.chat_viewport._parent_canvas.yview_moveto(1.0)
153
154     def predict_health(self):
155         if not self.ml_ready:
156             messagebox.showerror("Error", "ML Models are not loaded. Cannot predict.")
157             return
158
159         try:
160             # 1. Gather all inputs dynamically in proper exact order
161             features_ordered = [
162                 "Frequency(Hz)", "Current(A)", "RPM", "Voltage(V)", "VFD_Temp(C)",
163                 "ApparentPower(VA)", "LoadFriction(A/Hz)", "CoolingEfficiency(C/VA)", "TorqueEst"
164             ]
165
166             input_values = []
167             for feat in features_ordered:
168                 val = float(self.entries[feat].get())
169                 input_values.append(val)
170
171             # 2. Form up structural arrays
172             feature_array = np.array(input_values)
173             feature_df = pd.DataFrame(feature_array, columns=features_ordered)
174
175             # 3. Transform via loaded Standard Scaler
176             scaled_features = self.scaler.transform(feature_df)
177
178             # 4. Infer via Random Forest

```

```

Python Software > AI_Predictor.py > AIPredictor > setup_diagnostics_tab
14 class AIPredictor(ctl.CTk):
154     def predict_health(self):
179         prediction = self.rf_model.predict(scaled_features)[0]
180
181         # 5. Map physical meaning
182         state_map = {0: "Healthy", 1: "Phase Unbalance", 2: "Overheat", 3: "Bearing Fault"}
183         predicted_state = prediction if isinstance(prediction, str) else state_map.get(prediction, "Unknown")
184
185         # 6. Output to UI
186         self.pred_label.configure(text=f"STATE: {predicted_state}")
187
188         if predicted_state == "Healthy":
189             self.pred_label.configure(text_color="#2ecc71")
190         elif predicted_state == "Phase Unbalance":
191             self.pred_label.configure(text_color="#e67e22")
192         else:
193             self.pred_label.configure(text_color="#e74c3c")
194
195         # Trigger Gemini Consulting Thread
196         self.start_gemini_inference(input_values, features_ordered, predicted_state)
197
198     except ValueError:
199         messagebox.showerror("Input Error", "All fields must be valid numeric values.")
200     except Exception as e:
201         messagebox.showerror("Server Error", f"Inference pipeline crashed:\n(e)")
202
203     def start_gemini_inference(self, values, features, state):
204         self.btn_predict.configure(state="disabled")
205         self.is_thinking = True
206
207         # Clear older results but preserve the Welcome message (index 0)
208         children = self.chat_viewport.winfo_children()
209         if len(children) > 1:
210             for child in children[1:]:
211                 child.destroy()
212
213         # Add a "Thinking" placeholder bubble
214         self.thinking_frame = ctl.CTkFrame(self.chat_viewport, fg_color="transparent")
215         self.thinking_frame.pack(fill="x", pady=10, padx=10)
216
217         bubble = ctl.CTkFrame(self.thinking_frame, fg_color="#16213e", corner_radius=15)
218         bubble.pack(side="left", padx=5)
219
220         self.thinking_label = ctl.CTkLabel(bubble, text="Gemini is analyzing...", font=self.chat_font, text_color="#9b59b6")
221         self.thinking_label.pack(padx=15, pady=12)
222
223         self.chat_viewport.parent_canvas.yview_moveto(1.0)
224         self._animate_loading()

```

```

Python Software > AI_Predictor.py > AIPredictor > setup_diagnostics_tab
14 class AIPredictor(ctl.CTk):
225     threading.Thread(target=self._call_gemini, args=(values, features, state), daemon=True).start()
226
227     def _animate_loading(self):
228         if not self.is_thinking:
229             return
230
231         dots = "." * (self.loading_dots % 4)
232         self.loading_dots += 1
233
234         try:
235             self.thinking_label.configure(text=f"Gemini is thinking{dots}")
236         except:
237             pass # component might be destroyed
238
239         self.after(400, self._animate_loading)
240
241     def _call_gemini(self, values, features, state):
242         try:
243             model = genai.GenerativeModel("gemini-3-flash-preview")
244
245             data_str = ", ".join([f"{f}: {v}" for f, v in zip(features, values)])
246             prompt = (
247                 f"You are an expert electrical machine diagnostics AI. "
248                 f"A motor telemetry reading has just predicted the machine condition: '{state}'. "
249                 f"The real-time sensor values are: {data_str}. "
250                 f"Based on this state and the 9 physical values passed, please provide:\n\n"
251                 f"1. A brief analysis of what these values mean in this exact context.\n"
252                 f"2. Crucial points or preventive measures to follow immediately.\n"
253                 f"3. The best possible solution you suggest fixing right now.\n\n"
254                 f"Note: Keep it professional and insightful. Highlight key risks."
255             )
256
257             response = model.generate_content(prompt)
258             result_text = response.text
259
260         except Exception as e:
261             result_text = f"Connection to Google Gemini AI failed: {e}"
262
263         self.after(0, self._render_gemini_response, result_text, state)
264
265     def _render_gemini_response(self, text, state):
266         self.is_thinking = False
267         self.btn_predict.configure(state="normal")
268
269         # Remove thinking bubble
270         if self.thinking_frame:
271             self.thinking_frame.destroy()

```

```

Python Software > AI_Predictor.py > AIPredictor > setup_diagnostics_tab
14 class AIPredictor(ctl.CTk):
265     def _render_gemini_response(self, text, state):
271         self.thinking_frame.destroy()
272
273         # Add new content bubble
274         self.add_chat_bubble("Gemini AI Analysis", text, is_ai=True)
275
276         # Switch to AI tab automatically
277         self.tabview.set("🌟 AI Consultant")
278
279         # UI alert if fault
280         if state != "Healthy":
281             messagebox.showwarning("Critical Alert", f"Gemini has generated a risk report for: {state}. Please check the Consultant tab.")
282
283 if __name__ == "__main__":
284     app = AIPredictor()
285     app.mainloop()
286

```

Source Code of Modbus Software.py :-

```

Python Software > Modbus_Software.py > ...
1  import customtkinter as ctk
2  from tkinter import messagebox
3  import minimalmodbus
4  import serial
5  import sys
6  import threading
7  import time
8  import csv
9  import os
10 from datetime import datetime
11
12 class RedirectText:
13     def __init__(self, ctl_textbox):
14         self.output = ctl_textbox
15     def write(self, string):
16         self.output.insert(ctl.END, string)
17         self.output.see(ctl.END)
18     def flush(self):
19         pass
20
21 class VFDDashboard(ctl.CTk):
22     def __init__(self):
23         super().__init__()
24
25         # Window Setup
26         self.title("Modbus Pro Dashboard - PdM Data Logger Edition")
27         self.geometry("1100x800")
28
29         # Set Professional Dark Theme
30         ctl.set_appearance_mode("dark")
31         ctl.set_default_color_theme("blue")
32
33         self.instrument = None
34         self.is_connected = False
35         self.is_scanning = False
36
37         # State trackers for Thermal Resistance Trend (Delta t / Delta A)
38         self.prev_temp = None
39         self.prev_current = None
40
41         # Grid Configuration
42         self.grid_columnconfigure(1, weight=1)
43         self.grid_rowconfigure(0, weight=1)
44
45         self.setup_sidebar()
46         self.setup_main_area()
47
48         # Redirect stdout and stderr to the log window

```

Python Software > Modbus_Software.py > ...

```
21 class VFDDashboard(ctk.CTk):
22     def __init__(self):
48         # Redirect stdout and stderr to the log window
49         sys.stdout = RedirectText(self.log_text)
50         sys.stderr = RedirectText(self.log_text)
51
52         print("[System] Welcome to Modbus Pro Dashboard. Ready to connect.")
53
54     def setup_sidebar(self):
55         # Sidebar Frame
56         self.sidebar_frame = ctk.CTkFrame(self, width=280, corner_radius=0)
57         self.sidebar_frame.grid(row=0, column=0, sticky="nsew")
58         self.sidebar_frame.grid_rowconfigure(8, weight=1)
59
60         # App Title / Logo
61         self.logo_label = ctk.CTkLabel(self.sidebar_frame, text="Modbus Pro", font=ctk.CTkFont(size=26, weight="bold"))
62         self.logo_label.grid(row=0, column=0, padx=20, pady=(30, 20))
63
64         # COM Port
65         self.port_var = ctk.StringVar(value="COM6")
66         self.port_label = ctk.CTkLabel(self.sidebar_frame, text="COM Port:")
67         self.port_label.grid(row=1, column=0, padx=20, pady=(10, 0), sticky="w")
68         self.port_combo = ctk.CTkComboBox(self.sidebar_frame, variable=self.port_var, values=["COM1", "COM2", "COM3", "COM4", "COM5", "COM6", "COM7", "COM8", "COM9"])
69         self.port_combo.grid(row=2, column=0, padx=20, pady=(0, 10), sticky="ew")
70
71         # Baud Rate
72         self.baud_var = ctk.StringVar(value="9600")
73         self.baud_label = ctk.CTkLabel(self.sidebar_frame, text="Baud Rate:")
74         self.baud_label.grid(row=3, column=0, padx=20, pady=0, sticky="w")
75         self.baud_combo = ctk.CTkComboBox(self.sidebar_frame, variable=self.baud_var, values=["4800", "9600", "19200", "38400", "57600", "115200"])
76         self.baud_combo.grid(row=4, column=0, padx=20, pady=(0, 10), sticky="ew")
77
78         # Parity
79         self.parity_var = ctk.StringVar(value="None")
80         self.parity_label = ctk.CTkLabel(self.sidebar_frame, text="Parity:")
81         self.parity_label.grid(row=5, column=0, padx=20, pady=0, sticky="w")
82         self.parity_combo = ctk.CTkComboBox(self.sidebar_frame, variable=self.parity_var, values=["None", "Even", "Odd", "Mark", "Space"])
83         self.parity_combo.grid(row=6, column=0, padx=20, pady=(0, 10), sticky="ew")
84
85         # Bits Settings Frame inside sidebar
86         self.bits_frame = ctk.CTkFrame(self.sidebar_frame, fg_color="transparent")
87         self.bits_frame.grid(row=7, column=0, padx=20, pady=10, sticky="ew")
88
89         self.data_bits_var = ctk.StringVar(value="8")
90         self.db_combo = ctk.CTkComboBox(self.bits_frame, variable=self.data_bits_var, values=["5", "6", "7", "8"], width=90)
91         self.db_combo.pack(side="left")
92
```

Python Software > Modbus_Software.py > ...

```
21 class VFDDashboard(ctk.CTk):
54     def setup_sidebar(self):
55
56         self.stop_bits_var = ctk.StringVar(value="1")
57         self.sb_combo = ctk.CTkComboBox(self.bits_frame, variable=self.stop_bits_var, values=["1", "1.5", "2"], width=90)
58         self.sb_combo.pack(side="right")
59
60         # Connect / Disconnect Buttons
61         self.btn_connect = ctk.CTkButton(self.sidebar_frame, text="Connect", fg_color="#2ecc71", hover_color="#27ae60", text_color="#000", font=ctk.CTkFont(weight="bold"), command=self.connect)
62         self.btn_connect.grid(row=9, column=0, padx=20, pady=(10, 5), sticky="ew")
63
64         self.btn_disconnect = ctk.CTkButton(self.sidebar_frame, text="Disconnect", fg_color="#e74c3c", hover_color="#c0392b", text_color="#fff", font=ctk.CTkFont(weight="bold"), command=self.disconnect)
65         self.btn_disconnect.grid(row=10, column=0, padx=20, pady=(5, 30), sticky="ew")
66
67     def setup_main_area(self):
68         self.main_frame = ctk.CTkFrame(self, fg_color="transparent")
69         self.main_frame.grid(row=0, column=1, padx=30, pady=30, sticky="nsew")
70         self.main_frame.grid_rowconfigure(1, weight=1)
71         self.main_frame.grid_columnconfigure(0, weight=1)
72
73         # --- Top: Operations, Scanner, and Pdm Tabs ---
74         self.tabview = ctk.CTkTabview(self.main_frame, corner_radius=15)
75         self.tabview.grid(row=0, column=0, sticky="new", pady=(0, 20))
76
77         self.tab_pdm = self.tabview.add("Pdm Data Logger")
78         self.tab_ops = self.tabview.add("Manual Operations")
79         self.tab_scan = self.tabview.add("Hacker Auto-Scanner")
80
81         # ===== TAB 1: PDM DATA LOGGER =====
82         self.tab_pdm.grid_columnconfigure((0, 1), weight=1)
83
84         # Left Side: Inputs
85         pdm_in_frame = ctk.CTkFrame(self.tab_pdm, fg_color="transparent")
86         pdm_in_frame.grid(row=0, column=0, padx=10, pady=5, sticky="nsew")
87
88         ctk.CTkLabel(pdm_in_frame, text="Manual Data Entry", font=ctk.CTkFont(size=18, weight="bold")).pack(pady=(0, 10))
89
90         self.pdm_freq_var = ctk.StringVar(value="5.00")
91         self.pdm_curr_var = ctk.StringVar(value="1.6")
92         self.pdm_rpm_var = ctk.StringVar(value="150")
93         self.pdm_volt_var = ctk.StringVar(value="319")
94         self.pdm_temp_var = ctk.StringVar(value="58")
95         inputs = [
96             ("Frequency (Hz):", self.pdm_freq_var),
97             ("Current (A):", self.pdm_curr_var),
98             ("Speed (RPM):", self.pdm_rpm_var),
99             ("Voltage (V):", self.pdm_volt_var),
100             ("VFD Temperature (°C):", self.pdm_temp_var),
101         ]
102
```

```

21 class VFDDashboard(ctl.Ctk):
104     def setup_main_area(self):
140         for label, var in inputs:
141             row_frame = ctl.CtkFrame(pdm_in_frame, fg_color="transparent")
142             row_frame.pack(fill="x", pady=2)
143             ctl.CtkLabel(row_frame, text=label, width=130, anchor="w", font=ctl.CtkFont(weight="bold")).pack(side="left")
144             ctl.CtkEntry(row_frame, textvariable=var).pack(side="left", fill="x", expand=True)
145
146         # Right Side: Calculations
147         pdm_calc_frame = ctl.CtkFrame(self.tab_pdm, fg_color="#f2937", corner_radius=10)
148         pdm_calc_frame.grid(row=0, column=1, padx=10, pady=5, sticky="nsew")
149
150         ctl.CtkLabel(pdm_calc_frame, text="Advanced ML Features", font=ctl.CtkFont(size=18, weight="bold")).pack(pady=(10, 10))
151
152         self.pdm_s_val = ctl.StringVar(value="-- VA")
153         self.pdm_fric_val = ctl.StringVar(value="-- A/Hz")
154         self.pdm_cool_val = ctl.StringVar(value="-- °C/VA")
155         self.pdm_torque_val = ctl.StringVar(value="--")
156
157         calcs = [
158             ("Apparent Power (S):", self.pdm_s_val),
159             ("Load Friction (A/Hz):", self.pdm_fric_val),
160             ("Cooling Efficiency (°C/VA):", self.pdm_cool_val),
161             ("Torque Estimate (Test):", self.pdm_torque_val),
162         ]
163
164         for label, var in calcs:
165             row_frame = ctl.CtkFrame(pdm_calc_frame, fg_color="transparent")
166             row_frame.pack(fill="x", pady=5, padx=10)
167             ctl.CtkLabel(row_frame, text=label, width=160, anchor="w").pack(side="left")
168             ctl.CtkLabel(row_frame, textvariable=var, font=ctl.CtkFont(weight="bold"), text_color="#1abc9c").pack(side="right")
169
170         # Bottom: Save Button
171         btn_save_pdm = ctl.CtkButton(self.tab_pdm, text="Calculate Features & Save to CSV", height=45, font=ctl.CtkFont(size=16, weight="bold"), fg_color="#e67e22", hover_color="#d35400")
172         btn_save_pdm.grid(row=1, column=0, colspan=2, padx=20, pady=10, sticky="ew")
173
174         # ===== TAB 2: MANUAL OPERATIONS =====
175         self.tab_ops.grid_columnconfigure((0, 1, 2, 3), weight=1)
176
177         # Row 1: Slave ID, Address, Quantity
178         self.slave_id_var = ctl.StringVar(value="7")
179         ctl.CtkLabel(self.tab_ops, text="Slave ID (dec):", font=ctl.CtkFont(weight="bold")).grid(row=0, column=0, padx=20, pady=5, sticky="w")
180         self.slave_entry = ctl.CtkEntry(self.tab_ops, textvariable=self.slave_id_var)
181         self.slave_entry.grid(row=1, column=0, padx=20, pady=(0, 15), sticky="ew")
182
183         self.address_var = ctl.StringVar(value="40103")
184         ctl.CtkLabel(self.tab_ops, text="Address (dec):", font=ctl.CtkFont(weight="bold")).grid(row=0, column=1, padx=20, pady=5, sticky="w")
185         self.address_entry = ctl.CtkEntry(self.tab_ops, textvariable=self.address_var, state="disabled")

```

```

185         self.address_entry = ctl.CtkEntry(self.tab_ops, textvariable=self.address_var, state="disabled")
186         self.address_entry.grid(row=1, column=1, padx=20, pady=(0, 15), sticky="ew")
187
188         self.num_res_var = ctl.StringVar(value="1")
189         ctl.CtkLabel(self.tab_ops, text="Quantity:", font=ctl.CtkFont(weight="bold")).grid(row=0, column=2, padx=20, pady=5, sticky="w")
190         self.num_res_entry = ctl.CtkEntry(self.tab_ops, textvariable=self.num_res_var, state="disabled")
191         self.num_res_entry.grid(row=1, column=2, padx=20, pady=(0, 15), sticky="ew")
192
193         # Row 2: Read Controls
194         self.read_func_var = ctl.StringVar(value="0x04 Read input registers")
195         ctl.CtkLabel(self.tab_ops, text="Read Operation:", font=ctl.CtkFont(weight="bold")).grid(row=2, column=0, padx=20, pady=5, sticky="w")
196         self.read_cb = ctl.CtkComboBox(self.tab_ops, variable=self.read_func_var, values=["0x03 Read holding registers", "0x04 Read input registers"], state="disabled")
197         self.read_cb.grid(row=3, column=0, colspan=2, padx=20, pady=(0, 15), sticky="ew")
198
199         self.btn_read = ctl.CtkButton(self.tab_ops, text="Read Data", command=self.do_read, state="disabled", fg_color="#3498db", hover_color="#2980b9", font=ctl.CtkFont(weight="bold"))
200         self.btn_read.grid(row=3, column=2, padx=20, pady=(0, 15), sticky="w")
201
202         # Row 3: Write Controls
203         self.write_func_var = ctl.StringVar(value="0x06 Write single register")
204         ctl.CtkLabel(self.tab_ops, text="Write Operation:", font=ctl.CtkFont(weight="bold")).grid(row=4, column=0, padx=20, pady=5, sticky="w")
205         self.write_cb = ctl.CtkComboBox(self.tab_ops, variable=self.write_func_var, values=["0x06 Write single register", "0x10 Write multiple registers"], state="disabled")
206         self.write_cb.grid(row=5, column=0, colspan=2, padx=20, pady=(0, 25), sticky="ew")
207
208         self.write_val_var = ctl.StringVar(value="0")
209         ctl.CtkLabel(self.tab_ops, text="Value to write:", font=ctl.CtkFont(weight="bold")).grid(row=4, column=2, padx=20, pady=5, sticky="w")
210         self.write_val_entry = ctl.CtkEntry(self.tab_ops, textvariable=self.write_val_var, state="disabled")
211         self.write_val_entry.grid(row=5, column=2, padx=20, pady=(0, 25), sticky="ew")
212
213         self.btn_write = ctl.CtkButton(self.tab_ops, text="Write Data", command=self.do_write, state="disabled", fg_color="#f39c12", hover_color="#d35400", font=ctl.CtkFont(weight="bold"))
214         self.btn_write.grid(row=5, column=3, padx=20, pady=(0, 25), sticky="w")
215
216         # ===== TAB 3: HACKER AUTO-SCANNER =====
217         self.tab_scan.grid_columnconfigure((0, 1, 2), weight=1)
218
219         ctl.CtkLabel(self.tab_scan, text="Hunt for hidden LIVE data registers:", font=ctl.CtkFont(size=16, weight="bold")).grid(row=0, column=0, colspan=3, padx=20, pady=(5, 10))
220
221         self.scan_start_var = ctl.StringVar(value="40000")
222         ctl.CtkLabel(self.tab_scan, text="Start Address (dec):", font=ctl.CtkFont(weight="bold")).grid(row=1, column=0, padx=20, pady=5, sticky="w")
223         self.scan_start_entry = ctl.CtkEntry(self.tab_scan, textvariable=self.scan_start_var, state="disabled")
224         self.scan_start_entry.grid(row=2, column=0, padx=20, pady=(0, 15), sticky="ew")
225
226         self.scan_end_var = ctl.StringVar(value="40150")
227         ctl.CtkLabel(self.tab_scan, text="End Address (dec):", font=ctl.CtkFont(weight="bold")).grid(row=1, column=1, padx=20, pady=5, sticky="w")
228         self.scan_end_entry = ctl.CtkEntry(self.tab_scan, textvariable=self.scan_end_var, state="disabled")
229         self.scan_end_entry.grid(row=2, column=1, padx=20, pady=(0, 15), sticky="ew")

```

```

Python Software > Modbus_Software.py > ...
21 class VFDDashboard(ctk.CTk):
184     def setup_main_area(self):
230
231         self.btn_scan = ctk.CTkButton(self.tab_scan, text="Start Hacker Scan", command=self.do_scan, state="disabled", fg_color="#9b59b6", hover_color="#8e44ad", font=ctk.CTkFont(
232         self.btn_scan.grid(row=2, column=2, padx=20, pady=(0, 15), sticky="w")
233
234         self.btn_stop_scan = ctk.CTkButton(self.tab_scan, text="Stop Scan", command=self.stop_scan, state="disabled", fg_color="#e74c3c", hover_color="#c0392b", font=ctk.CTkFont(
235         self.btn_stop_scan.grid(row=2, column=3, padx=20, pady=(0, 15), sticky="w")
236
237         # --- Bottom: Log Panel ---
238         self.log_frame = ctk.CTkFrame(self.main_frame, corner_radius=15)
239         self.log_frame.grid(row=1, column=0, sticky="nsew", pady=(10,0))
240         self.log_frame.grid_rowconfigure(1, weight=1)
241         self.log_frame.grid_columnconfigure(0, weight=1)
242
243         log_header = ctk.CTkFrame(self.log_frame, fg_color="transparent")
244         log_header.grid(row=0, column=0, sticky="ew", padx=20, pady=10)
245
246         ctk.CTkLabel(log_header, text="Network Traffic & System Logs", font=ctk.CTkFont(size=18, weight="bold")).pack(side="left")
247         ctk.CTkButton(log_header, text="Clear Output", width=100, fg_color="#7f8c8d", hover_color="#95a5a6", command=lambda: self.log_text.delete("1.0", ctk.END)).pack(side="right")
248
249         self.log_text = ctk.CTkTextbox(self.log_frame, font=ctk.CTkFont(family="Consolas", size=13), text_color="#00FF00", fg_color="#181818", corner_radius=10)
250         self.log_text.grid(row=1, column=0, padx=20, pady=(0, 20), sticky="nsew")
251
252     def save_pdm_data(self):
253     try:
254         f = float(self.pdm_freq_var.get())
255         A = float(self.pdm_curr_var.get())
256         r = float(self.pdm_rpm_var.get())
257         U = float(self.pdm_volt_var.get())
258         t = float(self.pdm_temp_var.get())
259
260         # A. Apparent Power
261         S = U * A
262
263         # B. Load Friction Index (A/Hz)
264         friction_idx = A / f if f > 0 else 0.0
265
266         # C. Cooling Efficiency Index (C/VA)
267         cooling_idx = t / S if S > 0 else 0.0
268
269         # D. Torque Estimate (S / f)
270         torque_est = S / f if f > 0 else 0.0
271
272         # Update state for next calculation
273         self.prev_temp = t
274         self.prev_current = A
275

```

```

Python Software > Modbus_Software.py > ...
21 class VFDDashboard(ctk.CTk):
252     def save_pdm_data(self):
275
276         # Update GUI Strings (Formatting logic)
277         self.pdm_s_val.set(f"{S:.2f} VA")
278         self.pdm_fric_val.set(f"{friction_idx:.4f} A/Hz")
279         self.pdm_cool_val.set(f"{cooling_idx:.4f} C/VA")
280         self.pdm_torque_val.set(f"{torque_est:.2f}")
281
282         # Save to CSV
283         file_path = "pdm_motor_log.csv"
284         file_exists = os.path.isfile(file_path)
285         with open(file_path, "a", newline="") as csvfile:
286             writer = csv.writer(csvfile)
287             if not file_exists:
288                 writer.writerow([
289                     "Timestamp", "Frequency(Hz)", "Current(A)", "RPM",
290                     "Voltage(V)", "VFD_Temp(C)", "ApparentPower(VA)",
291                     "LoadFriction(A/Hz)", "CoolingEfficiency(C/VA)", "TorqueEst"
292                 ])
293             writer.writerow([
294                 datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
295                 f, A, r, U, t,
296                 round(S, 2), round(friction_idx, 4), round(cooling_idx, 4), round(torque_est, 2)
297             ])
298
299         self.log_text.insert(ctk.END, f"\n[PdM Logger] Analytics created & safely appended to 'pdm_motor_log.csv'\n")
300         self.log_text.see(ctk.END)
301
302     except ValueError as e:
303         messagebox.showerror("Input Error", "Please ensure all PdM inputs contain valid numbers.")
304
305     def get_parity(self, p_str):
306         if p_str == "None": return serial.PARITY_NONE
307         elif p_str == "Literal['Odd']": return serial.PARITY_ODD
308         elif p_str == "Mark": return serial.PARITY_MARK
309         elif p_str == "Space": return serial.PARITY_SPACE
310         return serial.PARITY_NONE
311
312     def connect(self):
313     try:
314         port = self.port_var.get()
315         slave_id = int(self.slave_id_var.get())
316
317         self.instrument = minimalmodbus.Instrument(port, slave_id)
318         self.instrument.serial.baudrate = int(self.baud_var.get())
319         self.instrument.serial.bytesize = int(self.data_bits_var.get())
320

```



```

Python Software > Modbus_Software.py > ...
21 class VFDDashboard(ctk.CTk):
313     def connect(self):
322         self.instrument.serial.stopbits = float(self.stop_bits_var.get())
323         self.instrument.serial.timeout = 0.5
324         self.instrument.debug = True
325
326         self.is_connected = True
327         self.update_ui_state(True)
328         print(f"\n[+] Successfully connected to {port} (Targeted Slave ID: {slave_id}).")
329
330     except Exception as e:
331         print(f"\n[!] Connection Error: {e}")
332         messagebox.showerror("Connection Error", str(e))
333         self.disconnect()
334
335     def disconnect(self):
336         self.is_connected = False
337         self.is_scanning = False
338         if self.instrument and self.instrument.serial.is_open:
339             self.instrument.serial.close()
340         self.update_ui_state(False)
341         print("\n[-] Disconnected seamlessly.")
342
343     def update_ui_state(self, connected):
344         state = "normal" if connected else "disabled"
345         rev_state = "disabled" if connected else "normal"
346
347         self.btn_connect.configure(state=rev_state)
348         self.btn_disconnect.configure(state=state)
349
350         self.address_entry.configure(state=state)
351         self.num_res_entry.configure(state=state)
352         self.read_cb.configure(state=state)
353         self.btn_read.configure(state=state)
354         self.write_cb.configure(state=state)
355         self.write_val_entry.configure(state=state)
356         self.btn_write.configure(state=state)
357
358         self.scan_start_entry.configure(state=state)
359         self.scan_end_entry.configure(state=state)
360         self.btn_scan.configure(state=state)
361
362     def do_read(self):
363         if not self.is_connected: return
364         try:
365             self.instrument.address = int(self.slave_id_var.get())
366
367             address = int(self.address_var.get())

```

```

Python Software > Modbus_Software.py > ...
21 class VFDDashboard(ctk.CTk):
362     def do_read(self):
367         address = int(self.address_var.get())
368         num_regs = int(self.num_res_var.get())
369         func_str = self.read_func_var.get()
370         func_code = 3 if "0x03" in func_str else 4
371
372         print(f"\n---> READ INITIATED || Addr: {address} | Reg Count: {num_regs}")
373
374         if num_regs == 1:
375             val = self.instrument.read_register(address, 0, functioncode=func_code)
376             print(f"<--- READ SUCCESS || Interpreted Output: {val}")
377         else:
378             vals = self.instrument.read_registers(address, num_regs, functioncode=func_code)
379             print(f"<--- READ SUCCESS || Interpreted Outputs: {vals}")
380     except Exception as e:
381         print(f"[!] READ ERROR: {e}")
382
383     def do_write(self):
384         if not self.is_connected: return
385         try:
386             self.instrument.address = int(self.slave_id_var.get())
387
388             address = int(self.address_var.get())
389             func_str = self.write_func_var.get()
390             func_code = 6 if "0x06" in func_str else 16
391             val_str = self.write_val_var.get()
392
393             if func_code == 6:
394                 val = int(val_str, 16) if val_str.lower().startswith("0x") else int(val_str)
395                 print(f"\n---> WRITE INITIATED || Addr: {address} | Data: {val}")
396                 self.instrument.write_register(address, val, 0, functioncode=func_code)
397                 print(f"<--- WRITE SUCCESS")
398             else:
399                 vals = [int(v.strip(), 16) if v.strip().lower().startswith("0x") else int(v.strip()) for v in val_str.split(",")]
400                 print(f"\n---> WRITE INITIATED || Addr: {address} | Data Payload: {vals}")
401                 self.instrument.write_registers(address, vals)
402                 print(f"<--- WRITE SUCCESS")
403     except Exception as e:
404         print(f"[!] WRITE ERROR: {e}")
405
406     def do_scan(self):
407         if not self.is_connected: return
408         self.is_scanning = True
409         self.btn_scan.configure(state="disabled")
410         self.btn_stop_scan.configure(state="normal")
411         threading.Thread(target=self._scan_process, daemon=True).start()

```

```

Python Software > Modbus_Software.py > ...
21 class VFDDashboard(ctk.CTk):
406     def do_scan(self):
411         threading.Thread(target=self._scan_process, daemon=True).start()
412
413     def stop_scan(self):
414         self.is_scanning = False
415
416     def _scan_process(self):
417         try:
418             start_addr = int(self.scan_start_var.get())
419             end_addr = int(self.scan_end_var.get())
420             self.instrument.address = int(self.slave_id_var.get())
421
422             print(f"\n=====")
423             print(f"[Scanner] INITIALIZING AUTO-SCAN -> Range: {start_addr} to {end_addr}")
424             print(f"=====\\n")
425
426             original_debug = self.instrument.debug
427             self.instrument.debug = False
428
429             found_count = 0
430
431             for addr in range(start_addr, end_addr + 1):
432                 if not self.is_scanning or not self.is_connected:
433                     print("[Scanner] SCAN ABORTED BY USER.")
434                     break
435
436                 time.sleep(0.05)
437
438                 try:
439                     val = self.instrument.read_register(addr, 0, functioncode=3)
440                     if val != 0:
441                         print(f" [+] FOUND LIVE DATA -> Address: {addr} | Current Value: {val}")
442                         found_count += 1
443                 except Exception:
444                     pass
445
446             self.instrument.debug = original_debug
447             print(f"\n=====")
448             print(f"[Scanner] SCAN COMPLETED. Found {found_count} registers carrying non-zero data.")
449             print(f"=====\\n")
450
451         except Exception as e:
452             print(f"[Scanner] System Exception: {e}")
453
454         finally:
455             self.is_scanning = False
456             self.btn_scan.configure(state="normal")
457             self.btn_stop_scan.configure(state="disabled")

```

```

Python Software > Modbus_Software.py > ...
457         self.btn_stop_scan.configure(state="disabled")
458
459     if __name__ == "__main__":
460         app = VFDDashboard()
461         app.protocol("WM_DELETE_WINDOW", lambda: (app.disconnect(), app.destroy()))
462         app.mainloop()
463

```

Appendix B: Raw Modbus Telemetry Dataset

Dataset of motor pdm.csv :-

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
1	0.1	0.0	3.0	315.0	30.0	0.0	0.0	0.0	0.0	0
2	0.2	0.0	6.0	315.0	30.0	0.0	0.0	0.0	0.0	0
3	0.4	0.0	12.0	317.0	31.0	0.0	0.0	0.0	0.0	0
4	0.5	0.0	15.0	316.0	31.0	0.0	0.0	0.0	0.0	0
5	0.3	0.0	9.0	315.0	30.0	0.0	0.0	0.0	0.0	0
6	0.6	0.1	18.0	316.0	33.0	31.6	0.1667	1.0443	52.67	0
7	0.7	0.3	21.0	314.0	39.0	94.2	0.4286	0.414	134.57	0
8	0.8	0.5	24.0	313.0	41.0	156.5	0.625	0.262	195.62	0
9	0.9	0.8	27.0	314.0	44.0	251.2	0.8889	0.1752	279.11	0
10	1.0	1.1	30.0	312.0	50.0	343.2	1.1	0.1457	343.2	0
11	1.1	1.2	33.0	312.0	49.0	374.4	1.0909	0.1309	340.36	0
12	1.2	1.2	36.0	312.0	51.0	374.4	1.0	0.1362	312.0	0
13	1.3	1.2	39.0	313.0	48.0	375.6	1.0	0.1278	313.0	0
14	1.4	1.2	42.0	313.0	42.0	375.6	0.8571	0.1118	268.29	0
15	1.5	1.2	45.0	312.0	44.0	374.4	0.8	0.1175	249.6	0
16	1.6	1.3	48.0	312.0	50.0	405.6	0.8125	0.1233	253.5	0
17	1.7	1.3	51.0	312.0	52.0	405.6	0.7647	0.1282	238.59	0
18	1.8	1.3	54.0	312.0	54.0	405.6	0.7222	0.1331	225.33	0
19	1.9	1.3	57.0	311.0	56.0	404.3	0.6842	0.1385	212.79	0
20	2.0	1.5	60.0	310.0	58.0	465.0	0.75	0.1247	232.5	0
21	2.1	1.6	63.0	314.0	35.0	502.4	0.7619	0.0697	239.24	0
22	2.2	1.6	66.0	314.0	39.0	502.4	0.7273	0.0776	228.36	0
23	2.3	1.6	69.0	313.0	41.0	500.8	0.6957	0.0819	217.74	0
24	2.4	1.6	72.0	313.0	44.0	500.8	0.6667	0.0879	208.67	0
25	2.5	1.6	75.0	314.0	50.0	502.4	0.64	0.0995	200.96	0
26	2.6	1.6	78.0	314.0	50.0	502.4	0.6154	0.0995	193.23	0
27	2.7	1.6	81.0	314.0	51.0	502.4	0.5926	0.1015	186.07	0
28	2.8	1.6	84.0	315.0	54.0	504.0	0.5714	0.1071	180.0	0
29	2.9	1.6	87.0	315.0	55.0	504.0	0.5517	0.1091	173.79	0
30	3.0	1.6	90.0	316.0	57.0	505.6	0.5333	0.1127	168.53	0
31	3.1	1.6	93.0	314.0	59.0	502.4	0.5161	0.1174	162.06	0
32	3.2	1.6	96.0	315.0	59.0	504.0	0.5	0.1171	157.5	0
33	3.3	1.6	99.0	314.0	60.0	502.4	0.4848	0.1194	152.24	0
34	3.4	1.7	102.0	316.0	27.0	537.2	0.5	0.0503	158.0	0

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
35	3.5	1.7	105.0	317.0	35.0	538.9	0.4857	0.0649	153.97	0
36	3.6	1.7	108.0	317.0	40.0	538.9	0.4722	0.0742	149.69	0
37	3.7	1.7	111.0	315.0	49.0	535.5	0.4595	0.0915	144.73	0
38	3.8	1.7	114.0	315.0	53.0	535.5	0.4474	0.099	140.92	0
39	3.9	1.7	117.0	315.0	56.0	535.5	0.4359	0.1046	137.31	0
40	4.0	1.7	120.0	315.0	62.0	535.5	0.425	0.1158	133.88	0
41	4.1	1.7	123.0	317.0	63.0	538.9	0.4146	0.1169	131.44	0
42	4.2	1.6	126.0	317.0	67.0	507.2	0.381	0.1321	120.76	0
43	4.3	1.7	129.0	316.0	57.0	537.2	0.3953	0.1061	124.93	0
44	4.4	1.6	132.0	318.0	63.0	508.8	0.3636	0.1238	115.64	0
45	4.5	1.7	135.0	318.0	68.0	540.6	0.3778	0.1258	120.13	0
46	4.6	1.6	138.0	318.0	73.0	508.8	0.3478	0.1435	110.61	0
47	4.7	1.7	141.0	319.0	67.0	542.3	0.3617	0.1235	115.38	0
48	4.8	1.7	144.0	312.0	48.0	530.4	0.3542	0.0905	110.5	0
49	4.9	1.7	147.0	312.0	51.0	530.4	0.3469	0.0962	108.24	0
50	5.0	1.7	150.0	310.0	54.0	527.0	0.34	0.1025	105.4	0
51	5.1	1.7	153.0	318.0	26.0	540.6	0.3333	0.0481	106.0	0
52	5.2	1.7	156.0	318.0	30.0	540.6	0.3269	0.0555	103.96	0
53	5.3	1.7	159.0	317.0	39.0	538.9	0.3208	0.0724	101.68	0
54	5.4	1.7	162.0	319.0	43.0	542.3	0.3148	0.0793	100.43	0
55	5.5	1.6	165.0	319.0	51.0	510.4	0.2909	0.0999	92.8	0
56	5.6	1.6	168.0	317.0	57.0	507.2	0.2909	0.1124	92.22	0
57	5.7	1.6	171.0	320.0	56.0	512.0	0.2807	0.1094	89.82	0
58	5.8	1.6	174.0	318.0	60.0	508.8	0.2759	0.1179	87.72	0
59	5.9	1.6	177.0	318.0	64.0	508.8	0.2712	0.1258	86.24	0
60	6.0	1.6	180.0	318.0	68.0	508.8	0.2667	0.1336	84.8	0
61	6.1	1.7	183.0	317.0	48.0	538.9	0.2787	0.0891	88.34	0
62	6.2	1.7	186.0	319.0	54.0	542.3	0.2742	0.0996	87.47	0
63	6.3	1.6	189.0	318.0	60.0	508.8	0.254	0.1179	80.76	0
64	6.4	1.6	192.0	318.0	63.0	508.8	0.25	0.1238	79.5	0
65	6.5	1.6	195.0	318.0	68.0	508.8	0.2462	0.1336	78.28	0
66	6.6	1.6	198.0	318.0	70.0	508.8	0.2424	0.1376	77.09	0
67	6.7	1.6	201.0	318.0	69.0	508.8	0.2388	0.1356	75.94	0
68	6.8	1.6	204.0	319.0	71.0	510.4	0.2353	0.1391	75.06	0

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
69	6.9	1.6	207.0	319.0	72.0	510.4	0.2319	0.1411	73.97	0
70	7.0	1.5	210.0	320.0	78.0	480.0	0.2143	0.1625	68.57	0
71	7.1	1.6	213.0	315.0	31.0	504.0	0.2254	0.0615	70.99	0
72	7.2	1.6	216.0	316.0	39.0	505.6	0.2222	0.0771	70.22	0
73	7.3	1.6	219.0	315.0	42.0	504.0	0.2192	0.0833	69.04	0
74	7.4	1.6	222.0	314.0	40.0	502.4	0.2162	0.0796	67.89	0
75	7.5	1.6	225.0	314.0	48.0	502.4	0.2133	0.0955	66.99	0
76	7.6	1.5	228.0	317.0	55.0	475.5	0.1974	0.1157	62.57	0
77	7.7	1.6	231.0	309.0	26.0	494.4	0.2078	0.0526	64.21	0
78	7.8	1.6	234.0	309.0	29.0	494.4	0.2051	0.0587	63.38	0
79	7.9	1.5	237.0	311.0	39.0	466.5	0.1899	0.0836	59.05	0
80	8.0	1.5	240.0	311.0	43.0	466.5	0.1875	0.0922	58.31	0
81	8.1	1.5	243.0	310.0	40.0	465.0	0.1852	0.086	57.41	0
82	8.2	1.5	246.0	309.0	39.0	463.5	0.1829	0.0841	56.52	0
83	8.3	1.5	249.0	311.0	40.0	466.5	0.1807	0.0857	56.2	0
84	8.4	1.5	252.0	312.0	40.0	468.0	0.1786	0.0855	55.71	0
85	8.5	1.5	255.0	312.0	41.0	468.0	0.1765	0.0876	55.06	0
86	8.6	1.5	258.0	311.0	44.0	466.5	0.1744	0.0943	54.24	0
87	8.7	1.5	261.0	312.0	43.0	468.0	0.1724	0.0919	53.79	0
88	8.8	1.5	264.0	312.0	37.0	468.0	0.1705	0.0791	53.18	0
89	8.9	1.5	267.0	311.0	41.0	466.5	0.1685	0.0879	52.42	0
90	9.0	1.5	270.0	312.0	48.0	468.0	0.1667	0.1026	52.0	0
91	9.1	1.5	273.0	314.0	49.0	471.0	0.1648	0.104	51.76	0
92	9.2	1.4	276.0	313.0	56.0	438.2	0.1522	0.1278	47.63	0
93	9.3	1.4	279.0	313.0	60.0	438.2	0.1505	0.1369	47.12	0
94	9.4	1.4	282.0	313.0	64.0	438.2	0.1489	0.1461	46.62	0
95	9.5	1.4	285.0	314.0	67.0	439.6	0.1474	0.1524	46.27	0
96	9.6	1.4	288.0	312.0	71.0	436.8	0.1458	0.1625	45.5	0
97	9.7	1.4	291.0	313.0	73.0	438.2	0.1443	0.1666	45.18	0
98	9.8	1.4	294.0	312.0	61.0	436.8	0.1429	0.1397	44.57	0
99	9.9	1.4	297.0	312.0	64.0	436.8	0.1414	0.1465	44.12	0
100	10.0	1.4	300.0	311.0	60.0	435.4	0.14	0.1378	43.54	0
101	10.1	1.4	303.0	315.0	23.0	441.0	0.1386	0.0522	43.66	0
102	10.2	1.3	306.0	315.0	29.0	409.5	0.1275	0.0708	40.15	0

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
103	10.3	1.4	309.0	312.0	30.0	436.8	0.1359	0.0687	42.41	0
104	10.4	1.4	312.0	315.0	29.0	441.0	0.1346	0.0658	42.4	0
105	10.5	1.3	315.0	316.0	36.0	410.8	0.1238	0.0876	39.12	0
106	10.6	1.3	318.0	317.0	40.0	412.1	0.1226	0.0971	38.88	0
107	10.7	1.3	321.0	316.0	35.0	410.8	0.1215	0.0852	38.39	0
108	10.8	1.3	324.0	316.0	38.0	410.8	0.1204	0.0925	38.04	0
109	10.9	1.3	327.0	317.0	41.0	412.1	0.1193	0.0995	37.81	0
110	11.0	1.3	330.0	315.0	44.0	409.5	0.1182	0.1074	37.23	0
111	11.1	1.3	333.0	315.0	48.0	409.5	0.1171	0.1172	36.89	0
112	11.2	1.3	336.0	313.0	50.0	406.9	0.1161	0.1229	36.33	0
113	11.3	1.3	339.0	316.0	52.0	410.8	0.115	0.1266	36.35	0
114	11.4	1.2	342.0	314.0	55.0	376.8	0.1053	0.146	33.05	0
115	11.5	1.2	345.0	315.0	57.0	378.0	0.1043	0.1508	32.87	0
116	11.6	1.2	348.0	315.0	58.0	378.0	0.1034	0.1534	32.59	0
117	11.7	1.2	351.0	316.0	53.0	379.2	0.1026	0.1398	32.41	0
118	11.8	1.2	354.0	315.0	55.0	378.0	0.1017	0.1455	32.03	0
119	11.9	1.2	357.0	316.0	57.0	379.2	0.1008	0.1503	31.87	0
120	12.0	1.2	360.0	317.0	50.0	380.4	0.1	0.1314	31.7	0
121	12.1	1.2	363.0	318.0	50.0	381.6	0.0992	0.131	31.54	0
122	12.2	1.2	366.0	315.0	26.0	378.0	0.0984	0.0688	30.98	0
123	12.3	1.2	369.0	314.0	31.0	376.8	0.0976	0.0823	30.63	0
124	12.4	1.2	372.0	315.0	38.0	378.0	0.0968	0.1005	30.48	0
125	12.5	1.2	375.0	315.0	40.0	378.0	0.096	0.1058	30.24	0
126	12.6	1.2	378.0	315.0	42.0	378.0	0.0952	0.1111	30.0	0
127	12.7	1.2	381.0	320.0	48.0	384.0	0.0945	0.125	30.24	0
128	12.8	1.2	384.0	317.0	51.0	380.4	0.0937	0.1341	29.72	0
129	12.9	1.1	387.0	315.0	55.0	346.5	0.0853	0.1587	26.86	0
130	13.0	1.1	390.0	316.0	56.0	347.6	0.0846	0.1611	26.74	0
131	13.1	1.1	393.0	316.0	42.0	347.6	0.084	0.1208	26.53	0
132	13.2	1.1	396.0	316.0	46.0	347.6	0.0833	0.1323	26.33	0
133	13.3	1.1	399.0	317.0	51.0	348.7	0.0827	0.1463	26.22	0
134	13.4	1.1	402.0	316.0	54.0	347.6	0.0821	0.1554	25.94	0
135	13.5	1.1	405.0	317.0	57.0	348.7	0.0815	0.1635	25.83	0
136	13.6	1.1	408.0	317.0	57.0	348.7	0.0809	0.1635	25.64	0

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
137	13.7	1.1	411.0	317.0	53.0	348.7	0.0803	0.152	25.45	0
138	13.8	1.1	414.0	316.0	55.0	347.6	0.0797	0.1582	25.19	0
139	13.9	1.1	417.0	316.0	55.0	347.6	0.0791	0.1582	25.01	0
140	14.0	1.1	420.0	317.0	56.0	348.7	0.0786	0.1606	24.91	0
141	14.1	1.1	423.0	315.0	56.0	346.5	0.078	0.1616	24.57	0
142	14.2	1.1	426.0	315.0	56.0	346.5	0.0775	0.1616	24.4	0
143	14.3	1.1	429.0	319.0	40.0	350.9	0.0769	0.114	24.54	0
144	14.4	1.1	432.0	317.0	42.0	348.7	0.0764	0.1204	24.22	0
145	14.5	1.1	435.0	318.0	49.0	349.8	0.0759	0.1401	24.12	0
146	14.6	1.0	438.0	317.0	52.0	317.0	0.0685	0.164	21.71	0
147	14.7	1.0	441.0	318.0	54.0	318.0	0.068	0.1698	21.63	0
148	14.8	1.0	444.0	318.0	56.0	318.0	0.0676	0.1761	21.49	0
149	14.9	1.0	447.0	318.0	56.0	318.0	0.0671	0.1761	21.34	0
150	15.0	1.0	450.0	319.0	57.0	319.0	0.0667	0.1787	21.27	0
151	10.0	1.85	300.0	316.1	45.5	583.6	0.1846	0.0779	57.34	1
152	10.3	1.69	308.0536912751678	316.9	37.5	536.0	0.1647	0.0699	56.57	1
153	10.5	1.67	316.1073825503356	316.0	44.8	527.1	0.1583	0.085	59.64	1
154	10.8	1.68	324.16107382550337	317.1	51.8	531.9	0.1552	0.0974	62.88	1
155	11.1	1.89	332.2147651006712	319.2	50.2	602.8	0.1705	0.0833	44.41	1
156	11.3	1.89	340.2684563758389	316.8	36.6	599.5	0.1669	0.061	47.75	1
157	11.6	1.84	348.3221476510067	314.1	51.4	578.7	0.1587	0.0887	53.89	1
158	11.9	1.69	356.3758389261745	314.0	45.2	530.6	0.1422	0.0851	32.62	1
159	12.1	1.72	364.4295302013423	313.8	40.1	541.2	0.142	0.0741	55.44	1
160	12.4	1.91	372.4832214765101	314.6	38.4	601.5	0.154	0.0639	62.21	1
161	12.7	1.65	380.53691275167785	317.3	48.9	524.1	0.1302	0.0933	39.52	1
162	13.0	1.75	388.59060402684565	311.0	46.2	545.4	0.1354	0.0848	41.24	1
163	13.2	1.83	396.6442953020134	315.2	50.0	576.5	0.1383	0.0867	35.14	1
164	13.5	1.76	404.6979865771812	316.0	40.8	554.6	0.1301	0.0736	46.59	1
165	13.8	1.65	412.75167785234896	315.0	41.4	519.8	0.1199	0.0797	30.46	1
166	14.0	1.69	420.8053691275168	312.9	50.0	528.5	0.1204	0.0947	42.54	1
167	14.3	1.7	428.8590604026846	315.3	37.6	534.7	0.1186	0.0703	28.98	1
168	14.6	1.97	436.9127516778523	314.0	47.9	618.5	0.1353	0.0775	40.59	1
169	14.8	1.84	444.9664429530201	313.8	45.5	577.6	0.1241	0.0788	48.72	1
170	15.1	1.76	453.0201342281879	318.4	45.4	561.2	0.1167	0.0809	41.97	1

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
171	15.4	1.74	461.0738255033557	319.6	44.9	557.1	0.1134	0.0806	45.36	1
172	15.6	1.68	469.1275167785235	314.8	50.4	527.3	0.1071	0.0956	42.49	1
173	15.9	1.72	477.1812080536913	317.5	49.0	544.9	0.1079	0.0899	44.43	1
174	16.2	1.89	485.2348993288591	316.2	47.1	598.5	0.117	0.0787	34.31	1
175	16.4	1.76	493.2885906040268	317.6	30.8	559.2	0.1071	0.055	38.73	1
176	16.7	1.8	501.3422818791947	315.0	44.5	566.3	0.1076	0.0785	43.8	1
177	17.0	1.69	509.3959731543624	316.7	43.3	536.1	0.0997	0.0807	32.15	1
178	17.2	1.75	517.4496644295302	318.8	50.2	557.1	0.1013	0.0902	37.54	1
179	17.5	1.66	525.5033557046979	314.7	42.9	522.8	0.0948	0.082	30.22	1
180	17.8	1.45	533.5570469798657	312.3	37.5	451.5	0.0813	0.083	32.06	1
181	18.1	1.77	541.6107382550335	314.4	41.2	556.2	0.098	0.074	29.0	1
182	18.3	1.82	549.6644295302012	310.2	51.2	564.8	0.0994	0.0906	34.91	1
183	18.6	1.7	557.7181208053692	313.8	45.4	533.4	0.0914	0.0852	34.55	1
184	18.9	1.73	565.7718120805368	315.1	48.4	545.3	0.0918	0.0888	33.95	1
185	19.1	1.75	573.8255033557047	317.9	35.0	557.6	0.0917	0.0628	33.11	1
186	19.4	1.69	581.8791946308725	315.5	53.4	532.3	0.087	0.1003	30.4	1
187	19.7	1.67	589.9328859060403	314.0	49.7	523.5	0.0848	0.095	22.09	1
188	19.9	1.69	597.9865771812081	316.9	45.8	535.4	0.0848	0.0855	31.02	1
189	20.2	1.62	606.0402684563758	314.3	38.3	509.2	0.0802	0.0753	22.1	1
190	20.5	1.61	614.0939597315437	314.3	44.8	506.3	0.0787	0.0885	27.69	1
191	20.7	1.54	622.1476510067114	317.1	36.7	487.5	0.0741	0.0754	16.85	1
192	21.0	2.0	630.2013422818792	315.6	43.5	631.1	0.0952	0.0689	31.23	1
193	21.3	1.6	638.255033557047	317.5	51.8	506.8	0.075	0.1023	26.09	1
194	21.5	1.79	646.3087248322148	319.7	47.9	572.0	0.0831	0.0837	23.81	1
195	21.8	1.75	654.3624161073826	317.9	50.3	556.1	0.0802	0.0905	27.71	1
196	22.1	1.69	662.4161073825503	314.4	42.6	531.5	0.0766	0.0802	29.98	1
197	22.3	1.7	670.4697986577182	315.8	47.6	538.1	0.0762	0.0885	27.19	1
198	22.6	1.63	678.5234899328859	314.7	44.2	511.4	0.0719	0.0864	21.42	1
199	22.9	1.6	686.5771812080536	318.8	36.2	509.3	0.0698	0.071	16.42	1
200	23.2	1.81	694.6308724832214	316.8	48.2	574.8	0.0784	0.0839	22.93	1
201	23.4	1.56	702.6845637583892	316.9	43.2	495.9	0.0668	0.0872	25.92	1
202	23.7	1.81	710.738255033557	313.6	42.2	569.0	0.0766	0.0742	28.38	1
203	24.0	1.69	718.7919463087247	314.1	42.9	531.0	0.0705	0.0809	27.65	1
204	24.2	1.76	726.8456375838927	315.9	37.4	557.4	0.0728	0.0672	24.29	1

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
205	24.5	1.76	734.8993288590603	311.1	41.9	546.3	0.0717	0.0767	19.87	1
206	24.8	1.91	742.9530201342283	315.2	39.2	601.1	0.077	0.0652	22.4	1
207	25.0	1.95	751.006711409396	316.3	49.1	618.3	0.0781	0.0793	28.25	1
208	25.3	1.71	759.0604026845638	316.4	48.7	541.5	0.0676	0.09	19.37	1
209	25.6	1.96	767.1140939597316	315.0	47.7	618.4	0.0768	0.0772	19.61	1
210	25.8	1.57	775.1677852348993	313.2	45.0	491.2	0.0607	0.0915	17.94	1
211	26.1	1.72	783.2214765100672	314.2	39.0	539.4	0.0657	0.0723	16.63	1
212	26.4	1.7	791.2751677852349	316.9	45.2	537.6	0.0643	0.0841	20.68	1
213	26.6	1.95	799.3288590604027	314.8	47.3	612.8	0.0731	0.0772	22.59	1
214	26.9	1.77	807.3825503355705	312.8	54.3	554.3	0.0658	0.0979	22.36	1
215	27.2	1.63	815.4362416107383	313.4	47.1	510.8	0.06	0.0922	19.3	1
216	27.4	1.59	823.489932885906	313.5	47.9	497.0	0.0578	0.0965	14.36	1
217	27.7	1.82	831.5436241610738	313.5	44.2	571.0	0.0657	0.0774	17.98	1
218	28.0	1.54	839.5973154362416	315.6	37.9	486.0	0.055	0.078	15.51	1
219	28.3	1.73	847.6510067114093	316.1	46.1	547.5	0.0613	0.0841	18.69	1
220	28.5	1.64	855.7046979865771	311.3	53.4	509.7	0.0574	0.1048	17.53	1
221	28.8	1.67	863.7583892617449	319.3	54.1	533.8	0.0581	0.1013	19.82	1
222	29.1	1.84	871.8120805369127	315.1	45.1	579.1	0.0632	0.0779	15.56	1
223	29.3	1.61	879.8657718120805	316.0	38.9	508.4	0.0549	0.0765	22.19	1
224	29.6	1.59	887.9194630872483	313.5	48.4	497.9	0.0537	0.0972	13.6	1
225	29.9	1.56	895.9731543624162	310.1	46.1	485.0	0.0524	0.0951	15.09	1
226	30.1	1.8	904.0268456375838	314.9	47.3	566.9	0.0598	0.0835	13.32	1
227	30.4	1.55	912.0805369127517	318.3	47.2	492.9	0.0509	0.0957	18.15	1
228	30.7	1.75	920.1342281879196	317.5	46.9	554.8	0.057	0.0846	16.03	1
229	30.9	1.59	928.1879194630872	312.7	46.5	496.3	0.0513	0.0936	16.71	1
230	31.2	1.7	936.2416107382552	314.4	45.3	534.1	0.0544	0.0849	19.19	1
231	31.5	1.69	944.2953020134228	318.8	34.6	537.5	0.0536	0.0644	13.72	1
232	31.7	1.65	952.3489932885908	314.2	52.7	518.2	0.052	0.1018	11.92	1
233	32.0	1.71	960.4026845637584	311.5	44.8	533.1	0.0535	0.0841	13.74	1
234	32.3	1.89	968.4563758389264	314.5	46.4	593.5	0.0585	0.0782	22.21	1
235	32.6	1.89	976.510067114094	314.2	46.8	593.3	0.058	0.0789	17.24	1
236	32.8	1.83	984.5637583892616	314.1	45.5	573.4	0.0556	0.0794	13.78	1
237	33.1	1.73	992.6174496644296	312.9	45.6	542.3	0.0524	0.0841	14.09	1
238	33.4	1.78	1000.6711409395974	311.3	44.9	554.0	0.0534	0.081	15.6	1

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
239	33.6	1.66	1008.7248322147652	316.6	49.6	525.2	0.0493	0.0944	19.99	1
240	33.9	1.86	1016.7785234899328	318.7	36.7	592.4	0.0548	0.0619	15.3	1
241	34.2	1.79	1024.8322147651006	312.6	39.4	559.7	0.0524	0.0703	12.94	1
242	34.4	1.79	1032.8859060402683	313.0	50.2	559.6	0.0519	0.0898	19.23	1
243	34.7	1.68	1040.9395973154365	318.2	45.5	534.7	0.0484	0.0851	11.71	1
244	35.0	1.74	1048.9932885906042	315.3	48.9	548.8	0.0498	0.0891	11.95	1
245	35.2	1.77	1057.0469798657718	312.7	43.5	552.7	0.0502	0.0787	18.82	1
246	35.5	1.82	1065.1006711409395	314.2	40.6	570.5	0.0511	0.0711	18.89	1
247	35.8	1.67	1073.1543624161072	317.1	38.0	529.6	0.0467	0.0718	11.45	1
248	36.0	1.58	1081.2080536912752	317.4	49.8	502.4	0.0439	0.0991	13.62	1
249	36.3	1.75	1089.2617449664428	316.6	42.8	552.6	0.0481	0.0775	19.36	1
250	36.6	1.55	1097.3154362416103	312.7	47.2	485.4	0.0424	0.0972	14.86	1
251	36.8	1.59	1105.3691275167785	313.6	53.0	499.8	0.0433	0.1061	12.52	1
252	37.1	1.67	1113.4228187919464	316.0	44.2	528.5	0.0451	0.0836	14.05	1
253	37.4	1.68	1121.476510067114	315.6	45.2	531.6	0.0451	0.085	16.05	1
254	37.7	1.65	1129.5302013422818	317.6	39.6	524.3	0.0438	0.0756	17.34	1
255	37.9	1.57	1137.5838926174497	315.0	55.6	494.2	0.0414	0.1126	16.8	1
256	38.2	1.8	1145.6375838926174	317.0	49.5	570.9	0.0472	0.0867	12.56	1
257	38.5	1.7	1153.6912751677853	312.6	47.4	530.9	0.0442	0.0892	15.28	1
258	38.7	1.83	1161.744966442953	314.3	44.7	573.8	0.0472	0.0779	18.78	1
259	39.0	1.96	1169.7986577181207	314.0	42.6	614.3	0.0502	0.0693	18.78	1
260	39.3	1.66	1177.8523489932886	318.5	43.6	528.0	0.0422	0.0825	11.58	1
261	39.5	1.87	1185.9060402684563	313.4	45.5	585.1	0.0472	0.0778	12.73	1
262	39.8	1.74	1193.9597315436242	314.8	50.7	547.5	0.0437	0.0926	9.68	1
263	40.1	1.57	1202.013422818792	317.3	46.5	499.1	0.0393	0.0932	14.12	1
264	40.3	1.76	1210.0671140939596	315.8	44.6	556.2	0.0437	0.0803	10.83	1
265	40.6	1.65	1218.1208053691275	316.0	45.8	522.5	0.0407	0.0876	15.81	1
266	40.9	1.85	1226.1744966442957	311.0	44.9	575.6	0.0453	0.078	17.75	1
267	41.1	1.74	1234.2281879194632	316.1	45.0	548.5	0.0422	0.0821	17.09	1
268	41.4	1.79	1242.2818791946308	315.9	47.6	567.0	0.0433	0.0839	15.36	1
269	41.7	1.79	1250.3355704697983	312.1	39.0	559.6	0.043	0.0697	16.5	1
270	41.9	1.77	1258.3892617449665	313.4	43.8	553.9	0.0421	0.079	12.17	1
271	42.2	1.62	1266.4429530201344	313.5	45.4	509.1	0.0385	0.0892	15.6	1
272	42.5	1.64	1274.496644295302	313.0	47.1	512.0	0.0385	0.0919	12.95	1

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
273	42.8	1.89	1282.5503355704698	315.7	47.8	598.0	0.0443	0.08	16.48	1
274	43.0	1.63	1290.6040268456377	314.0	37.2	512.5	0.0379	0.0726	13.1	1
275	43.3	1.69	1298.6577181208054	313.8	48.5	531.8	0.0391	0.0912	8.95	1
276	43.6	1.69	1306.7114093959733	312.8	38.0	528.3	0.0388	0.0719	9.39	1
277	43.8	1.63	1314.765100671141	314.3	43.7	511.5	0.0371	0.0855	10.97	1
278	44.1	1.51	1322.818791946309	313.5	42.6	473.8	0.0343	0.09	10.44	1
279	44.4	1.68	1330.8724832214766	314.2	45.9	528.7	0.0379	0.0869	8.82	1
280	44.6	1.66	1338.9261744966443	310.5	43.9	516.1	0.0372	0.0851	9.51	1
281	44.9	1.66	1346.979865771812	316.3	41.4	525.9	0.037	0.0786	12.75	1
282	45.2	1.74	1355.0335570469797	316.5	42.9	550.7	0.0385	0.0778	13.89	1
283	45.4	1.85	1363.0872483221476	315.4	40.5	583.6	0.0407	0.0693	13.55	1
284	45.7	2.0	1371.1409395973153	317.5	44.8	633.4	0.0437	0.0707	17.27	1
285	46.0	1.73	1379.1946308724832	317.5	45.2	549.9	0.0377	0.0823	12.16	1
286	46.2	1.68	1387.248322147651	319.4	47.9	537.2	0.0364	0.0892	9.59	1
287	46.5	1.61	1395.3020134228186	317.0	45.4	511.5	0.0347	0.0887	11.78	1
288	46.8	1.72	1403.3557046979863	315.3	49.3	541.0	0.0367	0.0911	13.19	1
289	47.0	1.79	1411.4093959731542	314.0	42.5	560.9	0.038	0.0758	11.87	1
290	47.3	1.79	1419.4630872483222	313.9	50.0	561.5	0.0378	0.0891	15.06	1
291	47.6	1.65	1427.5167785234898	314.0	47.4	517.3	0.0346	0.0916	11.06	1
292	47.9	1.54	1435.5704697986578	314.7	46.6	485.9	0.0323	0.096	11.9	1
293	48.1	1.53	1443.6241610738257	317.5	44.8	485.5	0.0318	0.0923	11.62	1
294	48.4	1.8	1451.6778523489932	315.2	41.2	567.0	0.0372	0.0727	13.19	1
295	48.7	1.86	1459.731543624161	315.1	44.4	586.6	0.0383	0.0756	9.53	1
296	48.9	1.63	1467.7852348993288	316.1	43.3	515.0	0.0333	0.084	11.59	1
297	49.2	1.94	1475.8389261744967	312.1	46.6	604.2	0.0394	0.077	14.2	1
298	49.5	1.79	1483.8926174496644	313.9	40.2	562.6	0.0362	0.0714	11.16	1
299	49.7	1.67	1491.9463087248323	316.0	50.9	526.1	0.0335	0.0968	7.6	1
300	50.0	1.88	1500.0	315.3	43.9	592.0	0.0375	0.0742	9.98	1
301	10.0	1.47	300.0	314.7	92.4	462.9	0.1471	0.0399	46.29	2
302	10.3	1.55	308.0536912751678	313.2	96.9	486.3	0.1512	0.0399	47.36	2
303	10.5	1.43	316.1073825503356	313.4	99.4	447.7	0.1356	0.0444	42.49	2
304	10.8	1.6	324.16107382550337	313.7	87.3	503.1	0.1484	0.0347	46.56	2
305	11.1	1.31	332.2147651006712	314.9	100.6	413.2	0.1185	0.0487	37.31	2
306	11.3	1.63	340.2684563758389	315.6	101.3	514.0	0.1436	0.0394	45.32	2

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
307	11.6	1.62	348.3221476510067	313.0	104.9	506.7	0.1394	0.0414	43.64	2
308	11.9	1.41	356.3758389261745	315.5	87.9	445.8	0.1189	0.0394	37.53	2
309	12.1	1.52	364.4295302013423	314.7	94.1	479.4	0.1254	0.0393	39.46	2
310	12.4	1.49	372.4832214765101	318.5	101.7	473.0	0.1196	0.043	38.1	2
311	12.7	1.68	380.53691275167785	316.6	91.9	532.1	0.1325	0.0345	41.95	2
312	13.0	1.4	388.59060402684565	313.1	92.0	438.8	0.1082	0.0419	33.88	2
313	13.2	1.5	396.6442953020134	313.4	97.4	470.0	0.1134	0.0414	35.55	2
314	13.5	1.6	404.6979865771812	314.3	100.1	501.9	0.1184	0.0399	37.21	2
315	13.8	1.63	412.75167785234896	314.5	103.7	512.8	0.1185	0.0404	37.27	2
316	14.0	1.5	420.8053691275168	317.6	100.1	477.3	0.1071	0.0419	34.03	2
317	14.3	1.59	428.8590604026846	315.8	92.5	502.4	0.1113	0.0368	35.14	2
318	14.6	1.39	436.9127516778523	312.9	91.7	436.1	0.0957	0.0421	29.94	2
319	14.8	1.38	444.9664429530201	315.8	88.6	436.1	0.0931	0.0406	29.4	2
320	15.1	1.36	453.0201342281879	311.1	99.9	423.0	0.09	0.0472	28.01	2
321	15.4	1.48	461.0738255033557	317.2	100.0	468.2	0.096	0.0427	30.46	2
322	15.6	1.38	469.1275167785235	313.2	96.0	430.7	0.0879	0.0446	27.54	2
323	15.9	1.49	477.1812080536913	315.6	96.9	470.5	0.0937	0.0412	29.58	2
324	16.2	1.45	485.2348993288591	316.7	91.0	459.0	0.0896	0.0397	28.38	2
325	16.4	1.4	493.2885906040268	316.6	102.7	443.6	0.0852	0.0463	26.98	2
326	16.7	1.64	501.3422818791947	314.4	95.3	516.6	0.0983	0.0369	30.91	2
327	17.0	1.3	509.3959731543624	315.9	88.2	411.9	0.0768	0.0428	24.26	2
328	17.2	1.41	517.4496644295302	313.4	102.6	443.4	0.082	0.0463	25.71	2
329	17.5	1.62	525.5033557046979	315.4	96.5	512.0	0.0927	0.0377	29.23	2
330	17.8	1.49	533.5570469798657	314.5	91.9	469.0	0.0838	0.0392	26.37	2
331	18.1	1.47	541.6107382550335	315.9	98.3	463.7	0.0813	0.0424	25.68	2
332	18.3	1.46	549.6644295302012	315.5	96.0	459.9	0.0796	0.0417	25.1	2
333	18.6	1.47	557.7181208053692	316.1	95.4	464.4	0.079	0.0411	24.98	2
334	18.9	1.52	565.7718120805368	317.1	88.4	483.1	0.0808	0.0366	25.62	2
335	19.1	1.48	573.8255033557047	312.2	88.7	462.7	0.0775	0.0383	24.19	2
336	19.4	1.41	581.8791946308725	316.9	86.1	448.2	0.0729	0.0384	23.11	2
337	19.7	1.4	589.9328859060403	316.6	100.9	444.6	0.0714	0.0454	22.61	2
338	19.9	1.49	597.9865771812081	314.9	92.2	470.0	0.0749	0.0392	23.58	2
339	20.2	1.37	606.0402684563758	312.8	104.5	427.2	0.0676	0.0489	21.15	2
340	20.5	1.57	614.0939597315437	312.5	104.8	491.9	0.0769	0.0426	24.03	2

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
341	20.7	1.34	622.1476510067114	311.7	98.0	418.8	0.0648	0.0468	20.19	2
342	21.0	1.36	630.2013422818792	311.3	97.8	422.9	0.0647	0.0463	20.13	2
343	21.3	1.48	638.255033557047	314.7	92.2	466.1	0.0696	0.0396	21.91	2
344	21.5	1.48	646.3087248322148	318.0	95.8	472.1	0.0689	0.0406	21.91	2
345	21.8	1.68	654.3624161073826	315.8	87.3	531.7	0.0772	0.0328	24.38	2
346	22.1	1.56	662.4161073825503	316.7	89.6	495.2	0.0708	0.0362	22.43	2
347	22.3	1.4	670.4697986577182	313.9	88.6	439.1	0.0626	0.0404	19.65	2
348	22.6	1.55	678.5234899328859	316.0	93.0	490.0	0.0686	0.038	21.66	2
349	22.9	1.42	686.5771812080536	317.6	91.9	449.9	0.0619	0.0409	19.66	2
350	23.2	1.57	694.6308724832214	315.8	89.0	494.4	0.0676	0.036	21.35	2
351	23.4	1.51	702.6845637583892	318.1	94.8	481.6	0.0646	0.0394	20.56	2
352	23.7	1.35	710.738255033557	316.1	91.2	427.0	0.057	0.0427	18.02	2
353	24.0	1.46	718.7919463087247	314.7	96.1	458.7	0.0608	0.0419	19.14	2
354	24.2	1.42	726.8456375838927	317.2	91.8	448.8	0.0584	0.0409	18.52	2
355	24.5	1.59	734.8993288590603	317.8	86.5	506.2	0.065	0.0342	20.66	2
356	24.8	1.5	742.9530201342283	317.7	103.6	475.2	0.0604	0.0436	19.19	2
357	25.0	1.44	751.006711409396	313.1	103.8	451.0	0.0575	0.046	18.02	2
358	25.3	1.61	759.0604026845638	314.4	101.1	505.9	0.0636	0.04	19.99	2
359	25.6	1.6	767.1140939597316	319.8	94.4	512.4	0.0627	0.0368	20.04	2
360	25.8	1.51	775.1677852348993	312.0	102.0	472.2	0.0586	0.0432	18.27	2
361	26.1	1.52	783.2214765100672	317.1	90.9	480.8	0.0581	0.0378	18.42	2
362	26.4	1.57	791.2751677852349	311.7	101.3	489.9	0.0596	0.0414	18.57	2
363	26.6	1.5	799.3288590604027	316.3	91.0	475.3	0.0564	0.0383	17.84	2
364	26.9	1.64	807.3825503355705	315.0	91.9	515.3	0.0608	0.0357	19.15	2
365	27.2	1.45	815.4362416107383	313.4	89.4	454.3	0.0533	0.0394	16.71	2
366	27.4	1.59	823.489932885906	314.8	89.2	499.7	0.0578	0.0357	18.2	2
367	27.7	1.46	831.5436241610738	315.6	89.6	459.4	0.0525	0.039	16.57	2
368	28.0	1.38	839.5973154362416	312.5	101.7	432.5	0.0495	0.047	15.45	2
369	28.3	1.37	847.6510067114093	314.3	99.1	429.9	0.0484	0.0461	15.21	2
370	28.5	1.4	855.7046979865771	315.6	102.3	443.3	0.0492	0.0462	15.54	2
371	28.8	1.59	863.7583892617449	314.0	93.8	499.8	0.0553	0.0375	17.36	2
372	29.1	1.5	871.8120805369127	315.0	90.9	472.2	0.0516	0.0385	16.25	2
373	29.3	1.41	879.8657718120805	315.0	101.9	445.4	0.0482	0.0458	15.19	2
374	29.6	1.64	887.9194630872483	313.1	101.0	512.3	0.0553	0.0394	17.31	2

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
375	29.9	1.5	895.9731543624162	316.5	87.0	474.2	0.0502	0.0367	15.88	2
376	30.1	1.25	904.0268456375838	314.6	100.7	393.7	0.0415	0.0512	13.06	2
377	30.4	1.6	912.0805369127517	312.7	101.3	499.4	0.0525	0.0406	16.43	2
378	30.7	1.48	920.1342281879196	316.7	96.3	468.9	0.0483	0.0411	15.29	2
379	30.9	1.58	928.1879194630872	316.8	98.9	499.4	0.051	0.0396	16.14	2
380	31.2	1.52	936.2416107382552	313.5	98.9	476.9	0.0487	0.0415	15.28	2
381	31.5	1.47	944.2953020134228	312.8	88.8	458.2	0.0465	0.0388	14.56	2
382	31.7	1.35	952.3489932885908	314.0	87.8	424.5	0.0426	0.0414	13.37	2
383	32.0	1.62	960.4026845637584	313.8	91.0	507.3	0.0505	0.0359	15.85	2
384	32.3	1.51	968.4563758389264	314.7	99.5	473.8	0.0466	0.042	14.68	2
385	32.6	1.66	976.510067114094	315.7	91.2	524.3	0.051	0.0348	16.11	2
386	32.8	1.46	984.5637583892616	313.5	94.0	458.8	0.0446	0.041	13.98	2
387	33.1	1.59	992.6174496644296	317.1	99.0	504.4	0.0481	0.0393	15.24	2
388	33.4	1.57	1000.6711409395974	318.0	93.9	497.9	0.0469	0.0377	14.93	2
389	33.6	1.55	1008.7248322147652	315.2	104.3	487.6	0.046	0.0428	14.5	2
390	33.9	1.54	1016.7785234899328	316.1	103.2	487.7	0.0455	0.0423	14.39	2
391	34.2	1.59	1024.8322147651006	312.7	95.8	497.8	0.0466	0.0385	14.57	2
392	34.4	1.4	1032.8859060402683	317.0	88.4	444.5	0.0407	0.0398	12.91	2
393	34.7	1.62	1040.9395973154365	318.0	98.6	516.4	0.0468	0.0382	14.88	2
394	35.0	1.65	1048.9932885906042	315.7	100.1	521.2	0.0472	0.0384	14.91	2
395	35.2	1.47	1057.0469798657718	316.2	91.8	463.3	0.0416	0.0396	13.15	2
396	35.5	1.42	1065.1006711409395	315.3	99.4	447.0	0.0399	0.0445	12.59	2
397	35.8	1.46	1073.1543624161072	312.5	92.8	456.3	0.0408	0.0407	12.76	2
398	36.0	1.53	1081.2080536912752	316.4	99.1	483.0	0.0424	0.041	13.4	2
399	36.3	1.26	1089.2617449664428	313.5	102.5	394.3	0.0346	0.052	10.86	2
400	36.6	1.5	1097.3154362416103	315.1	103.4	473.7	0.0411	0.0437	12.95	2
401	36.8	1.52	1105.3691275167785	315.1	101.9	478.5	0.0412	0.0426	12.99	2
402	37.1	1.53	1113.4228187919464	314.0	91.6	479.0	0.0411	0.0382	12.91	2
403	37.4	1.46	1121.476510067114	310.4	97.5	454.4	0.0392	0.0429	12.16	2
404	37.7	1.39	1129.5302013422818	315.4	93.2	437.5	0.0368	0.0426	11.62	2
405	37.9	1.37	1137.5838926174497	314.2	93.8	429.5	0.0361	0.0437	11.33	2
406	38.2	1.72	1145.6375838926174	317.0	89.8	546.7	0.0452	0.0329	14.32	2
407	38.5	1.63	1153.6912751677853	317.2	97.4	517.6	0.0424	0.0376	13.46	2
408	38.7	1.5	1161.744966442953	314.7	92.3	471.3	0.0387	0.0392	12.17	2

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
409	39.0	1.31	1169.7986577181207	314.3	96.6	412.7	0.0337	0.0468	10.58	2
410	39.3	1.46	1177.8523489932886	314.4	93.5	458.8	0.0372	0.0408	11.69	2
411	39.5	1.46	1185.9060402684563	313.8	94.4	459.6	0.0371	0.0411	11.63	2
412	39.8	1.32	1193.9597315436242	312.4	103.6	412.9	0.0332	0.0502	10.37	2
413	40.1	1.45	1202.013422818792	313.7	86.1	453.9	0.0361	0.0379	11.33	2
414	40.3	1.57	1210.0671140939596	317.1	89.6	497.7	0.0389	0.036	12.34	2
415	40.6	1.42	1218.1288053691275	317.0	92.2	449.6	0.0349	0.041	11.07	2
416	40.9	1.49	1226.1744966442957	315.2	94.4	468.7	0.0364	0.0403	11.47	2
417	41.1	1.57	1234.2281879194632	316.5	101.7	498.0	0.0382	0.0408	12.1	2
418	41.4	1.51	1242.2818791946308	314.2	101.0	474.0	0.0364	0.0426	11.45	2
419	41.7	1.46	1250.3355704697983	315.7	98.8	461.9	0.0351	0.0428	11.08	2
420	41.9	1.62	1258.3892617449665	314.5	86.8	508.7	0.0386	0.0341	12.13	2
421	42.2	1.58	1266.4429530201344	313.1	91.2	495.9	0.0375	0.0368	11.75	2
422	42.5	1.45	1274.496644295302	313.9	93.4	454.9	0.0341	0.0411	10.71	2
423	42.8	1.41	1282.5503355704698	312.5	91.9	441.8	0.0331	0.0416	10.33	2
424	43.0	1.42	1290.6040268456377	319.9	104.5	455.4	0.0331	0.0459	10.59	2
425	43.3	1.57	1298.6577181208054	315.5	99.8	493.9	0.0362	0.0404	11.41	2
426	43.6	1.6	1306.7114093959733	313.5	93.7	500.0	0.0366	0.0375	11.48	2
427	43.8	1.63	1314.765100671141	313.5	87.2	510.2	0.0371	0.0342	11.64	2
428	44.1	1.3	1322.818791946309	315.5	92.5	410.0	0.0295	0.0451	9.3	2
429	44.4	1.53	1330.8724832214766	316.5	95.9	483.3	0.0344	0.0397	10.89	2
430	44.6	1.48	1338.9261744966443	313.9	87.3	465.2	0.0332	0.0375	10.42	2
431	44.9	1.57	1346.979865771812	315.1	89.2	495.8	0.035	0.036	11.04	2
432	45.2	1.59	1355.0335570469797	316.3	90.0	501.9	0.0351	0.0359	11.11	2
433	45.4	1.38	1363.0872483221476	314.3	103.7	433.3	0.0303	0.0479	9.54	2
434	45.7	1.46	1371.1409395973153	311.3	98.9	454.8	0.032	0.0435	9.95	2
435	46.0	1.48	1379.1946308724832	317.3	100.9	470.0	0.0322	0.0429	10.22	2
436	46.2	1.69	1387.248322147651	312.5	86.9	527.7	0.0365	0.0329	11.41	2
437	46.5	1.5	1395.3020134228186	314.1	86.6	471.4	0.0323	0.0367	10.14	2
438	46.8	1.47	1403.3557046979863	315.6	101.3	465.4	0.0315	0.0435	9.95	2
439	47.0	1.4	1411.4093959731542	315.9	92.3	441.0	0.0297	0.0419	9.37	2
440	47.3	1.42	1419.4630872483222	313.7	92.5	446.5	0.0301	0.0414	9.44	2
441	47.6	1.64	1427.5167785234898	311.8	98.3	511.1	0.0344	0.0385	10.74	2
442	47.9	1.54	1435.5704697986578	311.0	86.8	479.1	0.0322	0.0362	10.01	2

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
443	48.1	1.43	1443.6241610738257	310.9	98.9	445.3	0.0298	0.0444	9.25	2
444	48.4	1.59	1451.6778523489932	313.4	86.2	499.3	0.0329	0.0345	10.32	2
445	48.7	1.61	1459.731543624161	312.4	96.8	501.5	0.033	0.0386	10.31	2
446	48.9	1.5	1467.7852348993288	313.5	104.9	469.7	0.0306	0.0447	9.6	2
447	49.2	1.4	1475.8389261744967	318.2	96.8	444.7	0.0284	0.0435	9.04	2
448	49.5	1.43	1483.8926174496644	314.1	90.7	448.0	0.0288	0.0405	9.06	2
449	49.7	1.47	1491.9463087248323	315.3	89.7	462.0	0.0295	0.0388	9.29	2
450	50.0	1.51	1500.0	315.0	99.3	474.6	0.0301	0.0418	9.49	2
451	10.0	1.66	321.6	315.9	52.7	524.5	0.166	0.1005	52.45	3
452	10.3	1.56	312.4	313.7	47.8	489.4	0.1519	0.0976	47.66	3
453	10.5	4.14	297.5	316.6	36.8	1310.6	0.3929	0.0281	124.38	3
454	10.8	4.05	324.4	313.6	43.2	1270.0	0.3748	0.034	117.53	3
455	11.1	1.4	364.6	317.8	42.5	444.9	0.1264	0.0955	40.18	3
456	11.3	1.86	319.8	312.5	42.6	581.3	0.164	0.0733	51.25	3
457	11.6	1.54	314.1	312.2	37.0	480.7	0.1326	0.0771	41.4	3
458	11.9	1.39	374.5	316.3	44.8	439.6	0.117	0.1019	37.01	3
459	12.1	1.56	357.7	313.7	45.3	489.3	0.1284	0.0925	40.28	3
460	12.4	1.46	409.1	312.0	41.6	455.5	0.1176	0.0914	36.69	3
461	12.7	1.47	361.2	313.8	48.6	461.3	0.1159	0.1054	36.37	3
462	13.0	1.41	389.5	319.6	50.3	450.6	0.1089	0.1117	34.79	3
463	13.2	1.46	361.9	314.6	49.0	459.4	0.1104	0.1067	34.75	3
464	13.5	1.52	430.0	313.9	44.0	477.1	0.1127	0.0922	35.37	3
465	13.8	1.53	421.4	312.2	49.7	477.6	0.1112	0.104	34.71	3
466	14.0	1.61	410.4	315.4	55.2	507.9	0.1148	0.1087	36.21	3
467	14.3	4.1	436.6	313.8	32.4	1286.4	0.2868	0.0252	89.99	3
468	14.6	1.46	422.2	316.2	48.3	461.7	0.1002	0.1046	31.7	3
469	14.8	1.52	465.9	311.5	47.0	473.5	0.1025	0.0992	31.92	3
470	15.1	1.51	497.3	313.4	43.4	473.2	0.1	0.0916	31.34	3
471	15.4	1.82	452.3	315.2	43.3	573.6	0.1184	0.0755	37.32	3
472	15.6	1.73	455.6	315.0	39.4	545.0	0.1106	0.0723	34.85	3
473	15.9	3.93	498.2	316.1	49.1	1242.4	0.2471	0.0395	78.11	3
474	16.2	1.53	520.1	316.6	54.6	484.4	0.0946	0.1126	29.95	3
475	16.4	1.55	445.0	315.0	44.2	488.3	0.0943	0.0906	29.7	3
476	16.7	1.55	513.0	312.7	39.0	484.7	0.0928	0.0805	29.0	3

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
477	17.0	1.58	552.9	318.9	45.5	503.8	0.0931	0.0902	29.67	3
478	17.2	1.35	526.7	315.6	34.0	426.1	0.0783	0.0798	24.7	3
479	17.5	1.6	532.0	314.4	42.6	503.1	0.0913	0.0847	28.72	3
480	17.8	1.63	487.9	312.6	50.5	509.5	0.0916	0.0991	28.65	3
481	18.1	1.54	533.0	315.5	37.5	485.9	0.0853	0.0773	26.91	3
482	18.3	1.49	502.0	316.5	43.1	471.6	0.0813	0.0915	25.74	3
483	18.6	1.58	598.2	316.7	43.1	500.3	0.085	0.0862	26.91	3
484	18.9	1.46	576.9	313.4	48.7	457.6	0.0774	0.1063	24.26	3
485	19.1	1.36	611.4	313.4	48.8	426.2	0.0711	0.1145	22.28	3
486	19.4	1.49	628.6	314.8	45.9	469.1	0.0768	0.0977	24.19	3
487	19.7	1.48	615.4	315.3	49.2	466.7	0.0753	0.1055	23.73	3
488	19.9	1.48	584.2	313.1	43.1	463.5	0.0742	0.0931	23.25	3
489	20.2	1.81	631.8	313.8	43.4	568.0	0.0896	0.0764	28.12	3
490	20.5	1.52	594.6	314.5	45.9	478.1	0.0743	0.0961	23.36	3
491	20.7	1.47	625.5	316.0	43.3	464.5	0.0709	0.0933	22.4	3
492	21.0	4.05	603.8	311.8	40.5	1262.6	0.1928	0.0321	60.1	3
493	21.3	1.62	599.9	317.1	42.5	513.7	0.0761	0.0827	24.15	3
494	21.5	1.48	647.8	314.9	38.1	466.0	0.0687	0.0817	21.63	3
495	21.8	1.45	615.1	314.2	50.0	455.5	0.0665	0.1098	20.88	3
496	22.1	1.55	699.2	317.0	54.2	491.4	0.0702	0.1102	22.25	3
497	22.3	4.12	612.6	316.6	41.2	1304.2	0.1843	0.0316	58.36	3
498	22.6	1.67	672.6	316.7	43.7	528.8	0.0738	0.0827	23.38	3
499	22.9	1.66	720.4	314.0	46.1	521.2	0.0725	0.0885	22.77	3
500	23.2	1.48	652.5	316.9	44.7	469.0	0.0639	0.0953	20.26	3
501	23.4	1.41	633.6	314.9	40.6	444.0	0.0602	0.0915	18.96	3
502	23.7	1.42	711.7	312.2	51.0	443.3	0.0599	0.115	18.71	3
503	24.0	1.49	647.0	315.7	45.2	470.4	0.0622	0.096	19.63	3
504	24.2	1.4	796.7	313.4	48.3	438.8	0.0578	0.1102	18.11	3
505	24.5	3.98	719.2	316.2	35.5	1258.3	0.1625	0.0282	51.37	3
506	24.8	3.87	695.1	314.1	49.6	1215.5	0.1563	0.0408	49.08	3
507	25.0	3.99	801.6	318.9	54.1	1272.4	0.1594	0.0425	50.83	3
508	25.3	1.46	745.2	317.5	40.7	463.5	0.0577	0.0879	18.32	3
509	25.6	1.48	778.8	314.9	45.8	466.1	0.0579	0.0984	18.23	3
510	25.8	3.91	822.4	315.5	53.9	1233.4	0.1513	0.0437	47.73	3

	Frequency(Hz)	Current(A)	RPM	Voltage(V)	VFD_Temp(C)	ApparentPower(VA)	LoadFriction(A/Hz)	CoolingEfficiency(C/VA)	TorqueEst	State_Label
511	26.1	1.52	817.0	313.2	54.2	476.1	0.0582	0.1138	18.24	3
512	26.4	1.42	864.3	314.9	43.8	447.2	0.0538	0.098	16.95	3
513	26.6	1.47	770.1	315.2	37.4	463.3	0.0552	0.0808	17.39	3
514	26.9	3.91	829.5	314.8	51.8	1231.0	0.1453	0.042	45.74	3
515	27.2	1.47	895.4	315.0	54.7	463.0	0.0541	0.1182	17.03	3
516	27.4	1.43	756.8	314.0	47.2	449.1	0.0521	0.105	16.36	3
517	27.7	1.43	890.2	315.1	44.4	450.6	0.0516	0.0986	16.26	3
518	28.0	4.05	854.3	315.1	49.1	1276.3	0.1447	0.0385	45.6	3
519	28.3	1.61	792.3	312.5	42.7	503.1	0.057	0.0848	17.81	3
520	28.5	1.49	824.9	315.6	46.2	470.3	0.0522	0.0983	16.49	3
521	28.8	3.99	883.2	315.5	47.3	1258.9	0.1386	0.0376	43.72	3
522	29.1	1.3	807.3	312.6	52.9	406.4	0.0447	0.1301	13.98	3
523	29.3	1.42	919.2	314.7	33.6	446.8	0.0484	0.0751	15.23	3
524	29.6	3.87	812.4	313.4	41.5	1212.8	0.1308	0.0342	40.98	3
525	29.9	1.54	901.4	312.8	44.7	481.8	0.0516	0.0927	16.13	3
526	30.1	1.55	981.8	315.3	39.7	488.8	0.0514	0.0812	16.22	3
527	30.4	1.44	860.1	313.4	55.0	451.2	0.0474	0.1218	14.84	3
528	30.7	1.54	833.3	318.4	50.9	490.4	0.0502	0.1037	15.99	3
529	30.9	1.61	999.9	314.3	44.1	506.0	0.052	0.0871	16.35	3
530	31.2	1.56	952.2	317.4	44.0	495.1	0.05	0.0888	15.86	3
531	31.5	1.56	950.1	312.9	46.2	488.1	0.0496	0.0946	15.51	3
532	31.7	1.57	952.7	313.9	52.3	492.9	0.0495	0.106	15.53	3
533	32.0	1.42	901.3	318.2	44.5	451.9	0.0444	0.0984	14.12	3
534	32.3	1.72	1017.1	317.9	41.9	546.8	0.0533	0.0767	16.94	3
535	32.6	1.64	926.3	316.8	40.0	519.5	0.0504	0.077	15.96	3
536	32.8	1.37	1069.6	318.2	43.5	435.9	0.0417	0.0998	13.28	3
537	33.1	1.55	916.5	314.4	47.4	487.3	0.0468	0.0973	14.73	3
538	33.4	1.55	1021.6	312.5	50.9	484.4	0.0465	0.105	14.52	3
539	33.6	1.39	1035.0	314.5	45.5	437.1	0.0413	0.104	13.0	3
540	33.9	1.38	963.0	315.1	46.5	434.9	0.0407	0.107	12.83	3
541	34.2	1.53	1009.0	316.5	48.8	484.2	0.0448	0.1007	14.17	3
542	34.4	1.52	978.1	314.8	44.6	478.5	0.0441	0.0931	13.9	3
543	34.7	4.15	940.0	311.7	47.2	1293.6	0.1196	0.0365	37.28	3
544	35.0	1.65	981.1	309.8	52.8	511.2	0.0472	0.1032	14.62	3

10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **127 Not Cited or Quoted 10%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **3 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 8%  Internet sources
- 6%  Publications
- 7%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.